

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y
TELECOMUNICACIONES



“Herramienta CASE Colaborativa con Inteligencia Artificial ”

INTEGRANTES :

- Lopez Velazquez Marco Alejandro 222008891

Materia: Ingeniería de Software 1 (INF-422) SC

Docente: Ing. Rolando Antonio Martínez Canedo

Gestión: Semestre 2 - 2026

Santa Cruz - Bolivia

PARTE I - PERFIL DEL PROYECTO.....	8
1.1) Introducción.....	8
1.2) Objetivo General.....	8
1.3) Objetivos Específicos.....	9
1.4) Descripción del problema.....	10
1.5) Alcance del Sistema.....	11
1.5.1) Subsistema 1: Seguridad, Identidad y Control de Acceso (RBAC).....	11
1.5.2) Subsistema 2: Gestión de Espacios de Trabajo, Auditoría Forense y Trazabilidad.....	11
1.5.3) Subsistema 3: Modelador Visual Core y Certificación Formal UML.....	12
1.5.4) Subsistema 4: Interoperabilidad e Ingeniería Dirigida por Modelos (MDD/CASE).....	12
1.5.5) Subsistema 5: Inteligencia Artificial Multimodal y Concurrencia en Tiempo Real	13
PARTE II - FUNDAMENTO TEÓRICO.....	14
2.1) Arquitectura de Software y Frameworks de Desarrollo.....	14
2.1.1) Spring Boot 4.1.0 y Arquitectura en 4 Capas Desacopladas.....	14
2.1.2) React Flow y Modelado Grafocéntrico Reactivo en TypeScript.....	14
2.1.3) WebSockets y Protocolo STOMP para Sincronización Concurrente.....	15
2.2) Ingeniería CASE, CBSE y Generación de Código Guiada por Modelos.....	15
2.2.1) Clasificación CASE (Upper, Lower, I-CASE) y MDE.....	15
2.2.2) Motor de Plantillas FreeMarker y Árboles AST.....	16
2.2.3) Interoperabilidad OMG XMI 2.1 con ArchiTec y StarUML.....	16
2.3) Arquitectura de Seguridad, Gobernanza y Gestión de Sesión.....	17
2.3.1) Gestión de Proyectos, Espacios de Trabajo y Exportación Documental.....	17
2.3.2) Sesión Volátil con sessionStorage y Evento beforeunload.....	17
2.3.3) Bitácora Inmutable de Auditoría y Trazabilidad.....	18
2.4) Inteligencia Artificial Multimodal, Resiliencia y SLMs Offline.....	18
2.4.1) Procesamiento de Lenguaje Natural (PLN) para Modelado Incremental.....	18
2.4.2) Visión Artificial para Digitalización Óptica de Pizarras.....	18
2.4.3) Cadena de Resiliencia Multi-IA (Circuit Breaker en 4 Niveles).....	18
2.4.4) Modelos de Lenguaje Reducidos On-Device para Aplicaciones Móviles Offline.	19
2.5) Metodología de Ingeniería de Software (PUDS y Taxonomía UML 2.5+).....	19
2.5.1) Proceso Unificado de Desarrollo de Software (PUDS).....	19
2.5.2) Taxonomía Canónica de Diagramas UML y Normalización Relacional.....	20
Taxonomía Oficial de Diagramas OMG UML 2.5+.....	20
Reglas de Transformación Objeto-Modelo y Formas Normales (1NF a 3NF).....	21
Parte III: Metodología de Desarrollo (PUDS).....	22
3.1) Flujo de Trabajo: Captura de Requisitos.....	22
3.1.1) Identificación de Actores.....	22
3.1.2) Requerimientos Funcionales (RF).....	23
3.1.3) Requerimientos No Funcionales (RNF).....	24
3.1.4) Paquetes de Casos de Uso del Sistema.....	25
Paquete 1: Seguridad, Identidad y Gobernanza.....	25

Paquete 2: Modelador Visual Core y Reglas de Negocio UML.....	26
Paquete 3: Interoperabilidad XMI y Motor Generador CASE.....	26
Paquete 4: Asistencia con Inteligencia Artificial y Concurrencia Colaborativa.....	26
Clasificación de Casos de Uso por Entorno de Aplicación.....	27
3.1.5) Priorizar Casos de Uso.....	28
Distribución por Ciclos de Desarrollo PUDS.....	29
Ciclo #1: Seguridad, Workspaces, Modelador Core y Normalización Relacional.....	29
Ciclo #2: Interoperabilidad CASE y Generación Automatizada (MDD).....	29
Ciclo #3: Asistencia IA Multimodal y Concurrencia Colaborativa.....	29
3.1.6) Detallar Casos de Uso.....	30
Ciclo #1.....	30
CU01: Autenticación Stateless y Gestión de Cuentas.....	30
CU02: Control de Roles (RBAC) y Gobernanza.....	32
CU03: Gestión de Proyectos, Plantillas y Onboarding.....	34
CU04: Auditoría Forense Global y Trazabilidad del Modelo.....	36
CU05: Modelado Canónico de Clases y Relaciones UML 2.5.....	38
CU06: Certificación de Normalización Relacional (1NF a 3NF).....	40
Ciclo #2.....	42
CU07: Exportación Multiformato (XMI, PNG, PDF, Excel).....	42
CU08: Importación XMI 2.1 con Auto-Layout Jerárquico.....	44
CU09: Generación de Backend Spring Boot en 4 Capas (ZIP).....	45
CU10: Generación de Esquema DDL SQL y Suite Postman v2.1.....	47
Ciclo #3.....	49
CU11: Modelado Asistido por Dictado de Voz (PLN).....	49
CU12: Digitalización Óptica de Pizarras Físicas con Visión.....	51
CU13: Sincronización Colaborativa Concurrente WSS STOMP.....	52
3.1.7) Estructurar Modelo de Casos de Uso.....	54
Diagrama de Casos de Uso — Ciclo #1.....	54
Diagrama de Casos de Uso — Ciclo #2.....	55
Diagrama de Casos de Uso — Ciclo #3.....	56
3.1.8) Modelado de Procesos de Negocio (Diagramas de Actividad).....	57
Proceso 1: Flujo de Modelado Canónico, Certificación Relacional y Generación MDD.....	57
Proceso 2: Flujo de Asistencia con IA Multimodal.....	58
Proceso 3: Flujo de Co-Diseño Colaborativo Concurrente en Tiempo Real.....	59
3.2) Flujo de Trabajo: Análisis.....	60
3.2.1) Análisis de la arquitectura.....	60
3.2.1.1) Identificar paquetes.....	60
3.2.1.2) Relacionar paquetes.....	61
3.2.1.3) Vista de paquetes.....	63
3.2.2) Análisis de casos de uso.....	65
Diagrama de comunicación.....	65
CICLO 1.....	65

CU01 - Autenticación Stateless y Gestión de Cuentas.....	65
CU02 - Control de Roles (RBAC) y Gobernanza.....	65
CU03 - Gestión de Proyectos, Plantillas y Onboarding.....	65
CU04 - Auditoría Forense Global y Trazabilidad del Modelo.....	65
CU05 - Modelado Canónico de Clases y Relaciones UML 2.5.....	66
CU06 - Certificación de Normalización Relacional (1NF a 3NF).....	66
CICLO 2.....	67
CU07 - Exportación Multifomato (XMI, PNG, PDF, Excel).....	67
CU08 - Importación XMI 2.1 con Auto-Layout Jerárquico.....	67
CU09 - Generación de Backend Spring Boot en 4 Capas (ZIP).....	67
CU10 - Generación de Esquema DDL SQL y Suite Postman v2.1.....	67
CICLO 3.....	68
CU11 - Modelado Asistido por Dictado de Voz (PLN).....	68
CU12 - Digitalización Óptica de Pizarras Físicas con Visión.....	68
CU13 - Sincronización Colaborativa Concurrente WSS STOMP.....	68
3.2.3) Análisis de clase.....	69
CICLO 1.....	69
CU01 - Autenticación Stateless y Gestión de Cuentas.....	69
CU02 - Control de Roles (RBAC) y Gobernanza.....	69
CU03 - Gestión de Proyectos, Plantillas y Onboarding.....	69
CU04 - Auditoría Forense Global y Trazabilidad del Modelo.....	70
CU05 - Modelado Canónico de Clases y Relaciones UML 2.5.....	70
CU06 - Certificación de Normalización Relacional (1NF a 3NF).....	70
CICLO 2.....	71
CU07 - Exportación Multifomato (XMI, PNG, PDF, Excel).....	71
CU08 - Importación XMI 2.1 con Auto-Layout Jerárquico.....	71
CU09 - Generación de Backend Spring Boot en 4 Capas (ZIP).....	71
CU10 - Generación de Esquema DDL SQL y Suite Postman v2.1.....	71
CICLO 3.....	72
CU11 - Modelado Asistido por Dictado de Voz (PLN).....	72
CU12 - Digitalización Óptica de Pizarras Físicas con Visión.....	72
CU13 - Sincronización Colaborativa Concurrente WSS STOMP.....	72
3.2.4) Analizar paquete.....	73
3.2.5) Análisis de Ciclo de Vida (Diagrama de Máquina de Estados).....	74
Estado 1 - Ciclo de Vida del Proyecto UML.....	74
Estado 2 - Ciclo de Vida de la Sesión Colaborativa Concurrente.....	74
Estado 3 - Ciclo de Vida de la Certificación de Normalización Relacional.....	74
Estado 4 - Ciclo de Vida de la Sesión y Token JWT Volátil.....	75
Estado 5 - Ciclo de Vida del Nodo de Clase en el Canvas.....	75
3.3) Flujo de Trabajo: Diseño.....	76
3.3.1) Diseño de la Arquitectura.....	76
3.3.1.1) Diseño de arquitectura física - Diagrama de despliegue.....	76
3.3.1.2) Diseño de arquitectura lógica - Diagrama de paquete organizado en capas.....	77

3.3.1.3) Diagrama de arquitectura de red.....	77
3.3.1.4) Orquestación General (Diagrama Global de Interacciones).....	78
3.3.1.5) Diseño Interno de Microservicios (Diagrama de Estructura Compuesta)...	79
3.3.2) Diseño de caso de uso.....	80
Diagrama de Secuencia.....	80
CICLO 1: Núcleo Arquitectónico, Workspaces, Modelador Core y Normalización.....	80
CU01 - Autenticación Stateless y Gestión de Cuentas.....	80
CU02 - Control de Roles (RBAC) y Gobernanza.....	81
CU03 - Gestión de Proyectos, Plantillas y Onboarding.....	82
CU04 - Auditoría Forense Global y Trazabilidad del Modelo.....	83
CU05 - Modelado Canónico de Clases y Relaciones UML 2.5.....	84
CU06 - Certificación de Normalización Relacional (1NF a 3NF).....	85
CICLO 2: Interoperabilidad XMI y Generación MDA de Código.....	86
CU07 - Exportación Multifomato (XMI, PNG, PDF, Excel).....	86
CU08 - Importación XMI 2.1 con Auto-Layout Jerárquico.....	87
CU09 - Generación de Backend Spring Boot en 4 Capas (ZIP).....	88
CU10 - Generación de Esquema DDL SQL y Suite Postman v2.1.....	89
CICLO 3: Inteligencia Artificial Multimodal y Sincronización Concurrente.....	90
CU11 - Modelado Asistido por Dictado de Voz (PLN).....	90
CU12 - Digitalización Óptica de Pizarras Físicas con Visión.....	91
CU13 - Sincronización Colaborativa Concurrente WSS STOMP.....	92
3.3.2.1) Restricciones de Latencia y WebSockets (Diagrama de Tiempos).....	93
3.3.3) Diseño de la Base de Datos.....	94
3.3.3.1) Diseño Lógico.....	94
3.3.3.2) Mapeo.....	94
1. Entidad UserProfile ↔ Tabla user_profiles.....	94
2. Entidad DiagramProject ↔ Tabla diagram_projects.....	95
3. Entidad ClassNode ↔ Tabla class_nodes.....	95
4. Entidad Relationship ↔ Tabla relationships.....	96
5. Entidad DiagramHistory ↔ Tabla diagram_history.....	97
6. Entidad AuditLog ↔ Tabla audit_logs.....	98
7. Entidad DomainTemplate ↔ Tabla domain_templates.....	98
8. Entidad ProjectArtifact ↔ Tabla project_artifacts.....	99
9. Entidad AiPromptLog ↔ Tabla ai_prompt_logs.....	99
10. Entidad CollaborationSession ↔ Tabla collaboration_sessions.....	100
11. Entidad CollaborationParticipant ↔ Tabla collaboration_participants.....	100
12. Entidad ElementLock ↔ Tabla element_locks.....	101
13. Entidad CollaborationMessage ↔ Tabla collaboration_messages.....	101
14. Entidad NormalizationCertification ↔ Tabla normalization_certifications.....	102
3.3.3.3) DISEÑO FÍSICO (Tabla de Volumen).....	102
3.3.3.4) Script.....	104
3.4) Flujo de Trabajo: Implementación.....	111

3.4.1) Elección de la plataforma de desarrollo de software.....	111
3.4.1.1) Lenguaje de Programación Backend: Java 21 LTS + Spring Boot 4.1.0....	112
3.4.1.2) Lenguaje de Programación Frontend: React 18 + TypeScript + Vite....	112
3.4.1.3) Base de Datos: PostgreSQL 17 Cloud (Supabase) con Tipos JSONB.	112
3.4.1.4) Infraestructura y Despliegue en la Nube (AWS + Nginx):.....	112
3.4.2) Implementación de la arquitectura del software.....	113
3.4.3) Implementación de la arquitectura del subsistema.....	113
Diagrama de componentes de cada paquete.....	113
3.5) Flujo de Trabajo: Pruebas.....	115
3.5.1 Plan de pruebas.....	115
CICLO 1: Núcleo Arquitectónico, Workspaces, Modelador Core y Normalización..	115
Reporte de Prueba — CICLO 1.....	118
Prueba de CU01: Autenticación Stateless y Gestión de Cuentas.....	118
Prueba de CU02: Control de Roles (RBAC) y Gobernanza.....	120
Prueba de CU03: Gestión de Proyectos, Plantillas y Onboarding.....	122
Prueba de CU04: Auditoría Forense Global y Trazabilidad del Modelo.....	124
Prueba de CU05: Modelado Canónico de Clases y Relaciones UML 2.5....	126
Prueba de CU06: Certificación de Normalización Relacional (1NF a 3NF).	128
CICLO 2: Interoperabilidad CASE y Generación MDA de Código.....	130
Reporte de Prueba — CICLO 2.....	132
Prueba de CU07: Exportación Multiformato (XMI, PNG, PDF, Excel).....	132
Prueba de CU08: Importación XMI 2.1 con Auto-Layout Jerárquico.....	134
Prueba de CU09: Generación de Backend Spring Boot en 4 Capas (ZIP)..	136
Prueba de CU10: Generación de Esquema DDL SQL y Suite Postman v2.1....	138
CICLO 3: Inteligencia Artificial Multimodal y Sincronización Concurrente.....	140
Reporte de Prueba — CICLO 3.....	142
Prueba de CU11: Modelado Asistido por Dictado de Voz (PLN).....	142
Prueba de CU12: Digitalización Óptica de Pizarras Físicas con Visión.....	144
Prueba de CU13: Sincronización Colaborativa Concurrente WSS STOMP.	146
CONCLUSIÓN.....	148
1. Rigor Metodológico y Coherencia de Ingeniería (PUDS + CBD).....	148
2. Aceleración Radical del Desarrollo de Software (Paradigma MDD / CASE).....	148
3. Aseguramiento Preventivo de la Calidad de Datos (Reglas de Codd y Teoría TOM)	148
4. Interoperabilidad Abierta y Resistencia de Seguridad.....	149
5. Resiliencia y Alta Disponibilidad en IA Multimodal.....	149
6. Gobernanza, Concurrencia en Tiempo Real y Sesiones Seguras.....	149
7. Verificación Integral y Certificación Operativa.....	149
RECOMENDACIÓN.....	150
1. Ingeniería Inversa y Sincronización Bidireccional (Round-Trip Engineering).....	150
2. Expansión del Modelado UML a Diagramas de Comportamiento Dinámico.....	150
3. Desacoplamiento y Escalabilidad Horizontal del Broker WebSocket.....	150

4. Integración Continua de Modelos con Ecosistemas Git (GitOps para Modelos).	150
5. Generación de Pipelines CI/CD y Empaquetado en Contenedores.....	151
6. Compilación Nativa Ahead-of-Time (AOT) con GraalVM Native Image.....	151
7. Arquitectura de Plugins Extensible para Nuevos Stacks Tecnológicos.....	151
BIBLIOGRAFÍA.....	152
1. Estándares Internacionales y Especificaciones Formales.....	152
2. Metodología, Arquitectura y Fundamentos de Ingeniería de Software.....	152
3. Teoría de Bases de Datos y Normalización Relacional.....	153
4. Frameworks, Lenguajes y Ecosistemas de Implementación.....	153
5. Inteligencia Artificial, Visión Computacional y Modelos de Lenguaje.....	153
URL Y QR.....	154
URL A GITHUB.....	154
QR A GITHUB.....	154
URL EN LA NUBE.....	155
QR A LA NUBE.....	155
Manual de Usuario Oficial por Roles de Operación.....	156
1. Guía de Operación para Super Administrador (SUPER_ADMIN).....	156
2. Guía de Operación para Arquitecto de Software (ARQUITECTO).....	158
3. Guía de Operación para Colaborador (COLABORADOR).....	162
4. Guía de Atajos de Teclado y Controles Rápidos del Lienzo.....	163
ANEXOS TÉCNICOS FORMALES.....	164
Glosario de Términos Técnicos y Metodológicos.....	164

PARTE I - PERFIL DEL PROYECTO

1.1) Introducción

En el contexto actual de la industria del software, la demanda de soluciones empresariales complejas exige ciclos de desarrollo hiperacelerados sin sacrificar la calidad arquitectónica. Los equipos de ingeniería de software distribuidos geográficamente enfrentan una fricción constante durante las etapas iniciales de diseño: la desconexión crítica entre el modelado conceptual/lógico de datos y la implementación física del código fuente.

Tradicionalmente, los arquitectos de software utilizan herramientas de modelado visual aisladas (dibujadores de diagramas estáticos o herramientas Upper-CASE de escritorio monousuario) cuyos artefactos quedan desfasados u obsoletos en cuanto comienza la codificación manual. Para resolver este problema estructural, este proyecto concibe y construye una Plataforma I-CASE Colaborativa con Inteligencia Artificial Multimodal y Generación Automática de Software, diseñada específicamente para equipos de ingeniería, arquitectos de software y entornos académicos de alta exigencia metodológica.

La solución integra un lienzo de modelado UML 2.5+ reactivo y de alta precisión, un motor formal de auditoría y certificación de normalización relacional (1NF, 2NF y 3NF según las reglas de Codd y TOM), interoperabilidad bidireccional en el estándar internacional OMG XMI 2.1 con herramientas Upper-CASE de la industria (ArchiTec, Enterprise Architect, StarUML), asistencia inteligente multimodal por voz y visión artificial mediante una arquitectura de resiliencia Circuit Breaker en 4 niveles (Google Gemini Flash, Groq Llama 3.3, OpenRouter y Motor Heurístico Local Offline), y sincronización colaborativa multiusuario concurrente en tiempo real mediante WebSockets y subprotocolo STOMP. Asimismo, la plataforma implementa una arquitectura de gobernanza y control de acceso basado en roles (RBAC), administración integral del ciclo de vida de proyectos (CRUD, clonación profunda y versionado semántico) y una bitácora inmutable de auditoría forense para garantizar trazabilidad técnica absoluta.

1.2) Objetivo General

Desarrollar una plataforma web I-CASE (Integrated Computer-Aided Software Engineering) colaborativa en tiempo real, respaldada por Inteligencia Artificial Multimodal y un motor generador de código guiado por modelos (MDD), que permita a equipos de arquitectura e ingeniería de datos diseñar diagramas de clases OMG UML 2.5+ (con soporte de plantillas base para mitigar el síndrome de lienzo en blanco), certificar su normalización relacional (1NF, 2NF, 3NF), interoperar mediante el estándar OMG XMI 2.1 con herramientas de la industria como ArchiTec, y generar automáticamente proyectos backend completos en Spring Boot 4.1.0 / Java 21 (organizados en 4 capas limpias desacopladas en paquetes ZIP ejecutables con Maven Wrapper), esquemas relacionales PostgreSQL 17 (DDL con restricciones de integridad y claves foráneas en cascada), colecciones de pruebas automáticas en Postman v2.1 y exportación de documentación técnica en PNG de ultra alta definición, informes ejecutivos en PDF y diccionarios de datos estructurados en Microsoft Excel (.xlsx), garantizando gobernanza RBAC, auditoría inmutable y sesiones volátiles de alta seguridad en sessionStorage.

1.3) Objetivos Específicos

Para materializar el objetivo general, se plantean los siguientes objetivos específicos organizados en torno a los 5 subsistemas funcionales de la plataforma:

1. **Subsistema 1:** Seguridad, Identidad y Control de Acceso (RBAC):
 - Diseñar e implementar un mecanismo de autenticación stateless basado en Tokens JWT Bearer con almacenamiento exclusivo en sessionStorage y listeners defensivos de ventana para garantizar sesiones volátiles que se destruyen al cerrar el navegador.
 - Construir el módulo de gobernanza y control de acceso basado en roles (RBAC) con separación estricta de privilegios entre Super Administrador, Arquitecto de Software y Colaborador, protegiendo las contraseñas mediante hashing criptográfico BCrypt (cost factor 12).
 - Restringir los privilegios del Super Administrador para supervisión y reactivación de cuentas, prohibiendo la autodegradación o bloqueo accidental.
2. **Subsistema 2:** Gestión de Espacios de Trabajo, Auditoría Forense y Trazabilidad:
 - Desarrollar un módulo integral de gestión de proyectos y espacios de trabajo que soporte creación en blanco, metadatos enriquecidos, tags de dominio, versionado semántico (v1.0.0) y clonación profunda (Deep Copy) transaccional de grafos completos (clases y aristas con remapeo de identificadores).
 - Proveer un catálogo interactivo de plantillas de dominio preconfiguradas (Académico, Hospitalario, Facturación y Blanco) para scaffolding inmediato.
 - Implementar una bitácora inmutable de auditoría forense (audit_logs) y trazabilidad del proyecto (diagram_history) con papelera de reciclaje reversible y purga física definitiva controlada por el propietario del proyecto.
3. **Subsistema 3:** Modelador Visual Core y Certificación Formal UML:
 - Construir un lienzo de modelado UML reactivo sobre React Flow (@xyflow/react) y TypeScript, con soporte canónico OMG UML 2.5 (3 compartimentos, visibilidades +, -, #, ~, estereotipos, badges {PK}, catálogo Java 21 / PostgreSQL 17 y firmas de métodos tipadas).
 - Implementar trazado ortogonal y libre de relaciones estructurales (asociación dirigida, agregación, composición, generalización, realización y dependencia), con prevención de ciclos de herencia directos y transitivos vía DFS y handles magnéticos en los 4 extremos de cada nodo.
 - Desarrollar un motor heurístico de auditoría y certificación de normalización relacional que valide en tiempo de diseño 1NF (claves primarias obligatorias y atomicidad), 2NF (dependencias funcionales completas y FKs requeridas en relaciones 1..*) y 3NF (eliminación de dependencias transitivas y descomposición asociativa de relaciones *..*), emitiendo un certificado digital de calidad.

4. **Subsistema 4:** Interoperabilidad e Ingeniería Dirigida por Modelos (MDD/CASE):

- Construir un motor de compilación basado en plantillas Apache FreeMarker 2.3.33 para generar código fuente Java 21 / Spring Boot 4.1.0 estructurado en 4 capas limpias desacopladas (Controller, Service, Repository, Entity), empaquetado en un archivo .zip ejecutable con Maven Wrapper, Swagger OpenAPI y perfiles H2/PostgreSQL.
- Automatizar la generación de scripts DDL PostgreSQL 17 con tablas, secuencias IDENTITY, restricciones FOREIGN KEY ON DELETE CASCADE, índices B-Tree y comentarios explicativos.
- Generar colecciones de pruebas oficiales Postman v2.1.0 organizadas en carpetas por entidad con suite CRUD completa, scripts de aserción pm.test automáticos y generación contextual de datos mock sintéticos.
- Diseñar un parser e importador bidireccional XML/XMI OMG 2.1 con protección contra ataques XXE y motor de auto-layout jerárquico por capas, complementado con exportación gráfica PNG (1x, 2x, 3x), memoria técnica ejecutiva en PDF y libro Excel (.xlsx) de 3 hojas con tipos PostgreSQL y JPA.

5. **Subsistema 5:** Inteligencia Artificial Multimodal y Concurrency en Tiempo Real:

- Integrar procesamiento de lenguaje natural (PLN) mediante Web Speech API y modelos LLM para modelado incremental por dictado de voz sin destruir el grafo preexistente.
- Incorporar visión computacional para la digitalización óptica instantánea de fotografías de diagramas UML dibujados a mano alzada en pizarras físicas, con auto-layout no colisionante.
- Diseñar e implementar un patrón de resiliencia Circuit Breaker en 4 niveles de contingencia (Google Gemini 1.5 Flash → Groq Llama 3.3 70B → OpenRouter → Motor Heurístico Local Offline) para asegurar disponibilidad continua.
- Construir una infraestructura de sincronización colaborativa multiusuario en tiempo real sobre WebSockets y subprotocolo STOMP (< 50 ms latencia) con salas privadas, chat colaborativo con aislamiento de teclado, cursores remotos con transformación geométrica de viewport y candados optimistas de edición.

1.4) Descripción del problema

En el desarrollo de software contemporáneo, se identifican tres cuellos de botella críticos que impactan negativamente en los costos y tiempos de entrega de los proyectos de ingeniería:

1. **Desincronización entre el Modelado Conceptual y el Código Fuente:**

Las herramientas de diagramación genéricas o estáticas (Lucidchart, Draw.io, Visio) carecen de semántica formal de ingeniería de software; generan meras ilustraciones gráficas que quedan rápidamente obsoletas en cuanto los desarrolladores inician la programación manual. La codificación manual repetitiva de clases entidad @Entity, repositorios Spring Data JPA, servicios transaccionales y controladores REST consume entre el 30% y el 45% del tiempo inicial del proyecto, induciendo discrepancias graves de tipos de datos, omisión de restricciones de integridad

referencial y desajustes de arquitectura.

2. **Aislamiento, Rigidez y Obsolescencia de las Herramientas Upper-CASE Tradicionales:**

Entornos clásicos de modelado como ArchiTec o Enterprise Architect son aplicaciones de escritorio monousuario, pesadas y rígidas. Estas herramientas carecen de soporte nativo para colaboración concurrente en la nube en tiempo real, no cuentan con capacidades de asistencia inteligente mediante Inteligencia Artificial moderna, y presentan curvas de aprendizaje sumamente pronunciadas que desmotivan su adopción en equipos ágiles y entornos académicos dinámicos.

3. **Inexistencia de Validación de Normalización en Tiempo de Diseño:**

La ausencia de motores de auditoría en tiempo real que verifiquen las reglas de normalización de datos (1NF, 2NF y 3NF) según los postulados de Codd y TOM provoca que defectos estructurales críticos (claves foráneas faltantes en relaciones uno a muchos, relaciones muchos a muchos no descompuestas en tablas de unión, o claves primarias ausentes) pasen desapercibidos durante la etapa de modelado. Dichos errores solo se detectan tardíamente en fases avanzadas de desarrollo o en producción, elevando de forma exponencial los costos de refactorización de esquemas y migración de datos.

1.5) Alcance del Sistema

El sistema se delimita formalmente en 5 subsistemas funcionales que cubren desde la concepción del modelo hasta la generación de artefactos ejecutables:

1.5.1) Subsistema 1: Seguridad, Identidad y Control de Acceso (RBAC)

- **Autenticación Stateless Volátil (CU01):** Validación de credenciales de usuario con emisión de Tokens JWT Bearer. Almacenamiento seguro restringido a sessionStorage para garantizar el cierre de sesión ante el cierre de pestaña o ventana del navegador.
- **Registro de Usuarios y Perfiles (CU01):** Alta de nuevos ingenieros con asignación de rol base por defecto (ARQUITECTO), hashing de contraseñas con algoritmo BCrypt (factor de trabajo 12) y gestión de preferencias visuales del editor en formato JSONB.
- **Gobernanza RBAC Centralizada (CU02):** Consola administrativa /admin/users reservada para el Super Administrador, con capacidad de activar, suspender o cambiar roles de usuarios, aplicando políticas estrictas contra la autodegradación o bloqueo de cuentas maestras.

1.5.2) Subsistema 2: Gestión de Espacios de Trabajo, Auditoría Forense y Trazabilidad

- **Ciclo de Vida de Proyectos y Versionado Semántico (CU03):** Creación de proyectos desde cero, edición de metadatos (nombre, descripción, autor, versión v1.0.0, tags dinámicos) y filtrado reactivo en el Dashboard.

- **Clonación Profunda Transaccional (CU03):** Duplicación íntegra y profunda de diagramas UML (cloneProject), generando nuevas instancias físicas en base de datos para todas las clases y relaciones mediante mapeo de IDs en una única transacción atómica.
- **Plantillas de Dominio y Onboarding Rápido (CU03):** Catálogo de plantillas base de scaffolding rápido (Académico, Hospitalario, Facturación y Blanco) y tutorial guiado interactivo de inducción técnica completable en menos de 120 segundos.
- **Auditoría Forense y Trazabilidad Inmutable (CU04):** Registro inmutable de eventos de seguridad y gobernanza en audit_logs con IP de origen, timestamp y payload de datos. Timeline cronológico de mutaciones de diagramas en diagram_history con diffs antes/después, papelera de reciclaje con restauración reversible y purga física en cascada.

1.5.3) Subsistema 3: Modelador Visual Core y Certificación Formal UML

- **Modelado Canónico de Clases OMG UML 2.5 (CU05):** Creación de clases en 3 compartimentos estandarizados (encabezado con estereotipos y visibilidad, atributos tipados con catálogo Java 21/PostgreSQL 17 y badges {PK}, métodos con visibilidad y parámetros tipados). Validación de unicidad de nombres en tiempo real y clonación atómica de nodos con desplazamiento visual.
- **Conexión de Relaciones y Prevención de Herencia Circular (CU05):** Trazado de relaciones UML (asociación dirigida con flecha abierta, agregación con rombo hueco, composición con rombo relleno, generalización y realización con triángulo cerrado, y dependencia con línea discontinua). Restricciones de multiplicidad, inversión de dirección y detección de ciclos directos e indirectos de generalización mediante algoritmo DFS en backend y frontend.
- **Certificación de Normalización Relacional 1NF a 3NF (CU06):** Motor de auditoría lógica en tiempo de diseño que evalúa la presencia de llaves primarias y atomicidad (1NF), dependencias funcionales y claves foráneas en relaciones 1..* (2NF), y dependencias transitivas y relaciones muchos a muchos normalizadas en tablas asociativas (3NF). Cálculo de semáforo de calidad (0% a 100%) y emisión de certificado digital inmutable en audit_logs.

1.5.4) Subsistema 4: Interoperabilidad e Ingeniería Dirigida por Modelos (MDD/CASE)

- **Exportación Multiformato del Modelo (CU07):** Generación de especificaciones OMG XMI 2.1 compatibles con herramientas Upper-CASE (ArchiTec, Enterprise Architect, StarUML), renderizado de diagramas en PNG de alta fidelidad (1x, 2x, 3x), generación de Memoria Técnica ejecutiva en PDF con diagramas incrustados y diccionario de datos en Microsoft Excel (.xlsx) con especificación de tipos de datos JPA y PostgreSQL 17.
- **Importación XMI con Motor de Auto-Layout (CU08):** Parser bidireccional XML/XMI con protección contra inyecciones XXE, extracción de entidades, atributos, PKs y relaciones, integrando un motor de posicionamiento jerárquico no colisionante (LayoutEngineUtil) con modalidad dual (Nuevo Proyecto o Fusión en lienzo actual).
- **Generador de Backend Spring Boot en 4 Capas (CU09):** Motor de compilación basado en plantillas FreeMarker que produce un proyecto Maven completo

empaquetado en ZIP con código Java 21: Controllers REST, Services transaccionales, Repositories Spring Data JPA y Entities con anotaciones JPA/Hibernate, Maven Wrapper y Swagger UI preconfigurado.

- **Generador de Esquema DDL SQL y Suite Postman v2.1 (CU10):** Producción de scripts DDL PostgreSQL 17 con tablas, secuencias IDENTITY, claves foráneas con ON DELETE CASCADE, índices y comentarios de documentación. Generación complementaria de colecciones Postman v2.1.0 estructuradas por entidad con 5 endpoints CRUD predefinidos, validaciones automatizadas pm.test y captura de variables de entorno.

1.5.5) Subsistema 5: Inteligencia Artificial Multimodal y Concurrency en Tiempo Real

- **Modelado por Dictado de Voz (CU11):** Módulo de procesamiento de comandos hablados mediante Web Speech API y modelos de lenguaje, permitiendo la mutación incremental del grafo (adición de entidades, campos y relaciones) sin reinicializar el lienzo y con reversibilidad total (Ctrl+Z / Ctrl+Y).
- **Digitalización Óptica de Fotos de Pizarras (CU12):** Procesamiento de imágenes de diagramas UML dibujados a mano alzada mediante visión computacional, reconociendo clases, atributos, PKs y relaciones, con posicionamiento automático por capas y defensa de integridad relacional en 1NF.
- **Arquitectura de Resiliencia Circuit Breaker Multi-IA (CU11, CU12):** Conmutador automático de proveedores de inteligencia artificial en 4 niveles de contingencia: Nivel 1 (Google Gemini 1.5 Flash), Nivel 2 (Groq Llama 3.3 70B), Nivel 3 (OpenRouter) y Nivel 4 (Motor Heurístico Local Offline basado en reglas gramaticales y léxicas).
- **Sincronización Concurrente por WebSockets STOMP y Gobernanza de Sala (CU13):** Broker de mensajería bidireccional sobre SockJS para sesiones colaborativas en vivo (< 50 ms de latencia) entre usuarios registrados (Arquitectos y Colaboradores). Incluye expulsión inmediata de participantes por el Host (Kick), candados optimistas (element_locks) durante la edición de nodos, chat en vivo no invasivo con aislamiento de eventos de teclado y cursores remotos sincronizados.

PARTE II - FUNDAMENTO TEÓRICO

2.1) Arquitectura de Software y Frameworks de Desarrollo

2.1.1) Spring Boot 4.1.0 y Arquitectura en 4 Capas Desacopladas

Spring Boot es el marco de trabajo líder en el ecosistema Java empresarial para la construcción de microservicios y APIs REST de alto rendimiento y grado de producción. En este proyecto, tanto el backend de la plataforma como el código generado siguen de manera estricta el patrón de **Arquitectura en 4 Capas Desacopladas (Clean Layered Architecture)**:

1. **Capa de Controladores (Presentación / @RestController):**

Punto de entrada HTTP stateless. Expone los endpoints REST siguiendo las directrices de Richardson Maturity Model (Nivel 2/3), consume y produce payloads JSON, maneja validación de entrada mediante Bean Validation (@Valid, @NotNull) y delega de inmediato el procesamiento a la capa de servicios, retornando respuestas tipadas ResponseEntity<T> con códigos de estado canónicos (200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict, 500 Internal Server Error).

2. **Capa de Servicios (Lógica de Negocio / @Service):**

Encapsula las reglas de negocio puras, cálculos de dominio, orquestaciones transaccionales mediante @Transactional (con soporte para atomicidad y rollback automático ante excepciones RuntimeException), validaciones semánticas y transformaciones DTO-Entidad.

3. **Capa de Repositorios (Acceso a Datos / @Repository):**

Interfaces que extienden de JpaRepository<T, ID> provistas por Spring Data JPA. Abstraen el acceso a la base de datos relacional PostgreSQL 17 mediante Hibernate ORM, ofreciendo métodos CRUD optimizados y consultas derivadas o JPQL con pool de conexiones HikariCP.

4. **Capa de Entidades (Dominio / @Entity):**

Clases POJO que mapean las tablas físicas de la base de datos relacional mediante anotaciones Jakarta Persistence (@Entity, @Table, @Id, @GeneratedValue(strategy = GenerationType.IDENTITY), @Column, @ManyToOne, @OneToMany, @ManyToMany, @JoinColumn, @JoinTable).

2.1.2) React Flow y Modelado Grafocéntrico Reactivo en TypeScript

@xyflow/react (React Flow v12) es una biblioteca de ingeniería de software para la construcción de entornos basados en nodos, aristas y grafos interactivos en el navegador. En la plataforma, se aprovechan sus capacidades grafocéntricas para modelar clases UML como nodos personalizados (ClassNodeComponent) con compartimentos modulares:

- **Manejo de Estado Inmutable:** Integración con Zustand para una gestión de estado ultrarrápida fuera del árbol de renderizado de React, minimizando los ciclos de re-renderizado durante operaciones de arrastre (*drag & drop*) y redimensionamiento.
- **Handles Magnéticos Bidireccionales:** Implementación de puertos de conexión magnéticos en los cuatro extremos cardinales de cada nodo (*top*, *bottom*, *left*, *right*) operando bajo *ConnectionMode.Loose*, permitiendo conexiones libres y cálculo ortogonal de rutas vectoriales de aristas (*smoothstep* o *bezier*).
- **Aceleración por GPU:** Aprovechamiento de transformaciones CSS 3D y WebGL para soportar lienzos con decenas de entidades simultáneas manteniendo tasas de refresco de 60 FPS estables.

2.1.3) WebSockets y Protocolo STOMP para Sincronización Concurrente

Estandarizado bajo el RFC 6455, WebSocket establece un canal de comunicación dúplex completo sobre un único socket TCP persistente. Para estructurar los mensajes con semántica empresarial orientada a eventos, la plataforma implementa **STOMP (Simple Text Oriented Messaging Protocol)** sobre SockJS:

- **Broker de Mensajería en Memoria:** El servidor Spring Boot actúa como broker STOMP, enrutando mensajes hacia prefijos de destino de aplicación (*/app*) y reenviando eventos sincronizados a los suscriptores en tópicos de sala (*/topic/diagram/{projectId}*).
- **Latencia Ultra-Baja:** Distribución de mutaciones del grafo (movimiento de clases, edición de atributos, agregado de relaciones) con latencias inferiores a 50 milisegundos.
- **Candados de Edición Optimista (*element_locks*):** Mecanismo de concurrencia que bloquea temporalmente un nodo cuando un usuario lo selecciona o edita, evitando colisiones de escritura de dos arquitectos sobre la misma clase y liberando el recurso de inmediato al culminar la edición.

2.2) Ingeniería CASE, CBSE y Generación de Código Guiada por Modelos

2.2.1) Clasificación CASE (Upper, Lower, I-CASE) y MDE

La Ingeniería de Software Asistida por Computadora (CASE) clasifica sus herramientas según la etapa del ciclo de vida del software que automatizan:

- **Upper-CASE (Front-End CASE):** Asiste las fases tempranas del ciclo de desarrollo: captura de requisitos, análisis de dominio y modelado conceptual y lógico de diagramas (ej. ArchiTec, Enterprise Architect, StarUML).
- **Lower-CASE (Back-End CASE):** Automatiza las etapas de implementación física, generando código fuente, scripts de creación de bases de datos (DDL) y arneses de pruebas a partir de especificaciones de diseño.

- **I-CASE (Integrated CASE):** Fusiona las capacidades Upper y Lower en un ecosistema unificado y continuo. Nuestra plataforma se cataloga como un entorno **I-CASE Integral**, ya que sincroniza en tiempo real el diseño visual con la generación de código, la validación formal relacional y la persistencia de datos.

En el marco del **Desarrollo Basado en Componentes (CBSE)** y la **Ingeniería Guiada por Modelos (MDE / MDD)**, el diagrama de clases no es un artefacto documental pasivo, sino el *Modelo de Plataforma Independiente (PIM)* que un motor de transformación traduce automáticamente a un *Modelo Específico de Plataforma (PSM)* ejecutable (Java 21 / Spring Boot 4.1.0 y SQL PostgreSQL 17).

2.2.2) Motor de Plantillas FreeMarker y Árboles AST

Apache FreeMarker 2.3.33 es un motor de plantillas Java de alto rendimiento que genera texto estructurado a partir de árboles de datos en memoria. En el backend, el servicio **SpringBootGeneratorService** transforma el grafo del proyecto UML en un **Árbol de Sintaxis Abstracta (AST)** enriquecido con metadatos tipológicos (tipos de datos Java, nombres camelCase/PascalCase, mapeos JPA, relaciones y cardinalidades), evaluándolo sobre cuatro plantillas canónicas:

- **entity.ftl** → Compila las clases **@Entity** con atributos, constructores, getters/setters y relaciones **@OneToMany**, **@ManyToOne**, **@ManyToMany**.
- **repository.ftl** → Compila las interfaces **JpaRepository<Entity, Long>** con consultas derivadas.
- **service.ftl** → Compila los contratos e implementaciones de servicios con lógica transaccional **@Transactional**.
- **controller.ftl** → Compila los endpoints REST con contratos DTO y respuestas HTTP canónicas.
- **pom.xml** y **application.yml** → Scaffolding completo del proyecto Maven empaquetado en formato **.zip** con Maven Wrapper para ejecución inmediata (**./mvnw spring-boot:run**).

2.2.3) Interoperabilidad OMG XMI 2.1 con ArchiTec y StarUML

El estándar **XML Metadata Interchange (XMI 2.1)**, formulado por la Object Management Group (OMG) y la ISO/IEC 19509, estandariza el intercambio de modelos entre herramientas CASE de diferentes fabricantes. La plataforma implementa un serializador y deserializador bidireccional XMI 2.1:

- **Exportador XMI:** Traduce el grafo a un documento XML con esquemas **xmi:version="2.1"**, encapsulando clases en elementos **<packagedElement xmi:type="uml:Class">**, atributos en **<ownedAttribute>** con visibilidad y tipos primitivos UML, operaciones en **<ownedOperation>** y asociaciones en **<packagedElement xmi:type="uml:Association">**.
- **Importador XMI:** Parser con protección contra vulnerabilidades XXE (XML External Entity), que extrae el modelo de ArchiTec, Enterprise Architect o StarUML y lo

transforma al grafo del lienzo, aplicando el motor de auto-layout jerárquico `LayoutEngineUtil` para organizar automáticamente las clases en capas visuales legibles.

2.3) Arquitectura de Seguridad, Gobernanza y Gestión de Sesión

2.3.1) Gestión de Proyectos, Espacios de Trabajo y Exportación Documental

La plataforma reemplaza esquemas comerciales rígidos mediante un entorno de trabajo centrado en la ingeniería colaborativa:

- **Espacios de Trabajo y Clonación Profunda:** Permite la creación y gestión de proyectos con metadatos descriptivos (versión semántica `v1.0.0`, tags dinámicos) y la ejecución de **clonación profunda** (*Deep Copy*), la cual duplica en una única transacción atómica la cabecera del proyecto, todas sus clases y todas sus aristas relacionales remapeando consistentemente los identificadores primarios y foráneos sin alterar el proyecto base.
- **Exportación Documental Multiformato:** Complementa el estándar XMI con la generación de artefactos documentales para la comunicación técnica de ingeniería:
 - **PNG en Alta Definición:** Renderizado rasterizado en escalas 1x, 2x y 3x Ultra HD con fondos transparentes, oscuros o claros.
 - **Memoria Técnica en PDF:** Documento ejecutivo que incluye portada, metadatos del proyecto, diagrama de clases rasterizado incrustado y especificación tabular de entidades.
 - **Diccionario de Datos en Excel (.xlsx):** Libro de cálculo de 3 hojas (Entidades, Diccionario de Columnas con tipos PostgreSQL y JPA, e Inventario de Relaciones) generado mediante Apache POI.

2.3.2) Sesión Volátil con `sessionStorage` y Evento `beforeunload`

Para garantizar la máxima seguridad en equipos compartidos y entornos corporativos o académicos, la autenticación adopta una política **estrictamente volátil**:

- **Almacenamiento Restringido a `sessionStorage`:** A diferencia de `localStorage` o cookies persistentes, los datos almacenados en `sessionStorage` tienen un ciclo de vida atado a la pestaña del navegador; en cuanto el usuario cierra la ventana o pestaña, las credenciales y el Token JWT Bearer se destruyen físicamente de la memoria RAM del cliente.
- **Listener Defensivo `beforeunload`:** Antes de que la ventana se destruya, el cliente emite una notificación de desconexión inmediata al broker WebSocket STOMP para revocar candados activos y actualizar la presencia de usuarios en la sala.

2.3.3) Bitácora Inmutable de Auditoría y Trazabilidad

- **Auditoría Global de Gobernanza (`audit_logs`):** Registro forense inmutable de eventos administrativos y de seguridad (inicios de sesión, registros, mutaciones de roles RBAC, creación, eliminación y clonación de proyectos) con IP de origen, usuario actor, timestamp y payload de detalle en formato JSONB.
- **Trazabilidad del Modelo (`diagram_history`):** Registro cronológico granular de mutaciones estructurales del diagrama con diff antes/después, enriquecimiento de autores, papelera de reciclaje reversible y soporte de purga física definitiva (*hard delete*) restringida al propietario.

2.4) Inteligencia Artificial Multimodal, Resiliencia y SLMs Offline

2.4.1) Procesamiento de Lenguaje Natural (PLN) para Modelado Incremental

La integración de comandos por voz se fundamenta en la Web Speech API del navegador, que efectúa la conversión de audio a texto sin requerir librerías externas pesadas. El texto transcrito es procesado semánticamente por modelos de lenguaje que extraen entidades, atributos, visibilidad, tipos de datos y relaciones. La mutación del diagrama se aplica de forma reactiva e incremental en React Flow sin reinicializar el lienzo, permitiendo la reversibilidad total mediante atajos de teclado (Ctrl+Z y Ctrl+Y).

2.4.2) Visión Artificial para Digitalización Óptica de Pizarras

El módulo de visión por computadora aborda uno de los mayores problemas prácticos en sesiones de diseño ágiles: la pérdida de diagramas dibujados a manoalzada en pizarras físicas. Mediante modelos multimodales, el sistema detecta contornos rectangulares correspondientes a clases UML, reconoce texto manuscrito de atributos y operaciones, infiere flechas de conexión con sus puntas de flecha (asociación, herencia, agregación, composición) y construye un JSON topológico estructurado. El motor de auto-layout ubica los elementos de forma ordenada en el lienzo evitando superposiciones.

2.4.3) Cadena de Resiliencia Multi-IA (Circuit Breaker en 4 Niveles)

Para neutralizar la vulnerabilidad ante agotamiento de cuotas gratuitas, latencias elevadas o indisponibilidad de proveedores cloud, el servicio `MultiProviderAIService` implementa un patrón **Circuit Breaker** con conmutación en cascada a 4 niveles:

1. **Nivel 1 (Primario):** Google Gemini 1.5 / 2.5 Flash API — Procesamiento multimodal de alta precisión y velocidad subsegundo.
2. **Nivel 2 (Fallback Inmediato):** Groq Cloud Llama 3.3 70B / Llama 3.2 Vision — Motor de inferencia ultrarrápida LPU (< 250 ms).
3. **Nivel 3 (Contingencia):** OpenRouter API — Enrutamiento flexible hacia modelos de contingencia (Mistral, Claude, Qwen).

4. **Nivel 4 (Offline Local Heurístico):** Motor heurístico determinístico ejecutado en el propio servidor o navegador mediante expresiones regulares y autómatas finitos, garantizando que el usuario siempre pueda modelar incluso ante desconexión total de internet.

2.4.4) Modelos de Lenguaje Reducidos On-Device para Aplicaciones Móviles Offline

Como base de extensión arquitectónica hacia clientes móviles (ej. aplicaciones de campo en Flutter), la teoría de ingeniería de software contempla la incorporación de **Modelos de Lenguaje Reducidos (Small Language Models - SLMs)** ejecutados 100% on-device (en el procesador del dispositivo móvil sin conexión a internet):

- **Tecnologías de Ejecución Local:** Inferencia mediante ONNX Runtime Mobile, TensorFlow Lite o Google MediaPipe ejecutando modelos cuantizados a 4 bits (INT4) como **Gemma 2B, Llama 3.2 1B/3B o Phi-3 Mini**.
- **Estrategia Híbrida de Dos Modos:**
 - **Modo 2 (SLM On-Device Puro):** El modelo local corre en el dispositivo del usuario; procesa el dictado de voz y genera la orden o reporte directamente en SQLite local sin consumir datos ni depender de conectividad.
 - **Modo 1 (Híbrido de Fallback / Nube):** Ante consultas complejas o cuando se restablece la conexión de red, el sistema sincroniza con el backend central y delega el razonamiento analítico pesado al clúster de servidores

2.5) Metodología de Ingeniería de Software (PUDS y Taxonomía UML 2.5+)

2.5.1) Proceso Unificado de Desarrollo de Software (PUDS)

El **Proceso Unificado de Desarrollo de Software (PUDS)**, formulado por Ivar Jacobson, Grady Booch y James Rumbaugh, es un proceso de ingeniería de software caracterizado por ser:

- **Dirigido por Casos de Uso:** Los Casos de Uso guían todas las disciplinas técnicas del proyecto, desde la especificación de requisitos hasta los casos de prueba de aceptación.
- **Centrado en la Arquitectura:** Establece una arquitectura base robusta desde las primeras iteraciones para reducir drásticamente los riesgos técnicos.
- **Iterativo e Incremental:** El ciclo de vida se estructura en ciclos e iteraciones sucesivas que producen versiones operativas del sistema con valor tangible.

Las cinco disciplinas o flujos de trabajo del PUDS implementadas en este proyecto son:

1. **Flujo de Trabajo de Captura de Requisitos:** Identificación rigurosa de actores, requerimientos funcionales (RF) y no funcionales (RNF), estructuración de casos de uso y especificación detallada en fichas expandidas (Cockburn).

2. **Flujo de Trabajo de Análisis:** Refinamiento de casos de uso en diagramas de robustez BCE (Boundary, Control, Entity), identificación de paquetes de análisis, clases de análisis y máquinas de estado.
3. **Flujo de Trabajo de Diseño:** Definición de la arquitectura física (despliegue), arquitectura lógica en capas, diagramas de interacción (secuencia, comunicación, tiempos), diseño físico de base de datos relacional y volumetría.
4. **Flujo de Trabajo de Implementación:** Construcción del código ejecutable respetando la separación de capas, patrones arquitectónicos y directrices de codificación limpia.
5. **Flujo de Trabajo de Pruebas:** Verificación sistemática de la calidad mediante pruebas unitarias, de integración y casos de prueba funcionales de aceptación (CP).

2.5.2) Taxonomía Canónica de Diagramas UML y Normalización Relacional

Taxonomía Oficial de Diagramas OMG UML 2.5+

El estándar OMG UML 2.5 clasifica sus 14 tipos de diagramas en dos grandes familias:

1. Diagramas de Estructura:

- **Diagrama de Clases:** Modela la estructura estática del dominio, clases, atributos, tipos, operaciones y relaciones (asociación, agregación, composición, generalización y dependencia).
- **Diagrama de Despliegue:** Modela la topología de nodos físicos y de cómputo, entornos de ejecución y los artefactos de software desplegados en ellos.
- **Diagrama de Objetos:** Muestra instancias concretas de clases y sus enlaces en un instante determinado de tiempo.
- **Diagrama de Componentes:** Describe la organización y dependencias entre módulos de software autónomos, puertos e interfaces provistas/requeridas.
- **Diagrama de Estructura Compuesta:** Detalla la descomposición interna de un clasificador, incluyendo sus partes, puertos y conectores.
- **Diagrama de Paquetes:** Ilustra la partición lógica del modelo en subsistemas y espacios de nombres, junto con las dependencias entre ellos.

2. Diagramas de Comportamiento:

- **Diagrama de Casos de Uso:** Modela las funcionalidades del sistema desde la perspectiva de los actores externos y sus relaciones («**include**», «**extend**», generalización).
- **Diagrama de Actividad:** Modela el flujo de control y de datos en un proceso de negocio o algoritmo paso a paso, con soporte de bifurcaciones, concurrencia y calles (*swimlanes*).
- **Diagrama de Máquina de Estados:** Representa el ciclo de vida de un objeto, los estados discretos por los que atraviesa y las transiciones detonadas por eventos.
- **Diagramas de Interacción:**

- **Diagrama de Secuencia:** Enfatiza la ordenación temporal de mensajes intercambiados entre líneas de vida (*lifelines*) mediante fragmentos combinados (**alt**, **opt**, **loop**).
- **Diagrama de Comunicación:** Enfatiza la organización estructural de los objetos participantes en una interacción y los mensajes numerados entre ellos.
- **Diagrama de Tiempos (Timing):** Modela cambios de estado a lo largo de una escala de tiempo continua, fundamental para sistemas concurrentes y en tiempo real.
- **Diagrama Global de Interacciones (Interaction Overview):** Variación de los diagramas de actividad donde los nodos son marcos de interacción (secuencias o comunicaciones), permitiendo visualizar flujos complejos de orquestación.

Reglas de Transformación Objeto-Modelo y Formas Normales (1NF a 3NF)

La conversión de diagramas conceptuales de clases UML a esquemas físicos relacionales se rige por la teoría relacional de Edgar F. Codd (1970) y las reglas de TOM:

- **Primera Forma Normal (1NF):** Todo atributo debe ser atómico (indivisible) y no deben existir grupos repetitivos. En términos de modelado, cada entidad debe poseer una Clave Primaria unívoca (**{PK}** / **@Id**) de tipo compatible (**Long IDENTITY** o **UUID**).
- **Segunda Forma Normal (2NF):** Cumple 1NF y ningún atributo no clave depende parcialmente de una clave primaria compuesta. En relaciones 1..*, la Clave Foránea (**{FK}**) debe situarse estrictamente en la entidad del extremo muchos (*), referenciando la clave primaria del extremo uno (1).
- **Tercera Forma Normal (3NF):** Cumple 2NF y ningún atributo no clave depende transitivamente de la clave primaria (eliminación de dependencias transitivas A \to B \to C y atributos calculados). Toda relación muchos a muchos (*..*) debe descomponerse formalmente en dos relaciones 1..* vinculadas a una tabla intermedia asociativa que consolida las claves foráneas compuestas.

Parte III: Metodología de Desarrollo (PUDS)

3.1) Flujo de Trabajo: Captura de Requisitos

La meta fundamental en esta etapa del Proceso Unificado es comprender a fondo las necesidades técnicas, metodológicas y operativas de los arquitectos de software, desarrolladores y administradores, definiendo con rigurosidad los servicios que la plataforma CASE debe ofrecer para agilizar el modelado visual canónico bajo el estándar UML 2.5+, la interoperabilidad de modelos mediante OMG XMI 2.1, la generación automatizada de backend en Spring Boot 3.x / Java 21 estructurado en 4 capas desacopladas, la producción de scripts relacionales DDL PostgreSQL 17, la suite de aserciones Postman v2.1 y la colaboración multiusuario en tiempo real asistida por Inteligencia Artificial multimodal.

3.1.1) Identificación de Actores

Para garantizar una operación segura, robusta y con estricto control de privilegios, la plataforma establece tres roles humanos fundamentales:

- **A1 — Administrador Principal (SUPER_ADMIN):** Responsable de la gobernanza técnica de la plataforma, la gestión del control de acceso basado en roles (RBAC) en la consola /admin/users, la supervisión inmutable de eventos globales de seguridad y transacciones en audit_logs, y la inspección y restauración de proyectos archivados en la papelera de reciclaje (/admin/projects). Ejerce la restricción inviolable de no poder eliminar lógicamente ni purgar definitivamente proyectos pertenecientes a otros usuarios.
- **A2 — Arquitecto de Software (ARQUITECTO):** Actor central del ciclo de ingeniería y diseño de software. Crea y administra espacios de trabajo a partir de plantillas de dominio (evitando lienzos en blanco), modela clases y relaciones canónicas UML 2.5+ en el lienzo interactivo, ejecuta la certificación heurística de normalización relacional (1NF a 3NF), opera las herramientas de asistencia IA (dictado por voz PLN y digitalización óptica de pizarras físicas con Circuit Breaker resiliente), gestiona la importación y exportación de archivos estándar OMG XMI 2.1 (ArchiTec / StarUML) y detona la generación automatizada del backend en Spring Boot (4 capas en ZIP con Maven Wrapper y Swagger), esquemas DDL PostgreSQL 17 y suites de pruebas Postman v2.1. En sesiones de co-diseño colaborativo, actúa como anfitrión (Host).
- **A3 — Colaborador / Ingeniero de Software (COLABORADOR):** Desarrollador o ingeniero de datos que participa activamente en sesiones de co-diseño en tiempo real mediante WebSockets STOMP ingresando mediante código de sala y DNI/nombre. Co-diseña el grafo de clases bajo candados optimistas de edición (element_locks), valida la integridad relacional de los esquemas, participa en el chat colaborativo con aislamiento de teclado y consulta el historial de trazabilidad del proyecto.

3.1.2) Requerimientos Funcionales (RF)

Representan las capacidades operativas y contractuales que el sistema garantiza en cada uno de sus módulos:

- **RF-01 (Autenticación Stateless y Sesión Volátil):** El sistema debe proveer autenticación segura sin estado (stateless), emitiendo tokens JWT Bearer firmados con HMAC-SHA256, almacenados estrictamente en memoria volátil (sessionStorage) para garantizar el cierre automático de sesión al cerrar la pestaña o ventana del navegador. Asimismo, debe gestionar el registro de cuentas con contraseñas cifradas bajo BCrypt (costo 12) y persistir preferencias visuales en formato JSONB.
- **RF-02 (Control de Roles RBAC y Gobernanza):** La plataforma debe proveer una consola centralizada exclusiva para el Administrador Principal (/admin/users) que permita supervisar usuarios registrados, conmutar su estado operativo (activo/suspendido) y reasignar roles de seguridad, aplicando políticas automáticas anti-autodegradación y prevención de autobloqueo.
- **RF-03 (Gestión de Proyectos, Plantillas y Onboarding):** El sistema debe permitir la administración integral del ciclo de vida de los proyectos UML en el Dashboard (creación desde cero, edición de metadatos, tags de dominio, versionado semántico v1.0.0), clonación profunda (Deep Copy) transaccional de proyectos con replicación en cascada de nodos y aristas, catálogo de plantillas base de dominio (Académico, Hospitalario, E-Commerce) y un tutorial de onboarding interactivo guiado en 5 pasos (< 120s).
- **RF-04 (Auditoría Forense Global y Trazabilidad del Modelo):** El sistema debe registrar eventos administrativos y de seguridad en una bitácora inmutable (audit_logs) con filtros analíticos y exportación a Excel, CSV y JSON para el Super Admin. Asimismo, debe registrar el historial cronológico de mutaciones del proyecto en diagram_history con diffs antes/después, papelera de reciclaje reversible y purga física definitiva en cascada (hard delete) restringida al propietario.
- **RF-05 (Modelado Canónico de Clases y Relaciones UML 2.5):** El lienzo de diseño debe proveer modelado reactivo de clases con 3 compartimentos estándar, tipos de datos fuertemente tipados (Java/SQL), badge {PK}, relaciones ortogonales con 4 handles magnéticos, cardinalidades estándar (1, 0..1, , 1..), detección y prevención matemática de ciclos de herencia mediante algoritmo DFS, y soporte de reversibilidad total con Ctrl+Z y Ctrl+Y.
- **RF-06 (Certificación de Normalización Relacional 1NF a 3NF):** El motor relacional cliente debe evaluar el grafo de clases en tiempo real contra las reglas formales de TOM y Codd: existencia de claves primarias atómicas (1NF), claves foráneas en asociaciones de multiplicidad 1..* (2NF) y descomposición en tablas intermedias para relaciones *.* (3NF), desplegando un semáforo visual porcentual (0-100%) y certificación inmutable.
- **RF-07 (Exportación Multiformato XMI, PNG, PDF, Excel):** La herramienta debe permitir la exportación del modelo en 4 formatos estándar: archivo de intercambio estándar OMG XMI 2.1 compatible con Enterprise Architect y ArchiTec, captura gráfica en Ultra-HD PNG (resoluciones 1x, 2x y 3x), Memoria Técnica PDF ejecutiva con diagrama vectorial y tablas de atributos incrustadas, y Libro de Datos Excel (.xlsx) estructurado en 3 hojas técnicas con mapeos JPA y SQL.

- **RF-08 (Importación XMI 2.1 con Auto-Layout Jerárquico):** El sistema debe incorporar un parser XML defensivo anti-XXE (XMLConstants.FEATURE_SECURE_PROCESSING) capaz de deserializar paquetes UML estándar en formato XMI 2.1, extrayendo clases, atributos, visibilidades y relaciones, y aplicando un algoritmo de posicionamiento jerárquico no colisionante (LayoutEngineUtil) para renderizar el diagrama en el lienzo.
- **RF-09 (Generación de Backend Spring Boot en 4 Capas ZIP):** El generador de plantillas FreeMarker debe compilar el modelo de clases a código fuente Java 21 organizado rigurosamente en 4 capas desacopladas (Controller, Service, Repository, Entity), empaquetado en un archivo comprimido .zip listo para ejecución que incluya Maven Wrapper (mvnw), archivo pom.xml con dependencias Spring Boot 3.x, application.properties y documentación interactiva OpenAPI Swagger.
- **RF-10 (Generación de Esquema DDL SQL y Suite Postman v2.1):** El sistema debe producir scripts DDL SQL para PostgreSQL 17 con cláusulas CREATE TABLE, claves primarias IDENTITY, claves foráneas con integridad referencial CASCADE e índices B-Tree optimizados. Asimismo, debe generar la colección JSON Postman v2.1.0 correspondiente con peticiones CRUD preconfiguradas y aserciones automatizadas de pruebas funcionales pm.test.
- **RF-11 (Modelado Asistido por Dictado de Voz PLN):** El lienzo debe admitir comandos de modelado por voz capturados mediante la Web Speech API del navegador y procesados por un Circuit Breaker en 4 niveles (Gemini 2.5 Flash, Groq Llama 3.3 70B, OpenRouter y Motor Local Offline con regex), ejecutando mutaciones incrementales no destructivas y reversibles con Ctrl+Z.
- **RF-12 (Digitalización Óptica de Pizarras Físicas con Visión):** El sistema debe digitalizar diagramas dibujados a mano alzada en pizarras o papel mediante captura por cámara o subida de imagen, analizándolos con LLMs multimodales con fallback automático, aplicando auto-layout jerárquico, inyección defensiva de claves primarias 1NF ({PK} id: Long) y reversibilidad inmediata con Ctrl+Z.
- **RF-13 (Sincronización Colaborativa Concurrente WSS STOMP):** La plataforma debe habilitar salas de co-diseño multiusuario en tiempo real mediante WebSockets STOMP sobre SockJS (< 50 ms de latencia), acceso con código de sala y DNI/nombre, candados optimistas (element_locks), chat en vivo con aislamiento de teclado, cursores remotos y expulsión disciplinaria (kick) controlada por el Host.

3.1.3) Requerimientos No Funcionales (RNF)

Definen las restricciones de calidad, desempeño, resiliencia y seguridad técnica de la plataforma:

- **RNF-01 (Rendimiento y Latencia):** La compilación FreeMarker y empaquetado del proyecto Spring Boot en ZIP debe completarse en un tiempo inferior a 500 ms. La retransmisión de eventos colaborativos vía WebSockets STOMP debe presentar una latencia menor a 50 ms. El lienzo React Flow debe mantener una tasa constante de 60 FPS con grafos de hasta 50 nodos.
- **RNF-02 (Disponibilidad y Resiliencia Multi-IA):** La integración con modelos de lenguaje e inferencia multimodal debe operar bajo un Circuit Breaker con cadena de fallback de 4 niveles: Gemini 2.5 Flash → Groq Llama 3.3 70B → OpenRouter → Motor Local Regex Offline. El sistema debe garantizar respuesta continua incluso en ausencia total de conectividad a internet.

- **RNF-03 (Seguridad, Privacidad y Cero Persistencia de Secretos):** Los tokens JWT Bearer deben ser estrictamente volátiles (sessionStorage) destruyéndose al cerrar la ventana. Las contraseñas se almacenan mediante hash BCrypt (factor de costo 12). La base de datos Supabase PostgreSQL 17 aplica Row-Level Security (RLS) y el parser XML bloquea ataques de entidad externa (XXE).
- **RNF-04 (Interoperabilidad y Conformidad de Estándares):** La serialización de diagramas debe cumplir con la especificación OMG UML 2.5+ y OMG XMI 2.1. El código backend Java generado debe alinearse con la Google Java Style Guide y las directrices de Spring Boot 3.x / 4.x. El esquema DDL debe ser compatible con ANSI SQL:2016 y PostgreSQL 17.
- **RNF-05 (Usabilidad y Curva de Aprendizaje Acelerada):** La plataforma debe incorporar un tutorial interactivo guiado en 5 pasos completable en menos de 120 segundos. La interfaz visual debe implementar las paletas de alta fidelidad ergonómica Titanio Cálido y Grafito Obsidiana, eliminando fondos monocromáticos fúnebres y empleando iconografía 100% vectorial con Lucide React (cero emojis).
- **RNF-06 (Concurrencia, Bloqueo Optimista e Integridad de Grafo):** En sesiones colaborativas se debe aplicar bloqueo optimista (element_locks) por elemento para impedir sobreescrituras concurrentes. La gestión de atajos de teclado debe aislar el lienzo durante la escritura en inputs. Las relaciones de generalización deben ser validadas matemáticamente mediante Depth-First Search (DFS) para impedir ciclos de herencia.

3.1.4) Paquetes de Casos de Uso del Sistema

La funcionalidad de la herramienta CASE se estructura formalmente en cuatro paquetes de casos de uso, caracterizados por una alta cohesión interna y un bajo acoplamiento funcional:

Paquete 1: Seguridad, Identidad y Gobernanza

Encapsula la gestión de acceso sin estado (stateless) con sesión volátil en sessionStorage, el control de roles (RBAC), la administración integral de proyectos en el Dashboard y la bitácora inmutable de auditoría.

- **CU01 — Autenticación Stateless y Gestión de Cuentas:** Validación de credenciales con emisión de Token JWT Bearer volátil en sessionStorage, registro de usuarios con contraseñas cifradas bajo BCrypt (costo 12) y almacenamiento de preferencias del entorno CASE en formato JSONB.
- **CU02 — Control de Roles (RBAC) y Gobernanza:** Administración centralizada de cuentas de usuario por el Super Admin en /admin/users, reasignación de roles y aplicación de políticas anti-autodegradación y prevención de autobloqueo.
- **CU03 — Gestión de Proyectos, Plantillas y Onboarding:** Administración integral del ciclo de vida de proyectos UML (CRUD, versionado semántico v1.0.0, tags), clonación profunda transaccional (Deep Copy), plantillas de dominio y tutorial interactivo guiado (< 120s).
- **CU04 — Auditoría Forense Global y Trazabilidad del Modelo:** Bitácora inmutable audit_logs con filtros y exportación (Excel/CSV/JSON) para el Administrador, historial cronológico diagram_history con diffs, papelera de reciclaje reversible y purga física definitiva.

Paquete 2: Modelador Visual Core y Reglas de Negocio UML

Gestiona el modelado grafocéntrico reactivo en el lienzo visual, la consistencia estructural bajo el estándar OMG UML 2.5+ y la certificación de normalización relacional según las reglas de TOM y Codd.

- **CU05 — Modelado Canónico de Clases y Relaciones UML 2.5:** Creación y edición de clases en 3 compartimentos estándar, tipos de datos fuertemente tipados (Java/SQL), badge {PK}, relaciones ortogonales con 4 handles magnéticos cardinales, prevención matemática de ciclos de herencia mediante algoritmo DFS y reversibilidad completa con Ctrl+Z y Ctrl+Y.
- **CU06 — Certificación de Normalización Relacional (1NF a 3NF):** Motor heurístico de evaluación de integridad relacional (existencia de claves primarias en 1NF, claves foráneas en relaciones 1..* en 2NF y tablas intermedias en *.* en 3NF), con semáforo porcentual en tiempo real (0-100%) y certificación inmutable.

Paquete 3: Interoperabilidad XMI y Motor Generador CASE

Comprende la transformación bidireccional de modelos hacia estándares internacionales de la industria y la compilación automatizada de código fuente ejecutable en arquitecturas multicapa.

- **CU07 — Exportación Multiformato (XMI, PNG, PDF, Excel):** Exportación de modelos a estándar OMG XMI 2.1 compatible con ArchiTec y StarUML, capturas gráficas UHD PNG (1x, 2x, 3x), Memoria Técnica ejecutiva PDF con diagrama incrustado y Libro Excel (.xlsx) de 3 hojas con tipos JPA y SQL.
- **CU08 — Importación XMI 2.1 con Auto-Layout Jerárquico:** Deserialización defensiva anti-XXE de archivos XMI 2.1 externos y renderizado en lienzo mediante algoritmo de posicionamiento jerárquico no colisionante (LayoutEngineUtil).
- **CU09 — Generación de Backend Spring Boot en 4 Capas (ZIP):** Compilación mediante plantillas Apache FreeMarker 2.3.33 hacia microservicios Java 21 estructurados en Controller, Service, Repository y Entity, empaquetados en un archivo ZIP con Maven Wrapper (mvnw) y Swagger OpenAPI.
- **CU10 — Generación de Esquema DDL SQL y Suite Postman v2.1:** Producción de scripts DDL PostgreSQL 17 con cláusulas IDENTITY, claves foráneas ON DELETE CASCADE e índices B-Tree, y colección JSON Postman v2.1.0 con suite CRUD completa y aserciones automáticas pm.test.

Paquete 4: Asistencia con Inteligencia Artificial y Concurrencia Colaborativa

Agrupar las capacidades de interacción mediante lenguaje natural por dictado de voz, digitalización óptica de pizarras físicas y sincronización multiusuario concurrente en tiempo real.

- **CU11 — Modelado Asistido por Dictado de Voz (PLN):** Mutación incremental no destructiva del diagrama a partir de comandos de voz capturados por Web Speech API y procesados mediante Circuit Breaker en 4 niveles (Gemini Flash, Groq Llama 3.3, OpenRouter y Motor Local Offline), con reversibilidad Ctrl+Z.
- **CU12 — Digitalización Óptica de Pizarras Físicas con Visión:** Extracción automática de clases y relaciones desde fotografías de diagramas dibujados a manoalzada procesadas por LLMs de visión multimodal, con auto-layout jerárquico y defensa relacional preventiva 1NF ({PK} id: Long).

- **CU13 — Sincronización Colaborativa Concurrente WSS STOMP:** Sesiones de co-diseño multiusuario en tiempo real sobre WebSockets STOMP (< 50 ms de latencia) con acceso por código de sala y DNI/nombre, candados optimistas (element_locks), panel de chat en vivo con aislamiento de teclado, cursores remotos y facultad de expulsión (kick) por el Host.

Clasificación de Casos de Uso por Entorno de Aplicación

ID	Caso de Uso	Web (React Flow / Canvas)	Desktop / Herramienta CASE	API / Backend (Spring Boot 4 Capas)
CU01	Autenticación Stateless y Gestión de Cuentas	X		X
CU02	Control de Roles (RBAC) y Gobernanza	X		X
CU03	Gestión de Proyectos, Plantillas y Onboarding	X	X	X
CU04	Auditoría Forense Global y Trazabilidad del Modelo	X		X
CU05	Modelado Canónico de Clases y Relaciones UML 2.5	X	X	X
CU06	Certificación de Normalización Relacional (1NF a 3NF)	X	X	
CU07	Exportación Multiformato (XMI, PNG, PDF, Excel)	X	X	X
CU08	Importación XMI 2.1 con Auto-Layout Jerárquico	X	X	X
CU09	Generación de Backend Spring Boot en 4 Capas (ZIP)	X	X	X
CU10	Generación de Esquema DDL SQL y Suite Postman v2.1	X	X	X
CU11	Modelado Asistido por Dictado de Voz (PLN)	X	X	X

CU12	Digitalización Óptica de Pizarras Físicas con Visión	X	X	X
CU13	Sincronización Colaborativa Concurrente WSS STOMP	X	X	X

3.1.5) Priorizar Casos de Uso

N.º	Caso de Uso	Actores Asociados	Ciclo PUDS	Prioridad
CU01	Autenticación Stateless y Gestión de Cuentas	A1, A2, A3	Ciclo 1	Alta
CU02	Control de Roles (RBAC) y Gobernanza	A1	Ciclo 1	Media
CU03	Gestión de Proyectos, Plantillas y Onboarding	A2, A3	Ciclo 1	Alta
CU04	Auditoría Forense Global y Trazabilidad del Modelo	A1, A2	Ciclo 1	Alta
CU05	Modelado Canónico de Clases y Relaciones UML 2.5	A2, A3	Ciclo 1	Alta
CU06	Certificación de Normalización Relacional (1NF a 3NF)	A2, A3	Ciclo 1	Alta
CU07	Exportación Multiformato (XMI, PNG, PDF, Excel)	A2, A3	Ciclo 2	Alta
CU08	Importación XMI 2.1 con Auto-Layout Jerárquico	A2	Ciclo 2	Alta
CU09	Generación de Backend Spring Boot en 4 Capas (ZIP)	A2	Ciclo 2	Alta
CU10	Generación de Esquema DDL SQL y Suite Postman v2.1	A2	Ciclo 2	Alta
CU11	Modelado Asistido por Dictado de Voz (PLN)	A2, A3	Ciclo 3	Media
CU12	Digitalización Óptica de Pizarras Físicas con Visión	A2	Ciclo 3	Media
CU13	Sincronización Colaborativa Concurrente WSS STOMP	A2, A3	Ciclo 3	Alta

Distribución por Ciclos de Desarrollo PUDS

Ciclo #1: Seguridad, Workspaces, Modelador Core y Normalización Relacional

N.º	Caso de Uso	Actor Iniciador	Prioridad
CU01	Autenticación Stateless y Gestión de Cuentas	A1, A2, A3	Alta
CU02	Control de Roles (RBAC) y Gobernanza	A1	Media
CU03	Gestión de Proyectos, Plantillas y Onboarding	A2	Alta
CU04	Auditoría Forense Global y Trazabilidad del Modelo	A1, A2	Alta
CU05	Modelado Canónico de Clases y Relaciones UML 2.5	A2	Alta
CU06	Certificación de Normalización Relacional (1NF a 3NF)	A2	Alta

Ciclo #2: Interoperabilidad CASE y Generación Automatizada (MDD)

N.º	Caso de Uso	Actor Iniciador	Prioridad
CU07	Exportación Multiformato (XMI, PNG, PDF, Excel)	A2	Alta
CU08	Importación XMI 2.1 con Auto-Layout Jerárquico	A2	Alta
CU09	Generación de Backend Spring Boot en 4 Capas (ZIP)	A2	Alta
CU10	Generación de Esquema DDL SQL y Suite Postman v2.1	A2	Alta

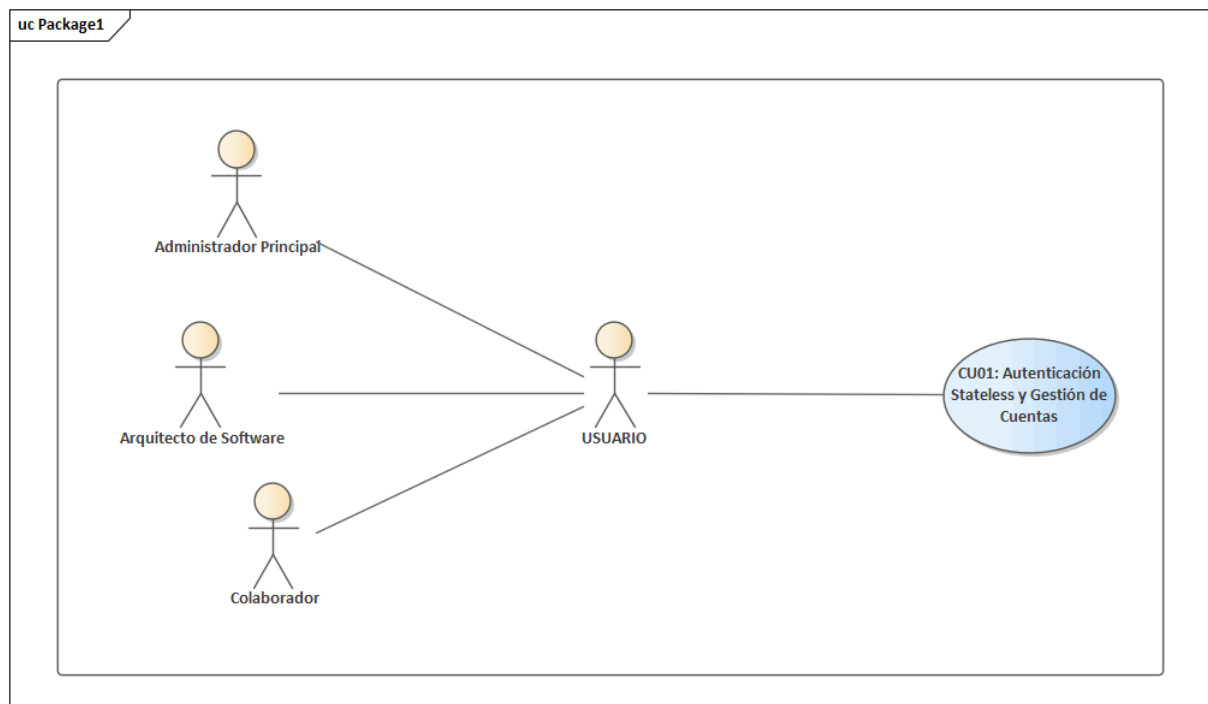
Ciclo #3: Asistencia IA Multimodal y Concurrencia Colaborativa

N.º	Caso de Uso	Actor Iniciador	Prioridad
CU11	Modelado Asistido por Dictado de Voz (PLN)	A2	Media
CU12	Digitalización Óptica de Pizarras Físicas con Visión	A2	Media
CU13	Sincronización Colaborativa Concurrente WSS STOMP	A2	Alta

3.1.6) Detallar Casos de Uso

Ciclo #1

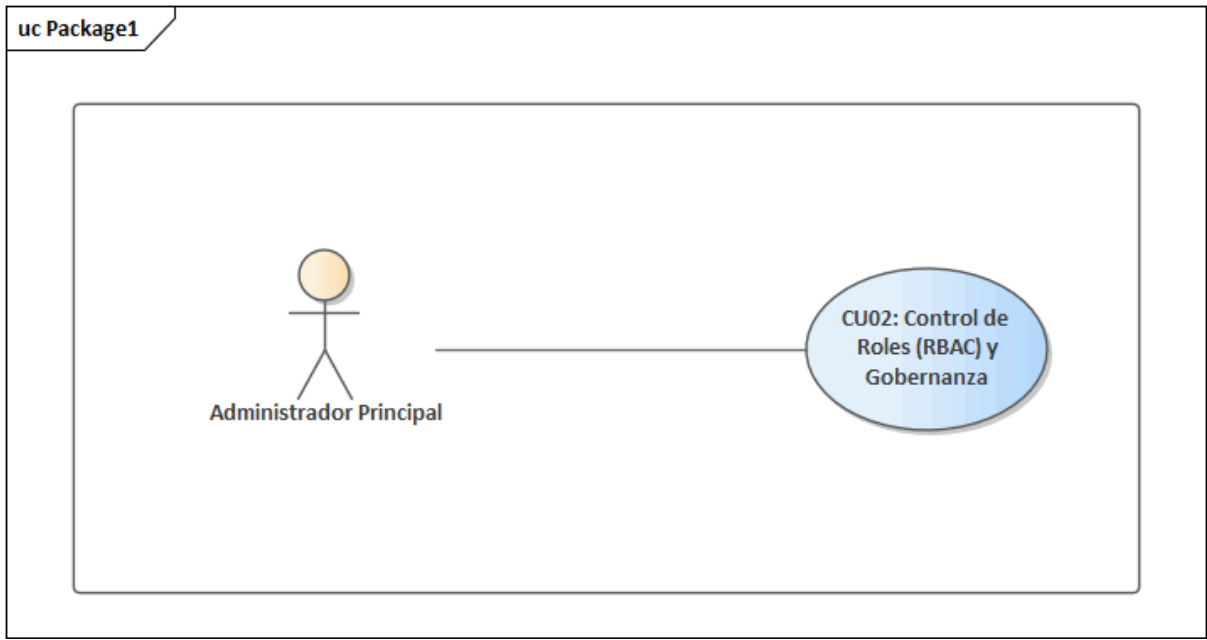
CU01: Autenticación Stateless y Gestión de Cuentas



Campo	Especificación Técnica
Propósito	Validar la identidad de los usuarios emitiendo un Token JWT Bearer administrado estrictamente en memoria volátil (sessionStorage), permitir el registro de cuentas con contraseñas cifradas bajo BCrypt (costo 12) y gestionar preferencias del entorno CASE en formato JSONB.
Actores	A1 — Administrador Principal, A2 — Arquitecto de Software, A3 — Colaborador
Actor Iniciador	A2 — Arquitecto de Software / A3 — Colaborador / A1 — Administrador Principal
Pre-Condición	Conectividad con el servidor backend y no poseer una sesión activa previa con el mismo token.

Flujo de Proceso Principal	1. El actor accede a la pantalla de login o registro en la aplicación web. 2. Si es registro: introduce nombre completo, correo electrónico, username y contraseña; el backend valida la no duplicidad, cifra la contraseña con BCrypt (costo 12), asigna el rol por defecto (ARQUITECTO) y emite el token. 3. Si es inicio de sesión: ingresa correo/username y contraseña; el backend valida credenciales en PostgreSQL 17 y verifica que la cuenta esté activa. 4. El backend emite el Token JWT firmado con claims de rol y vigencia. 5. El frontend almacena el Token JWT exclusivamente en sessionStorage para garantizar su destrucción al cerrar la ventana. 6. El sistema redirige al usuario a su entorno correspondiente (Dashboard para Arquitectos/Colaboradores o Consola de Gobernanza para Administrador).
Post-Condición	Sesión volátil iniciada con token en memoria. Al cerrar la pestaña del navegador, la sesión culmina automáticamente.
Excepciones	E1. Credenciales inválidas o cuenta suspendida: El backend retorna HTTP 401/403 y el frontend muestra alerta de rechazo sin otorgar acceso. E2. Correo duplicado en registro: El backend rechaza la transacción con HTTP 409 Conflict.

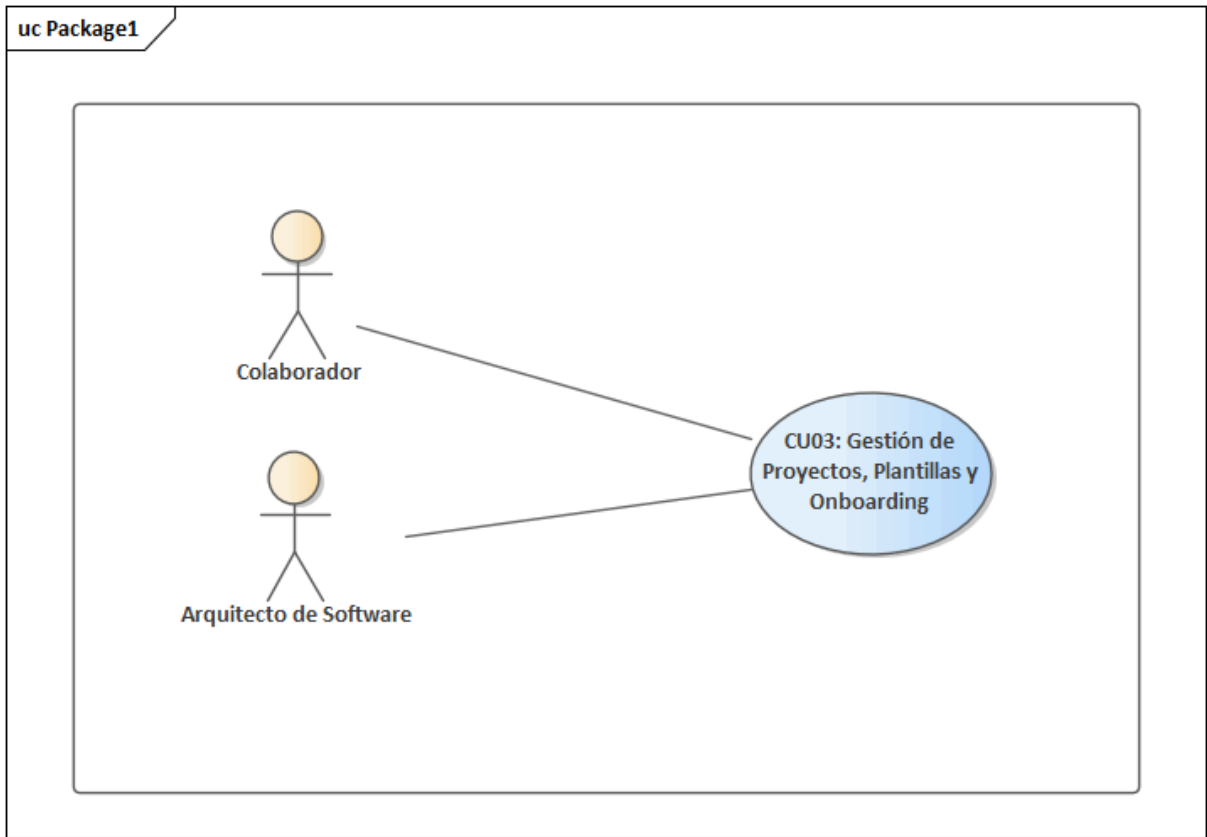
CU02: Control de Roles (RBAC) y Gobernanza



Campo	Especificación Técnica
Propósito	Proveer una consola centralizada exclusiva para el Administrador Principal que permita listar usuarios registrados, conmutar su estado operativo (activo/suspendido) y reasignar roles de seguridad, aplicando restricciones contra la autodegradación.
Actores	A1 — Administrador Principal
Actor Iniciador	A1 — Administrador Principal
Pre-Condición	Sesión autenticada con rol SUPER_ADMIN.
Flujo de Proceso Principal	1. El Administrador accede a la consola de gobernanza en /admin/users. 2. El sistema consulta la base de datos y despliega la lista paginada de usuarios con su nombre, correo, rol actual y estado. 3. El Administrador selecciona un usuario y modifica su rol (Super Admin, Arquitecto, Colaborador) o conmuta su estado entre activo y suspendido. 4. El backend valida que el Administrador no esté intentando modificar su propia cuenta para prevenir autodegradación o autobloqueo. 5. El sistema actualiza la fila en user_profiles y registra un evento inmutable en audit_logs con IP, actor y timestamp. 6. La interfaz actualiza reactivamente el badge de rol o estado del usuario.
Post-Condición	Rol o estado modificado en PostgreSQL 17 y trazado en bitácora inmutable de auditoría.

Excepciones	E1. Intento de autodegradación o autobloqueo: El backend bloquea la petición con HTTP 400 Bad Request ("No puede modificar su propio rol o estado"). E2. Usuario inexistente: El backend retorna HTTP 404 Not Found.
-------------	--

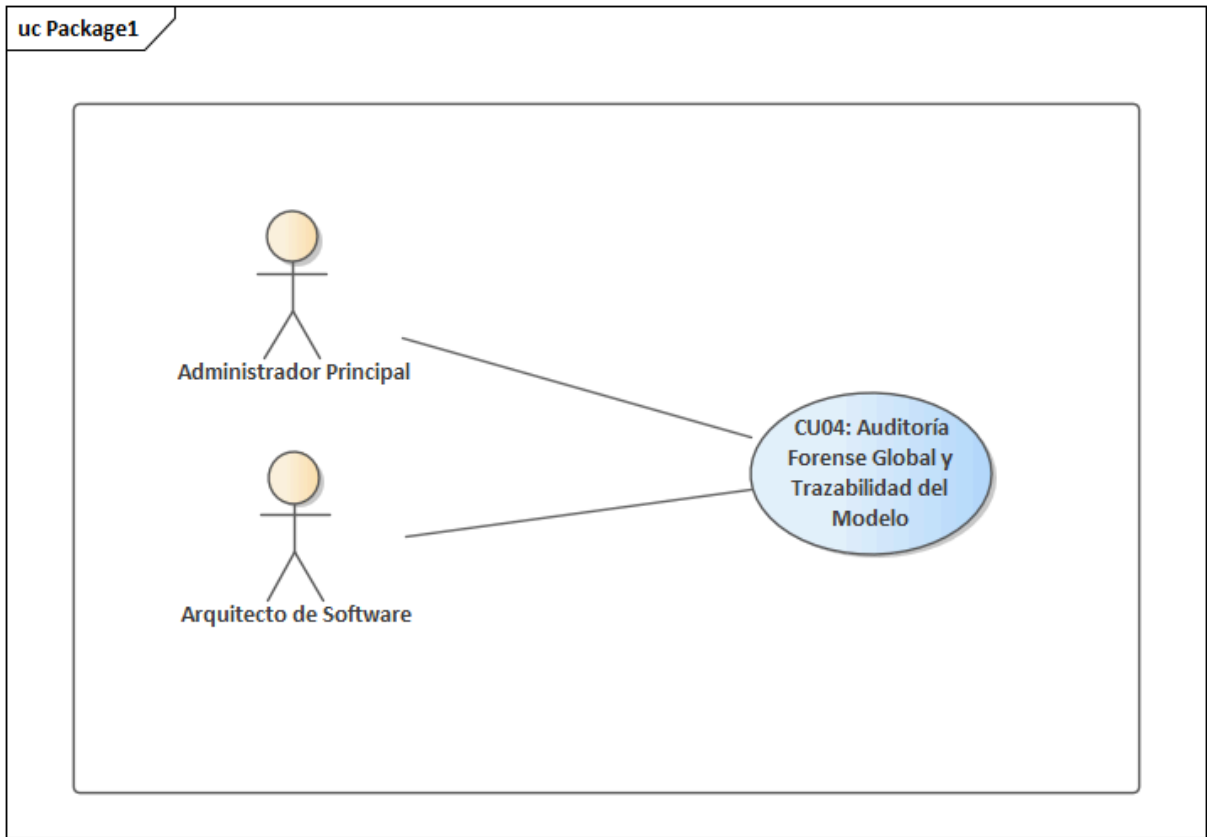
CU03: Gestión de Proyectos, Plantillas y Onboarding



Campo	Especificación Técnica
Propósito	Permitir la administración completa de proyectos en el Dashboard (creación, edición de metadatos, tags, versión semántica v1.0.0), la clonación profunda (Deep Copy) transaccional de grafos, el scaffolding inicial desde plantillas de dominio y la inducción mediante onboarding guiado.
Actores	A2 — Arquitecto de Software, A3 — Colaborador
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Usuario autenticado con rol ARQUITECTO o COLABORADOR.

Flujo de Proceso Principal	1. El actor ingresa a la vista de proyectos en el Dashboard (/projects). 2. Para crear: presiona "Nuevo Proyecto", selecciona entre lienzo en blanco o catálogo de plantillas (Académico, Hospitalario, Facturación) e ingresa título, descripción, versión semántica y tags. 3. El backend genera el registro en diagram_projects y, de ser plantilla, clona las clases y relaciones preconfiguradas. 4. Para clonar: presiona "Clonar Proyecto"; el backend ejecuta cloneProject, duplicando transaccionalmente el proyecto y replicando en cascada todos sus class_nodes y relationships con nuevos UUIDs y remapeo de identificadores. 5. Si el actor ingresa por primera vez al lienzo, el sistema activa un tutorial de onboarding de 5 pasos interactivos (< 120s), guardando la preferencia de completitud en user_profiles.preferences.
Post-Condición	Proyecto creado, clonado o tutorial completado con persistencia transaccional en PostgreSQL 17.
Excepciones	E1. Fallo de persistencia durante clonación: Se ejecuta Rollback íntegro preservando la integridad del proyecto original. E2. Título de proyecto vacío: El formulario bloquea el envío validando obligatoriedad en cliente y servidor.

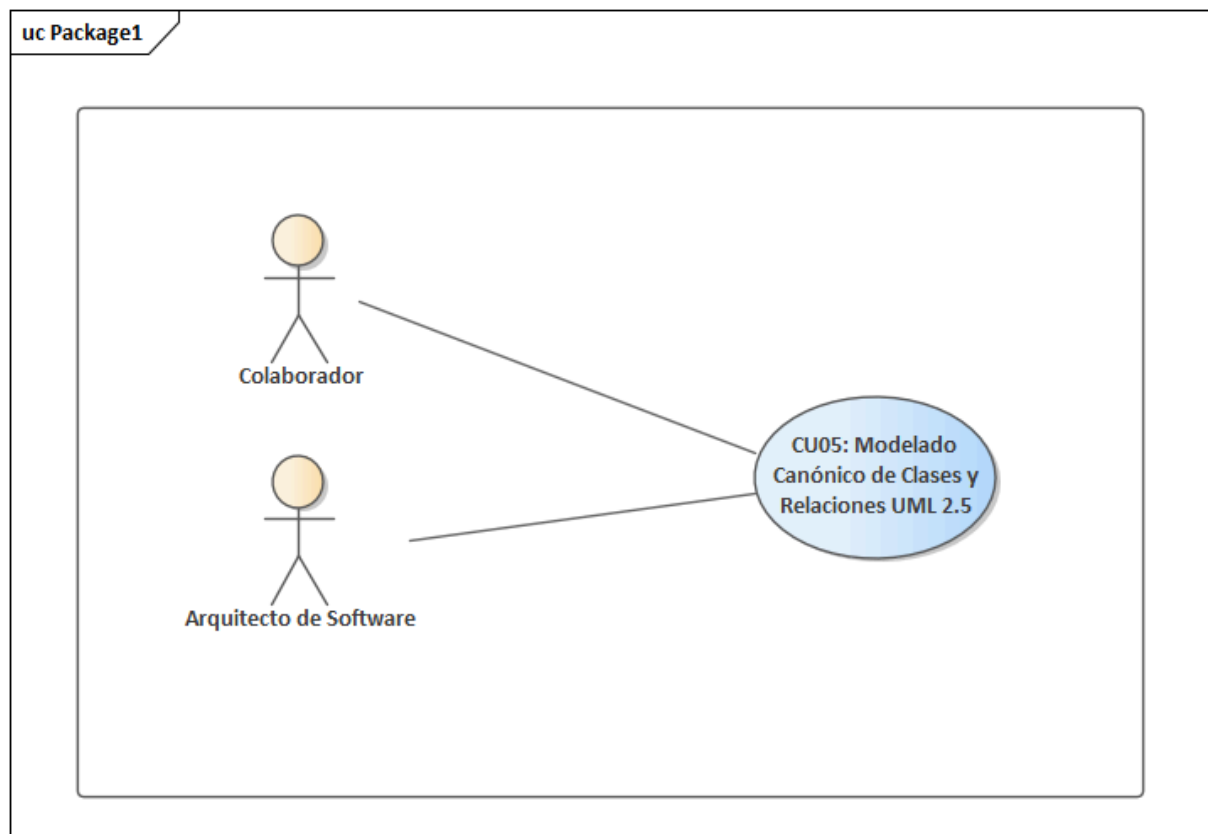
CU04: Auditoría Forense Global y Trazabilidad del Modelo



Campo	Especificación Técnica
Propósito	Proveer una bitácora inmutable de eventos de seguridad y gobernanza (audit_logs) con filtros analíticos y exportación, mantener el timeline de versiones del diagrama (diagram_history) con diffs antes/después, papelera de reciclaje reversible y purga definitiva en cascada.
Actores	A1 — Administrador Principal (Auditoría Global), A2 — Arquitecto de Software (Trazabilidad y Papelera)
Actor Iniciador	A1 — Administrador Principal o A2 — Arquitecto de Software
Pre-Condición	Para audit_logs, rol SUPER_ADMIN. Para trazabilidad del modelo, ser propietario o colaborador del proyecto.

Flujo de Proceso Principal	1. Para auditoría global: el Administrador ingresa a /admin/audit; el sistema consulta audit_logs con filtros de rango temporal (24h, 7d, 30d, Todo), usuario/IP y severidad, permitiendo exportar en Excel (.xlsx), CSV (UTF-8 BOM) y JSON. 2. Para trazabilidad del modelo: el Arquitecto abre el panel lateral de "Historial y Trazabilidad" en el editor; el sistema consulta diagram_history mostrando cada mutación con autor, timestamp y diff antes/después. 3. Para papelera: en el Dashboard, los proyectos con is_deleted = true pueden ser restaurados reversiblemente (restoreProject) o purgados definitivamente (hard delete) con eliminación física en cascada por su propietario.
Post-Condición	Consulta de bitácora forense sin alteración de datos, o proyecto restaurado/purgado físicamente.
Excepciones	E1. Intento de purga o eliminación por Super Admin: El backend rechaza la petición arrojando IllegalArgumentException ("El Administrador solo puede supervisar y restaurar proyectos, mas no eliminarlos").

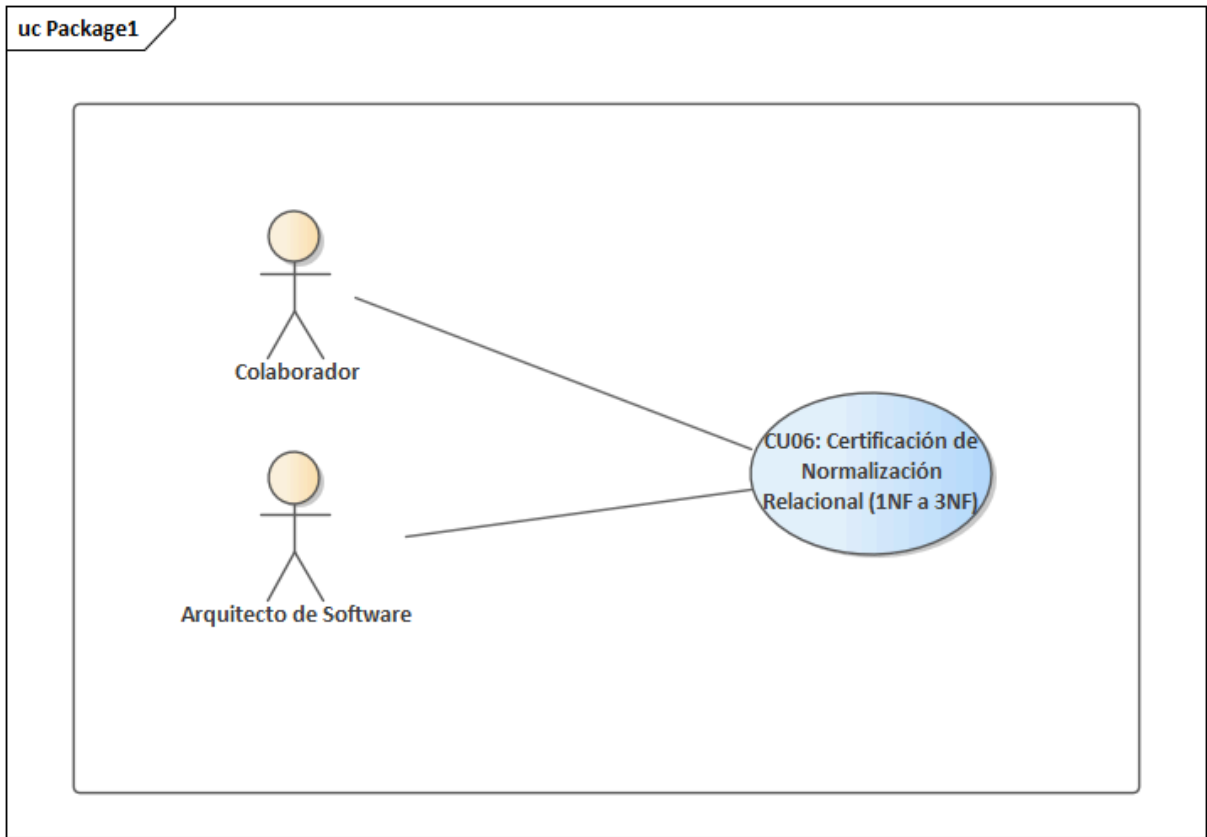
CU05: Modelado Canónico de Clases y Relaciones UML 2.5



Campo	Especificación Técnica
Propósito	Diseñar visualmente diagramas de clases estándar UML 2.5 en un lienzo grafocéntrico con nodos de 3 compartimentos, atributos tipados, badge {PK}, relaciones ortogonales sobre 4 handles magnéticos, prevención de ciclos DFS y soporte Ctrl+Z / Ctrl+Y.
Actores	A2 — Arquitecto de Software, A3 — Colaborador
Actor Iniciador	A2 — Arquitecto de Software / A3 — Colaborador
Pre-Condición	Proyecto abierto en el lienzo de diseño /editor/:id.

Flujo de Proceso Principal	1. El actor arrastra una nueva clase desde la barra de herramientas al canvas o hace clic en "Nueva Clase". 2. El sistema crea un nodo React Flow con 3 compartimentos: Nombre de la clase, Atributos y Métodos. 3. El actor edita la clase: añade atributos tipados (Java/SQL), visibilidad (+, -, #, ~), marca clave primaria {PK} y métodos. 4. Para conectar clases: el actor arrastra desde uno de los 4 handles magnéticos (top, bottom, left, right) hacia otra clase. 5. El sistema despliega el selector de relación: Asociación, Agregación, Composición, Herencia/Generalización o Dependencia, con cardinalidades estándar (1, 0..1, , 1..). 6. Si se traza una generalización, el algoritmo DFS verifica en tiempo real que no se generen ciclos circulares de herencia. 7. Cada acción se registra en la pila histórica de reversibilidad (deshacer con Ctrl+Z y rehacer con Ctrl+Y). 8. Al presionar "Guardar", se sincroniza el estado completo en class_nodes y relationships.
Post-Condición	Grafo UML validado y persistido transaccionalmente en PostgreSQL 17.
Excepciones	E1. Detección de ciclo de herencia: El sistema aborta la arista y notifica "Operación no permitida: Se detectó una dependencia circular de herencia".

CU06: Certificación de Normalización Relacional (1NF a 3NF)

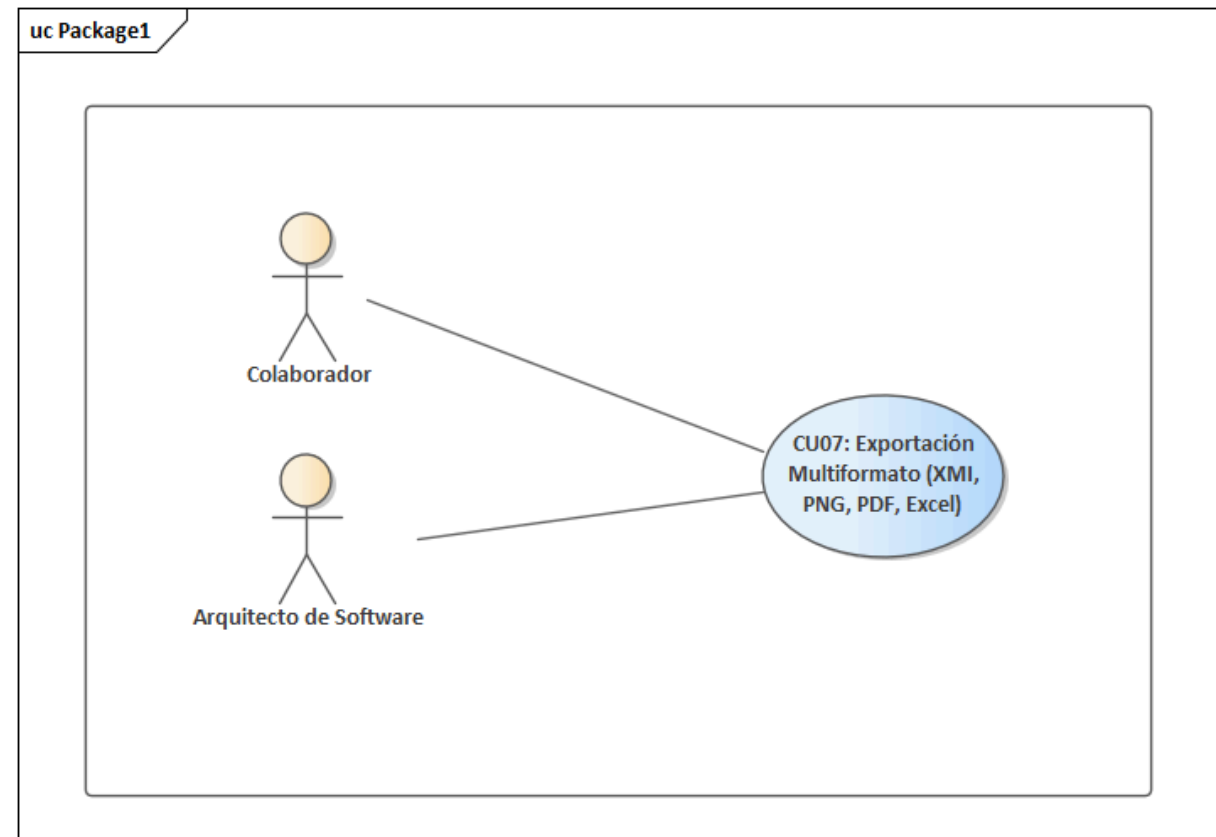


Campo	Especificación Técnica
Propósito	Evaluar el grafo de clases en tiempo real mediante un motor de inferencia relacional (teoremas de TOM y Codd) verificando 1NF (claves primarias), 2NF (claves foráneas) y 3NF (tablas intermedias en N:M), emitiendo un semáforo visual (0-100%) y certificación inmutable.
Actores	A2 — Arquitecto de Software, A3 — Colaborador
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Diagrama con al menos una clase modelada en el lienzo.

Flujo de Proceso Principal	1. El actor presiona el botón "Normalización Relacional (1NF-3NF)" en la barra lateral de herramientas. 2. El motor heurístico evalúa Primera Forma Normal (1NF): comprueba que cada clase posea al menos un atributo {PK} de tipo atómico no repetitivo. 3. El motor evalúa Segunda Forma Normal (2NF): verifica que en relaciones 1..* la entidad dependiente posea la clave foránea adecuada referenciando a la entidad principal. 4. El motor evalúa Tercera Forma Normal (3NF): valida que en relaciones *.* no existan dependencias transitivas directas y se proponga una entidad asociativa intermedia. 5. El sistema calcula el porcentaje de cumplimiento (0-100%) y despliega un panel flotante con recomendaciones de corrección automática. 6. Al alcanzar el 100%, el sistema habilita el botón "Certificar Modelo Relacional", registrando el hito en diagram_history.
Post-Condición	Modelo evaluado con semáforo relacional o certificado formalmente en base de datos.
Excepciones	E1. Modelo sin clases: El motor notifica "El diagrama debe contener clases para iniciar la auditoría relacional".

Ciclo #2

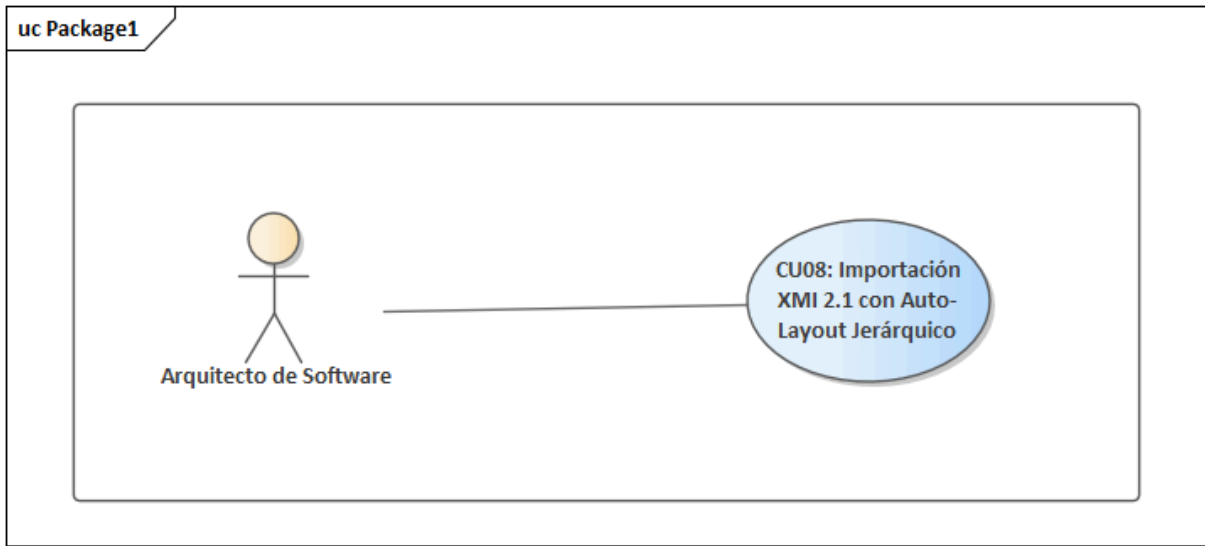
CU07: Exportación Multiformato (XMI, PNG, PDF, Excel)



Campo	Especificación Técnica
Propósito	Generar y descargar el modelo UML en cuatro formatos formales: archivo estándar internacional OMG XMI 2.1 (ArchiTec/StarUML), imagen gráfica Ultra-HD PNG (1x-3x), Memoria Técnica PDF ejecutiva con diagrama vectorial y tablas, y Libro Excel (.xlsx) estructurado en 3 hojas técnicas.
Actores	A2 — Arquitecto de Software, A3 — Colaborador
Actor Iniciador	A2 — Arquitecto de Software / A3 — Colaborador
Pre-Condición	Diagrama con clases guardadas en el proyecto actual.

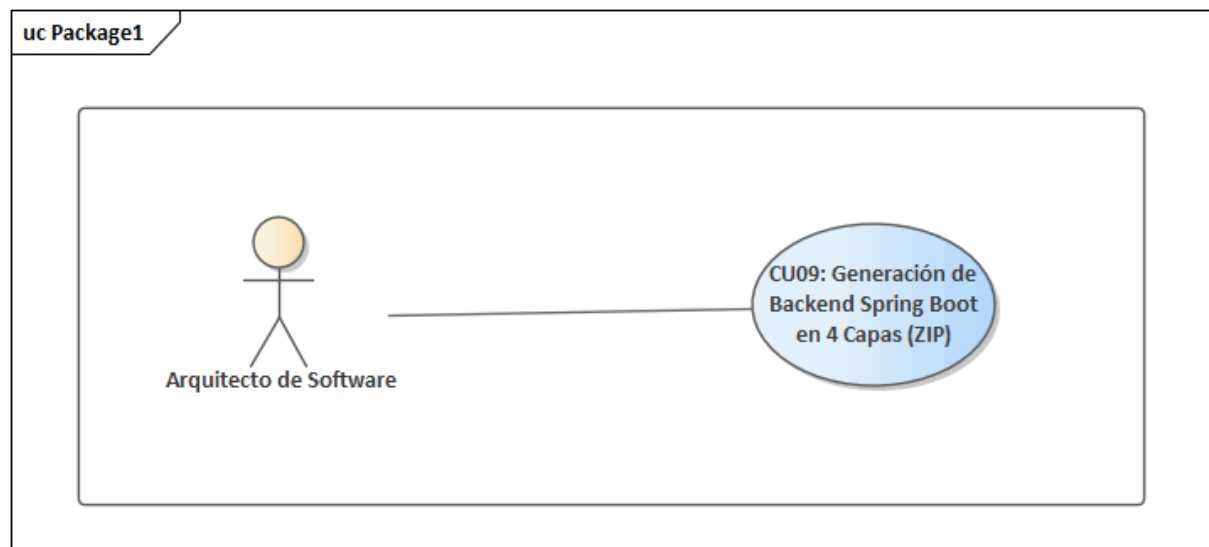
Flujo de Proceso Principal	1. El actor despliega el menú "Exportar" en el Toolbar lateral. 2. Para XMI 2.1: el backend serializa las clases, atributos, tipos y relaciones a XML bajo el estándar OMG XMI 2.1, descargando el archivo .xmi. 3. Para PNG UHD: el frontend calcula el bounding box de los nodos y renderiza mediante html-to-image en escala 1x, 2x o 3x. 4. Para Memoria Técnica PDF: el motor cliente genera un documento ejecutivo con portada formal, metadata del proyecto, captura del diagrama centrada y tablas de atributos/métodos por cada clase. 5. Para Libro Excel: el backend genera un archivo .xlsx estructurado en 3 hojas: "Resumen Ejecutivo", "Diccionario de Clases y Atributos (JPA/SQL)" y "Matriz de Relaciones y Cardinalidades".
Post-Condición	Archivo descargado en el equipo local del cliente sin alterar la persistencia del proyecto.
Excepciones	E1. Diagrama vacío: El sistema alerta "No hay elementos en el lienzo para exportar".

CU08: Importación XMI 2.1 con Auto-Layout Jerárquico



Campo	Especificación Técnica
Propósito	Deserializar archivos XML Metadata Interchange (XMI 2.1) generados por herramientas CASE externas (ArchiTec, Enterprise Architect, StarUML) mediante un parser seguro anti-XXE e inyectar el modelo en el canvas aplicando auto-layout jerárquico no colisionante.
Actores	A2 — Arquitecto de Software
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Archivo .xmi o .xml bien formado conforme al estándar OMG XMI 2.1.
Flujo de Proceso Principal	1. El Arquitecto selecciona la opción "Importar XMI" en el Toolbar lateral. 2. Carga el archivo .xmi desde su sistema de archivos local. 3. El backend procesa el documento con DocumentBuilderFactory configurado defensivamente contra ataques XXE (FEATURE_SECURE_PROCESSING, disallow-doctype-decl). 4. El parser extrae los elementos uml:Class, ownedAttribute, ownedOperation y uml:Association. 5. LayoutEngineUtil calcula las coordenadas (X, Y) de cada clase mediante distribución jerárquica por capas para evitar solapamientos. 6. El backend retorna el DTO estructurado y el frontend renderiza las clases y relaciones en el lienzo React Flow.
Post-Condición	Modelo importado, renderizado en el lienzo y persistido en la base de datos PostgreSQL 17.
Excepciones	E1. Archivo XML malformado o no conforme: El backend retorna HTTP 400 con el detalle del error de validación. E2. Detección de ataque XXE o DTD externo: El parser bloquea la lectura arrojando excepción de seguridad.

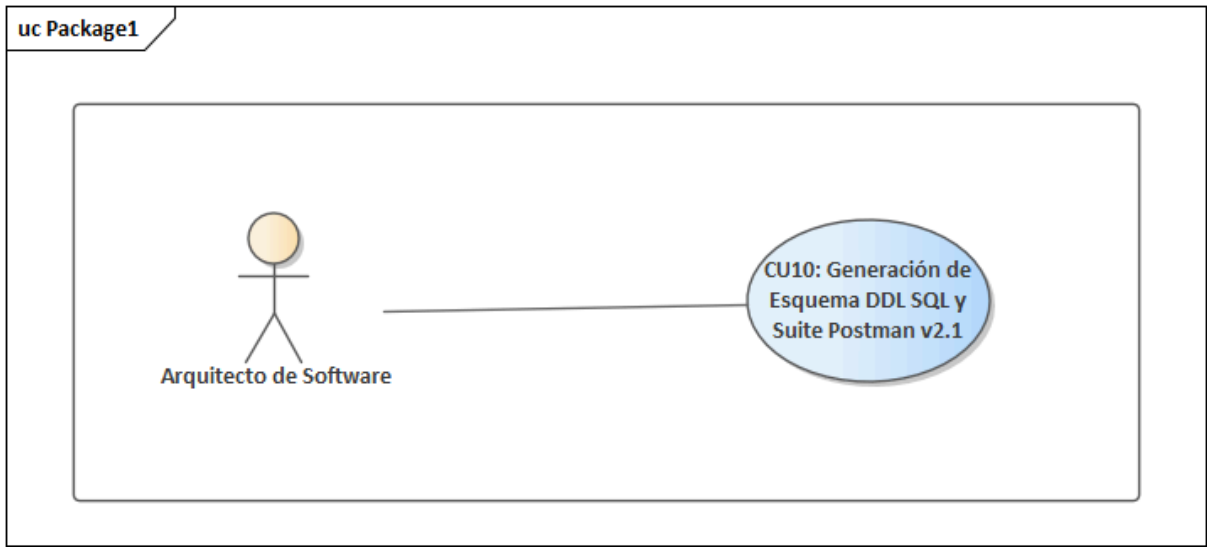
CU09: Generación de Backend Spring Boot en 4 Capas (ZIP)



Campo	Especificación Técnica
Propósito	Compilar el diagrama de clases UML a un proyecto backend ejecutable en Java 21 / Spring Boot 3.x estructurado en 4 capas desacopladas (Controller, Service, Repository, Entity), empaquetado en un archivo ZIP con Maven Wrapper (mvnw), Swagger OpenAPI y configuración de base de datos.
Actores	A2 — Arquitecto de Software
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Proyecto guardado con al menos una clase modelada y tipos de datos válidos.
Flujo de Proceso Principal	1. El actor presiona el botón "Generar Backend Spring Boot" en el Toolbar lateral. 2. El backend procesa el grafo de clases mediante el motor de plantillas FreeMarker (.ftl). 3. Genera la capa entity: clases Java con anotaciones JPA (@Entity, @Table, @Id, @GeneratedValue, @Column, @ManyToOne, @OneToMany). 4. Genera la capa repository: interfaces Spring Data JPA (JpaRepository<T, ID>). 5. Genera la capa service: interfaces e implementaciones @Service con transaccionalidad @Transactional. 6. Genera la capa controller: endpoints RESTful @RestController con documentación @Tag de OpenAPI. 7. Ensambla el archivo pom.xml, Maven Wrapper (mvnw, .mvn/), application.properties y la clase principal @SpringBootApplication. 8. Empaqueta el árbol de directorios en un flujo binario application/zip y lo transmite al navegador (< 500 ms).

Post-Condición	Archivo .zip descargado, listo para ser descomprimido y ejecutado con <code>./mvnw spring-boot:run</code> .
Excepciones	E1. Diagrama sin clases: El generador retorna HTTP 400 alertando la ausencia de entidades para compilar.

CU10: Generación de Esquema DDL SQL y Suite Postman v2.1

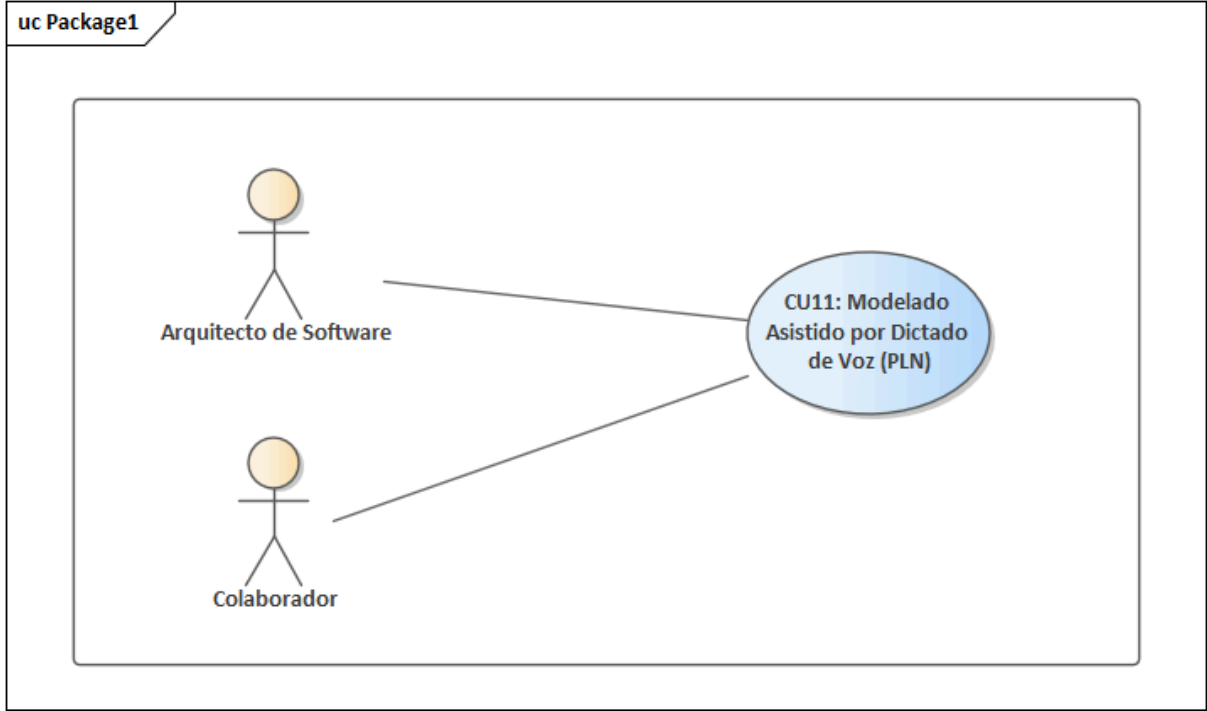


Campo	Especificación Técnica
Propósito	Generar el script DDL relacional para PostgreSQL 17 con restricciones de clave primaria, integridad referencial en cascada e índices B-Tree, junto con la colección de pruebas JSON Postman v2.1.0 preconfigurada con suite CRUD completa y aserciones automatizadas pm.test.
Actores	A2 — Arquitecto de Software
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Diagrama con clases y relaciones válidas en el proyecto.
Flujo de Proceso Principal	1. El actor selecciona "Generar Script DDL SQL" o "Generar Colección Postman" en el Toolbar lateral. 2. Para DDL SQL: el backend traduce las clases a sentencias CREATE TABLE, mapeando tipos Java a tipos nativos PostgreSQL (BIGINT, VARCHAR, TIMESTAMP, BOOLEAN), asignando claves primarias GENERATED BY DEFAULT AS IDENTITY, claves foráneas con ON DELETE CASCADE e índices sobre foreign keys. 3. Para Postman v2.1: el backend genera el archivo JSON con variables de entorno (baseUrl), carpetas organizadas por entidad y 5 peticiones CRUD tipadas (GET all, GET by id, POST create, PUT update, DELETE) con payloads mock realistas según el tipo de dato y tests automáticos pm.response.to.have.status(200). 4. El frontend despliega un visor modal con resaltado de sintaxis SQL/JSON y botones para copiar al portapapeles o descargar el archivo .sql o .postman_collection.json.
Post-Condición	Script DDL SQL y colección Postman generados y descargados.

Excepciones	E1. Tipos de datos incompatibles: El sistema aplica mapeo fallback a VARCHAR(255) para preservar la integridad del script.
-------------	--

Ciclo #3

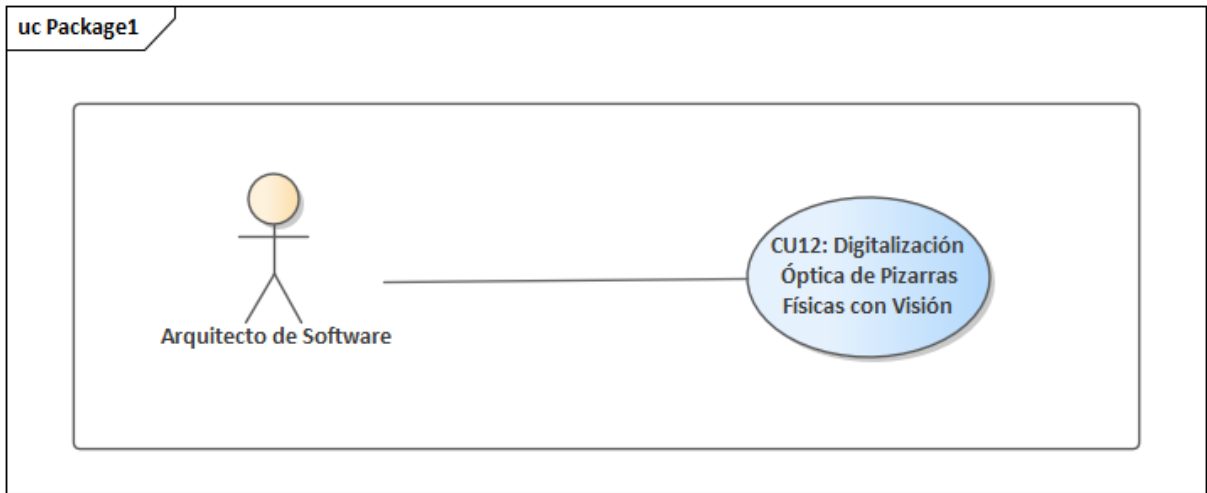
CU11: Modelado Asistido por Dictado de Voz (PLN)



Campo	Especificación Técnica
Propósito	Permitir el modelado y mutación incremental del diagrama mediante comandos de voz en lenguaje natural, transcribiendo con Web Speech API y procesando mediante Circuit Breaker en 4 niveles (Gemini, Groq, OpenRouter, Motor Local Offline), con reversibilidad total con Ctrl+Z.
Actores	A2 — Arquitecto de Software, A3 — Colaborador
Actor Iniciador	A2 — Arquitecto de Software / A3 — Colaborador
Pre-Condición	Micrófono habilitado en el navegador y proyecto abierto en el editor.

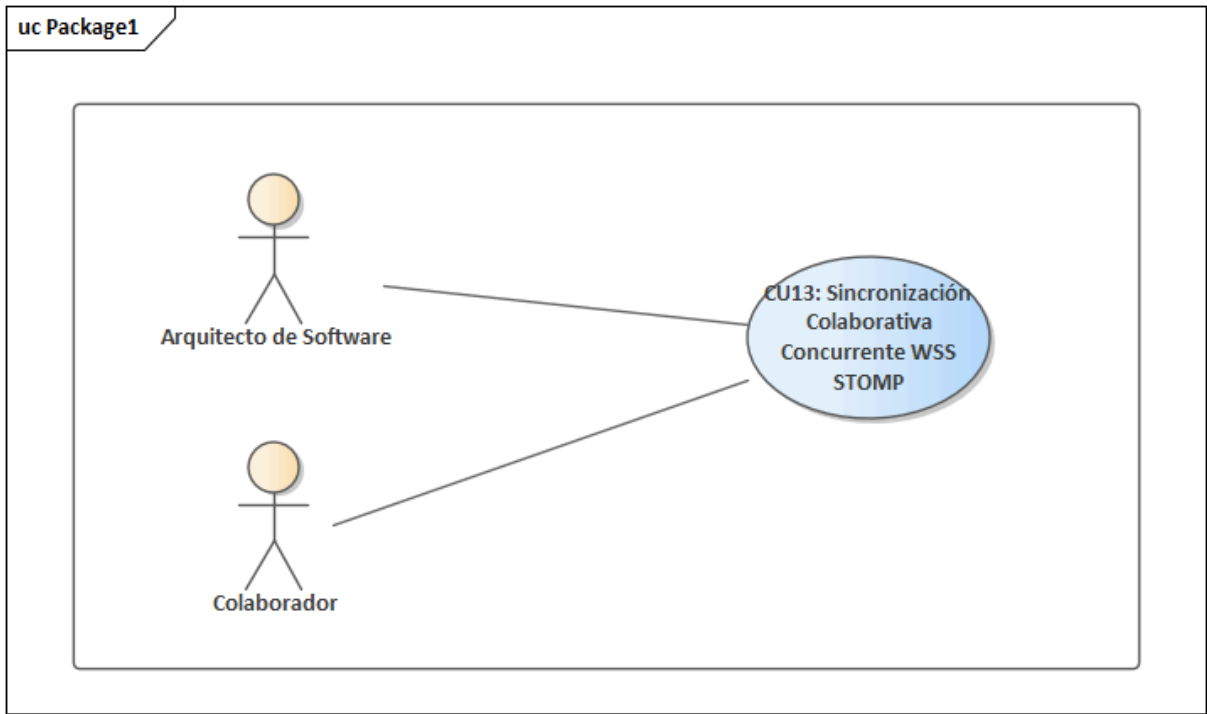
Flujo de Proceso Principal	1. El actor presiona el icono de micrófono en el dock flotante inferior del lienzo. 2. Dicta un comando en lenguaje natural (ej. "Crear una tabla Factura con id Long, total BigDecimal y fecha LocalDate, conectada con Cliente de muchos a uno"). 3. Web Speech API transcribe el audio a texto en tiempo real. 4. El backend evalúa la petición a través del Circuit Breaker: intenta Gemini 2.5 Flash; si falla por cuota (HTTP 429), conmuta a Groq Llama 3.3 70B, luego a OpenRouter y finalmente al Motor Local Regex Offline. 5. El motor extrae las acciones de mutación (creación de nodo, adición de atributos tipados, vinculación de arista) con defensa 1NF ({PK} id: Long). 6. El frontend aplica incrementalmente las mutaciones sobre el lienzo sin recargar ni alterar los elementos previos, permitiendo revertir con Ctrl+Z.
Post-Condición	Grafo UML enriquecido reactivamente según las órdenes de voz.
Excepciones	E1. Micrófono no autorizado: El sistema solicita permisos o permite la entrada manual mediante input de texto. E2. Ausencia total de internet: El Circuit Breaker activa el Motor Local Regex procesando el comando en < 5 ms.

CU12: Digitalización Óptica de Pizarras Físicas con Visión



Campo	Especificación Técnica
Propósito	Digitalizar diagramas de clases dibujados a mano alzada en pizarras o libretas mediante fotografía o archivo de imagen, procesándolos con LLMs multimodales bajo Circuit Breaker, aplicando auto-layout jerárquico e inyección preventiva de claves primarias 1NF.
Actores	A2 — Arquitecto de Software (Host)
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Imagen fotográfica de pizarra clara o cámara web disponible.
Flujo de Proceso Principal	1. El Arquitecto selecciona la herramienta "Digitalizar Pizarra con IA" en el dock flotante. 2. Captura una fotografía con la cámara o sube un archivo de imagen (PNG, JPG, WebP). 3. El backend codifica la imagen en Base64 y la somete al Circuit Breaker multimodal (Gemini 2.5 Flash \rightarrow Groq Llama 3.2 Vision). 4. El modelo de visión extrae los cuadros de clases, nombres, atributos, métodos y líneas conectoras con sus multiplicidades. 5. El backend valida defensivamente que cada entidad contenga su clave primaria atómica 1NF ({PK} id: Long). 6. El algoritmo de auto-layout jerárquico asigna coordenadas equilibradas para que los nodos no colisionen. 7. El frontend inyecta las clases y relaciones digitalizadas en el canvas, permitiendo editarlas de inmediato o deshacer con Ctrl+Z.
Post-Condición	Diagrama físico digitalizado y persistido como modelo UML interactivo.
Excepciones	E1. Imagen borrosa o incomprensible: El sistema solicita una nueva captura o complementación manual. E2. Cuota de API saturada: El Circuit Breaker conmuta al proveedor de visión secundario en menos de 200 ms.

CU13: Sincronización Colaborativa Concurrente WSS STOMP



Campo	Especificación Técnica
Propósito	Abrir una sesión de co-diseño multiusuario en tiempo real mediante WebSockets STOMP sobre SockJS (< 50 ms latencia), permitiendo el ingreso mediante código de sala y DNI/nombre, candados optimistas (element_locks), chat en vivo, cursores remotos y expulsión (kick) por el Host.
Actores	A2 — Arquitecto de Software (Host), A3 — Colaborador (Guest)
Actor Iniciador	A2 — Arquitecto de Software
Pre-Condición	Proyecto guardado y conexión de red activa.

Flujo de Proceso Principal	1. El Arquitecto (Host) presiona "Compartir en Vivo" en el Toolbar; el sistema genera una sala y un código alfanumérico único de 6 caracteres (ej. SW1-902). 2. El Colaborador (Guest) accede a /join, introduce el código de sala y su DNI y nombre completo. 3. El broker STOMP suscribe ambos clientes al tópico compartido /topic/room/{code}. 4. Cada movimiento de nodo, edición de texto o trazado de relación se retransmite en tiempo real (< 50 ms) a todos los participantes. 5. Cuando un usuario selecciona un nodo para editarlo, se propaga un candado optimista (element_locks), mostrando el nodo bloqueado con el avatar y color del usuario para evitar sobreescrituras. 6. Los participantes intercambian mensajes técnicos a través del panel de chat en vivo con aislamiento de teclado. 7. El Host cuenta con la facultad de expulsar (kick) a cualquier colaborador de la sesión. 8. Al cerrar la ventana, el evento beforeunload desconecta la sesión y libera los bloqueos y presencia.
Post-Condición	Sesión colaborativa sincronizada, con presencia en tiempo real y registro en session_participants.
Excepciones	E1. Código de sala inexistente: El sistema alerta "Sala no encontrada o sesión colaborativa finalizada". E2. Pérdida de conexión de red: SockJS activa reconexión automática con backoff exponencial.

3.1.7) Estructurar Modelo de Casos de Uso

Diagrama de Casos de Uso — Ciclo #1

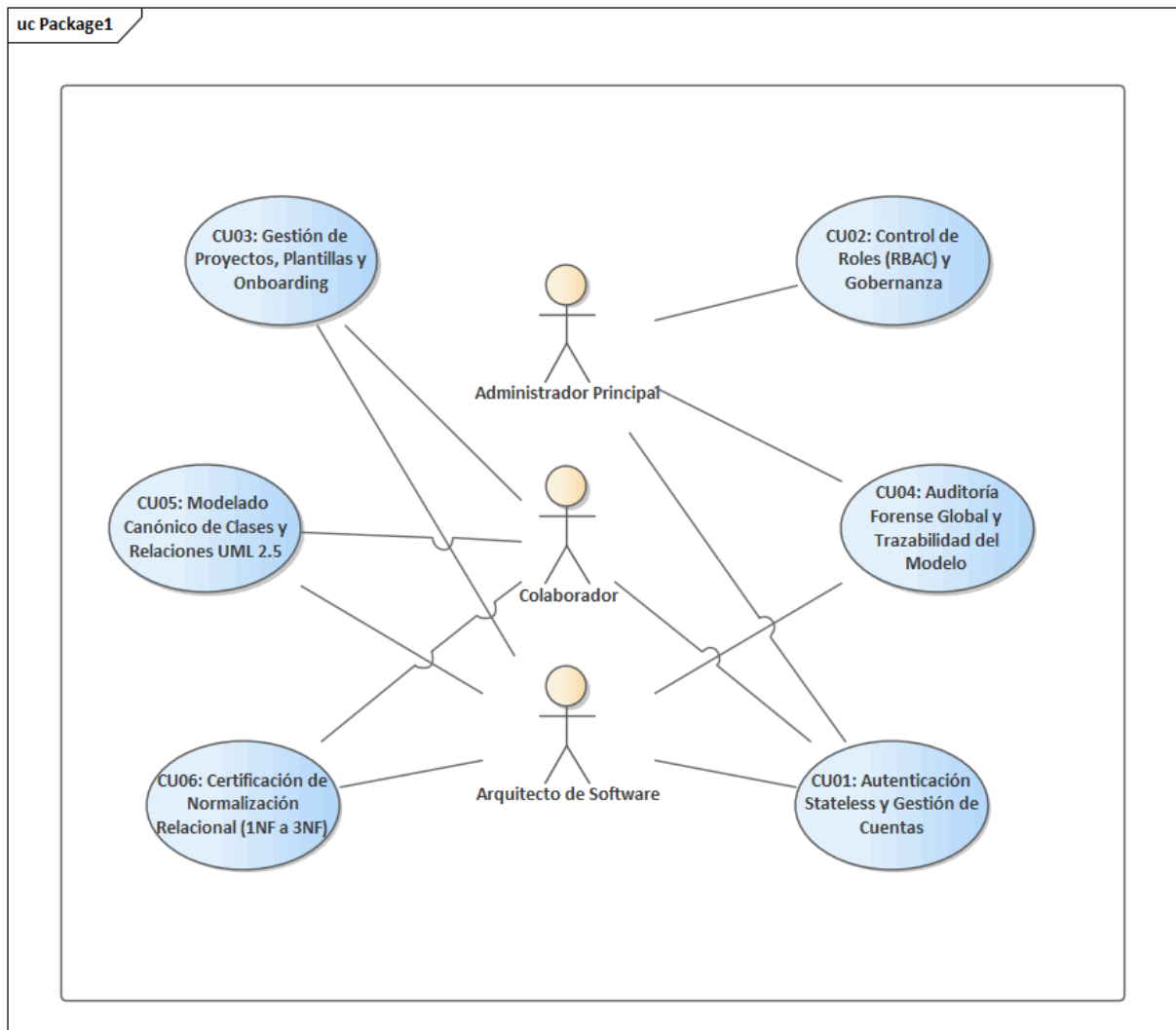


Diagrama de Casos de Uso — Ciclo #2

uc Package1

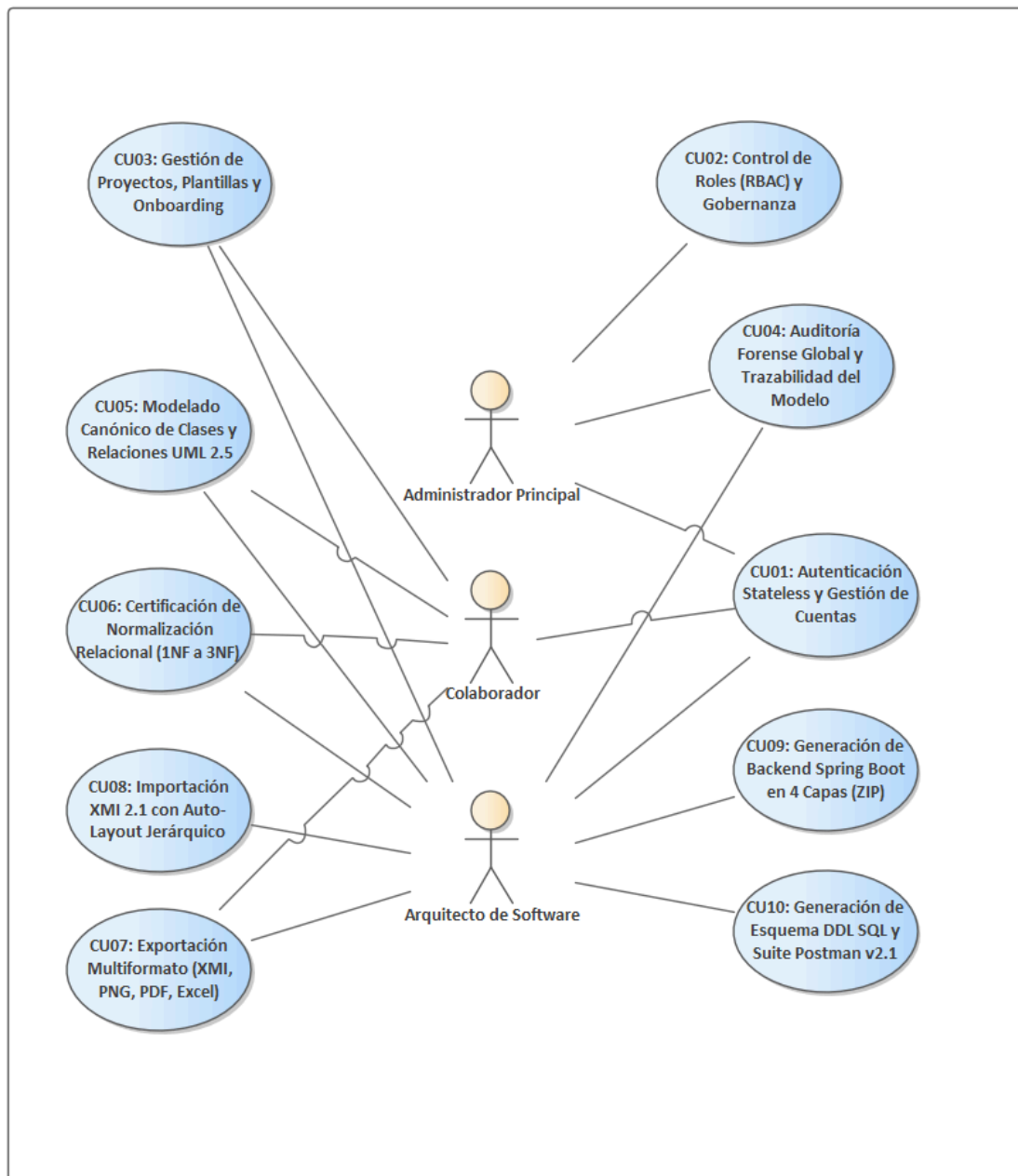
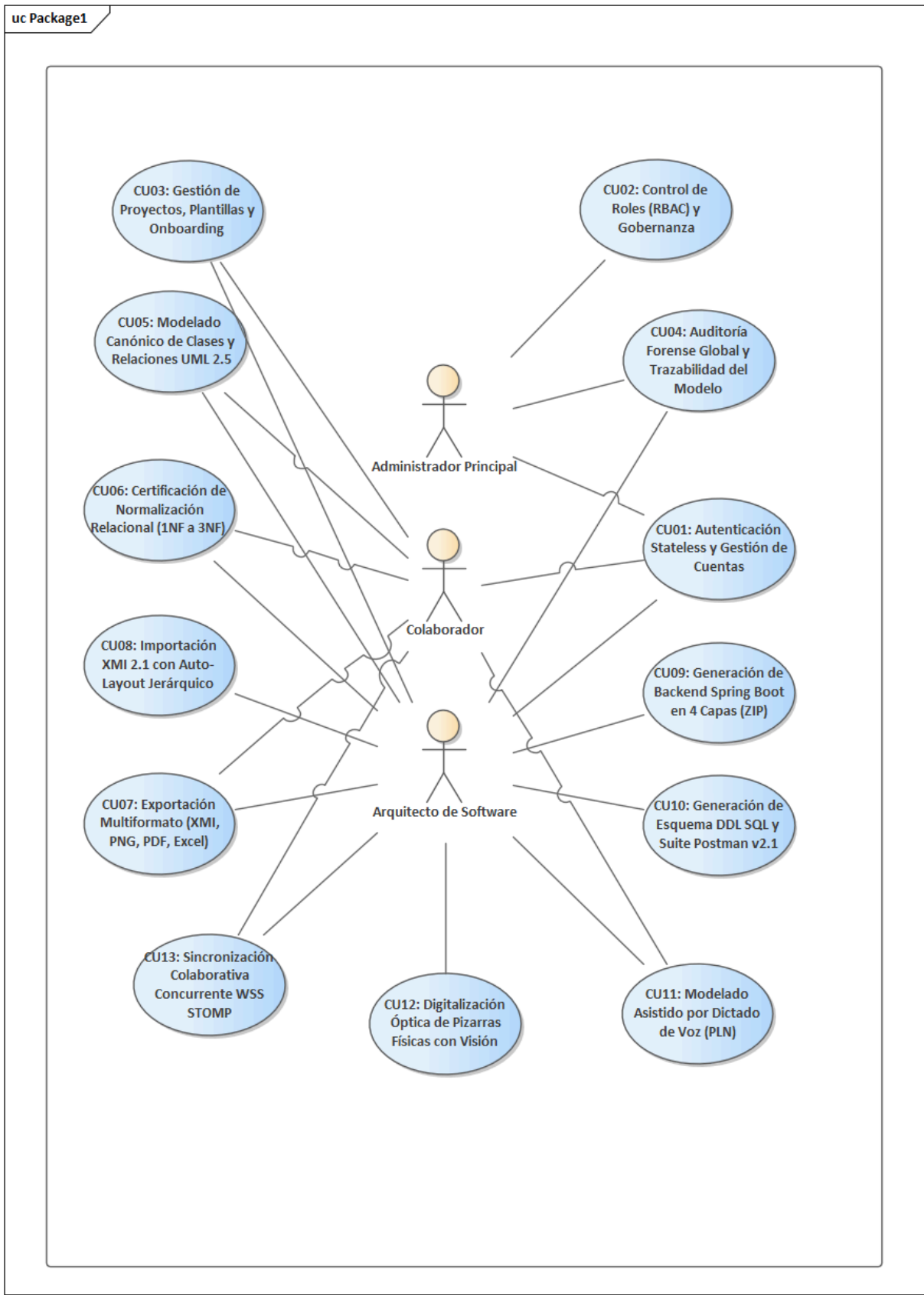
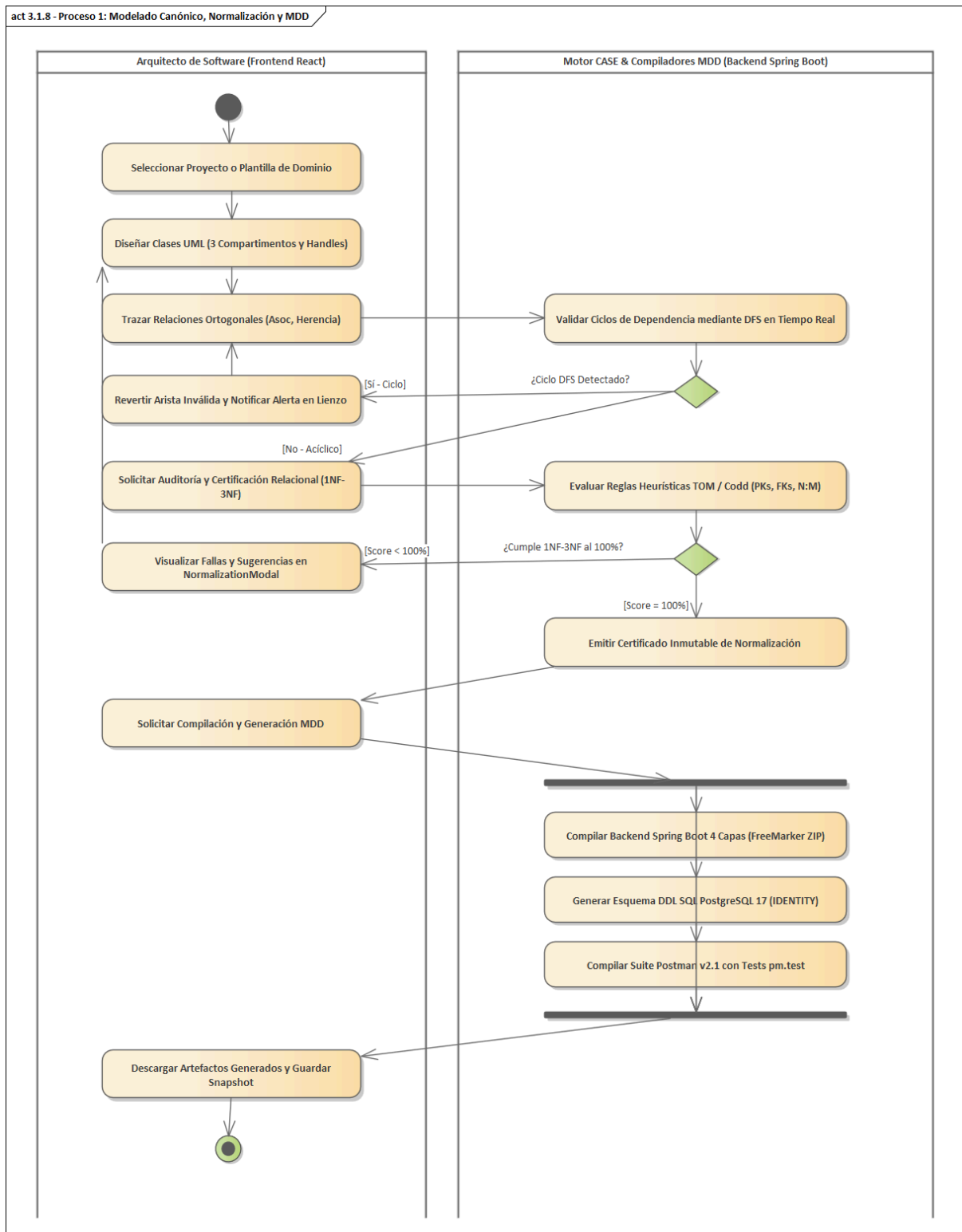


Diagrama de Casos de Uso — Ciclo #3

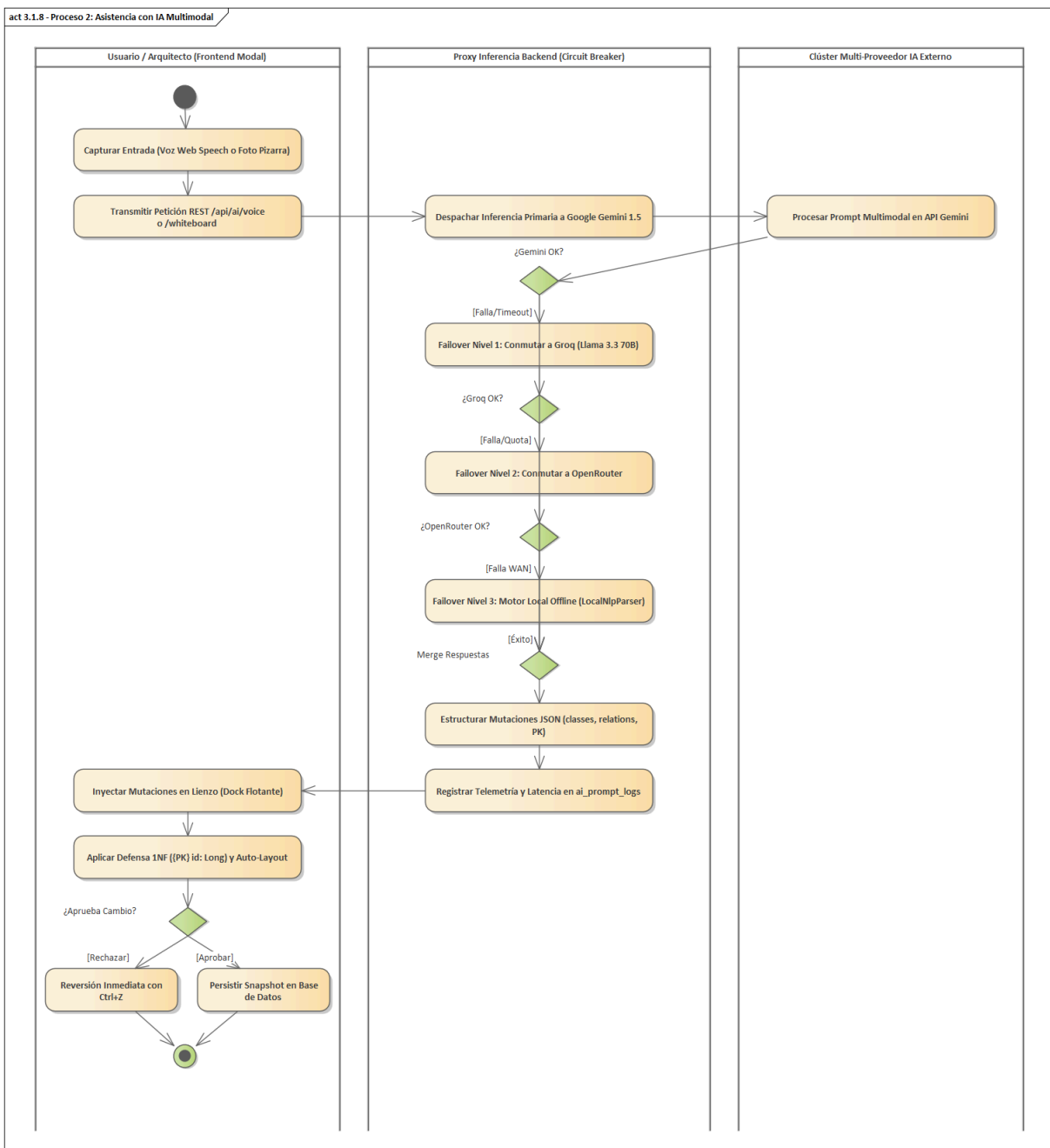


3.1.8) Modelado de Procesos de Negocio (Diagramas de Actividad)

Proceso 1: Flujo de Modelado Canónico, Certificación Relacional y Generación MDD

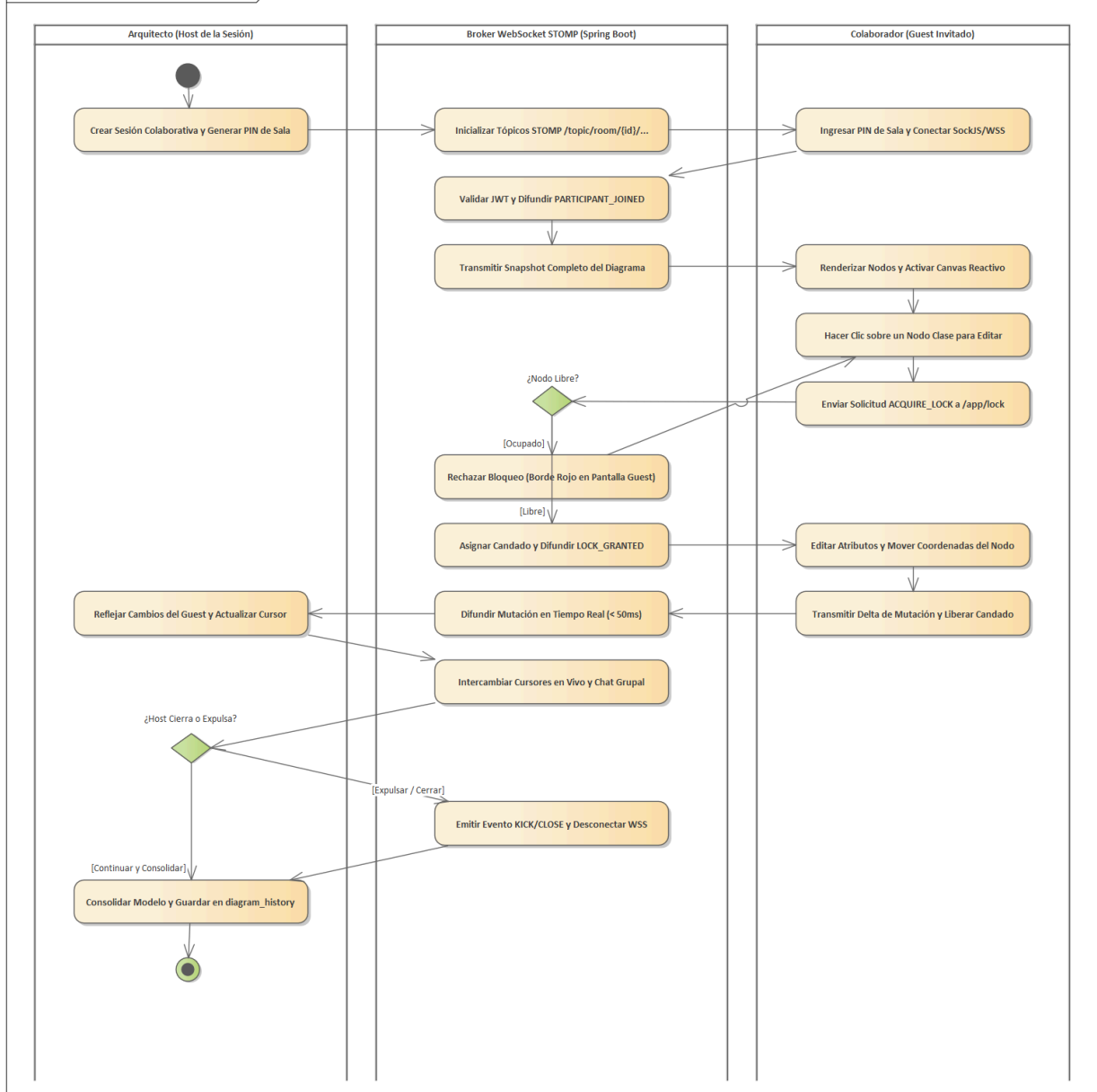


Proceso 2: Flujo de Asistencia con IA Multimodal



Proceso 3: Flujo de Co-Diseño Colaborativo Concurrente en Tiempo Real

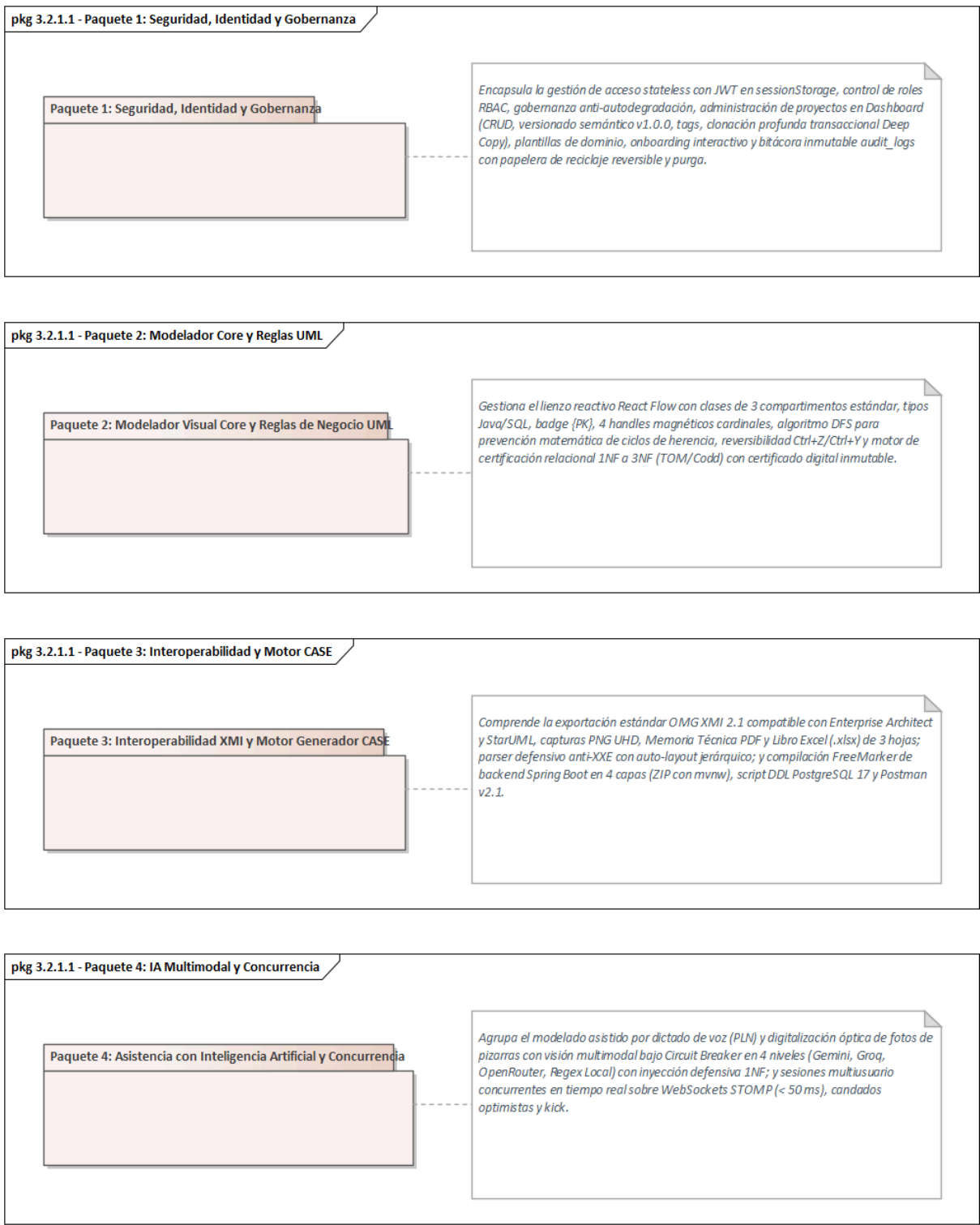
act 3.1.8 - Proceso 3: Co-Diseño Colaborativo Concurrente



3.2) Flujo de Trabajo: Análisis

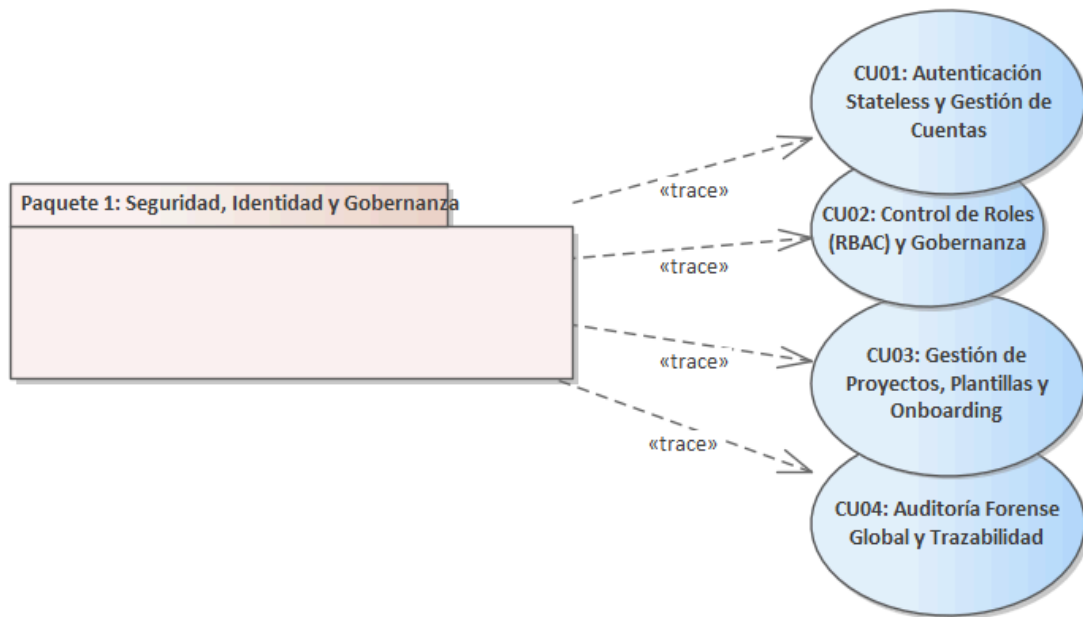
3.2.1) Análisis de la arquitectura

3.2.1.1) Identificar paquetes

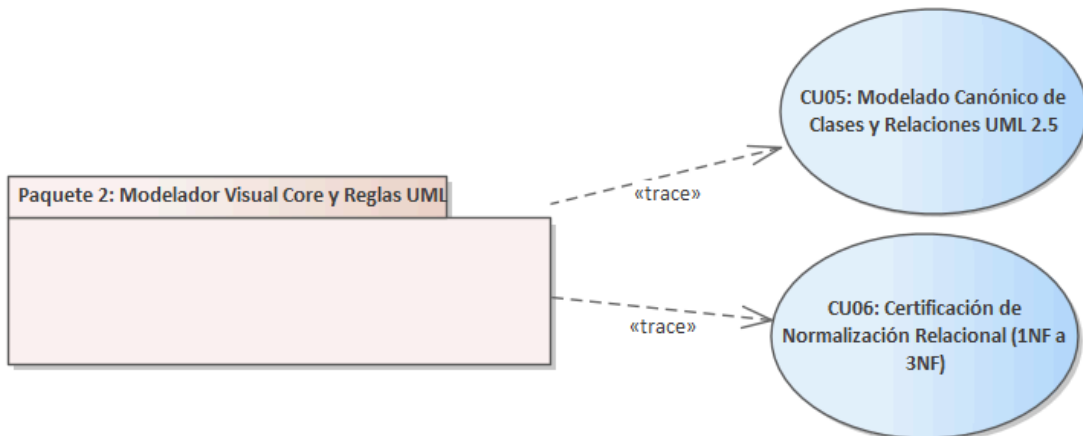


3.2.1.2) Relacionar paquetes

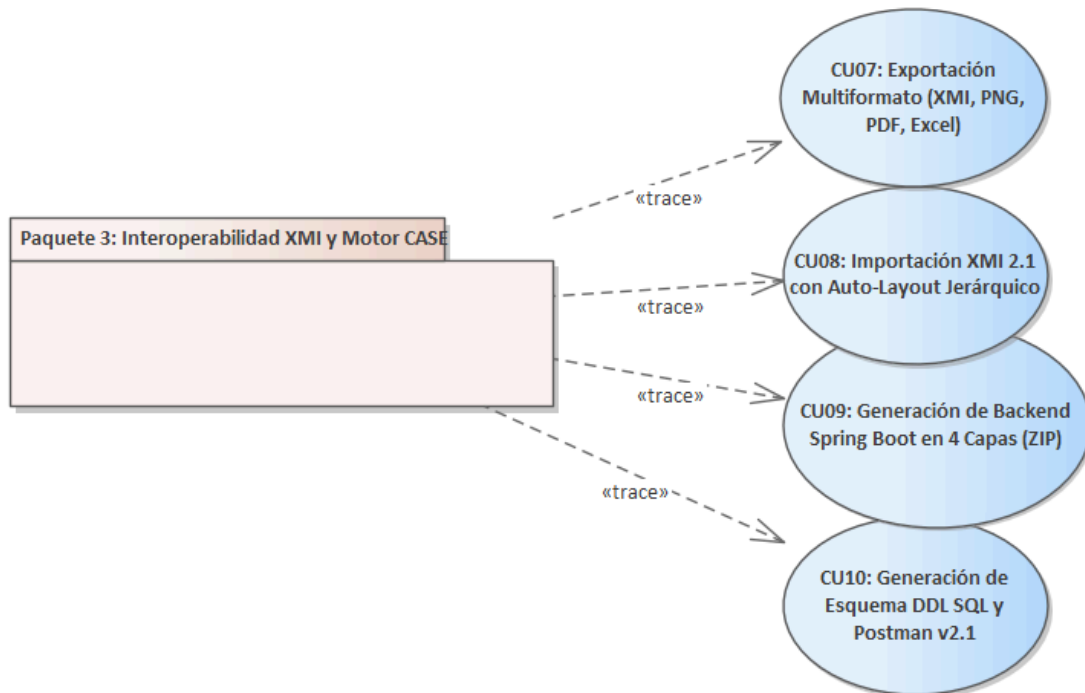
uc 3.2.1.2 - Trazabilidad Paquete 1: Seguridad



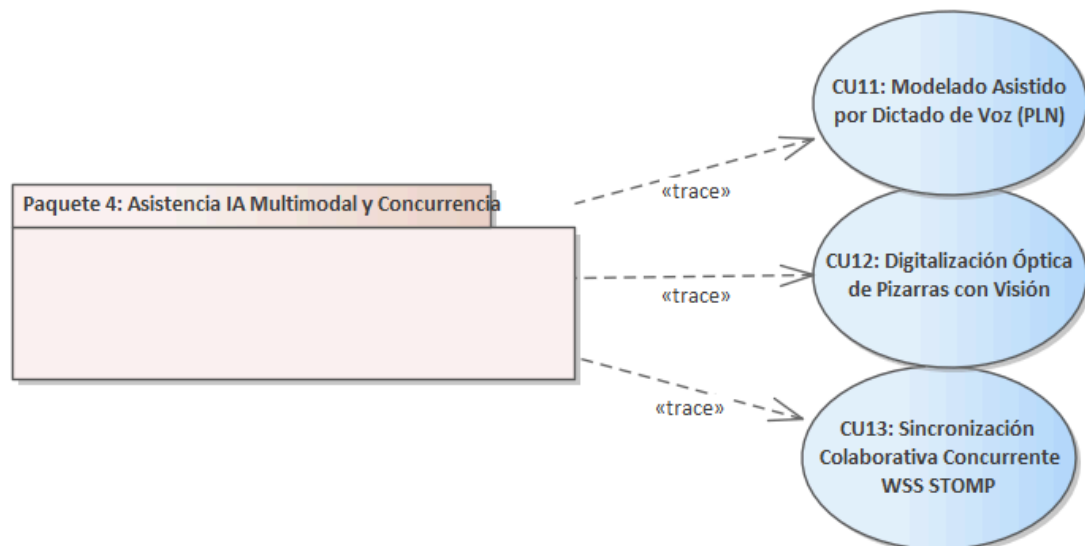
uc 3.2.1.2 - Trazabilidad Paquete 2: Modelador Core



uc 3.2.1.2 - Trazabilidad Paquete 3: Interoperabilidad

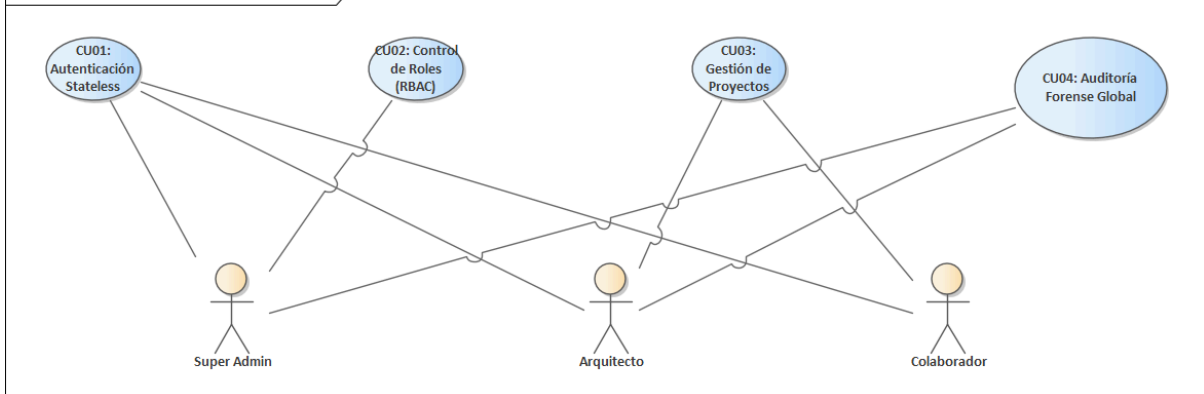


uc 3.2.1.2 - Trazabilidad Paquete 4: IA y Concurrencia

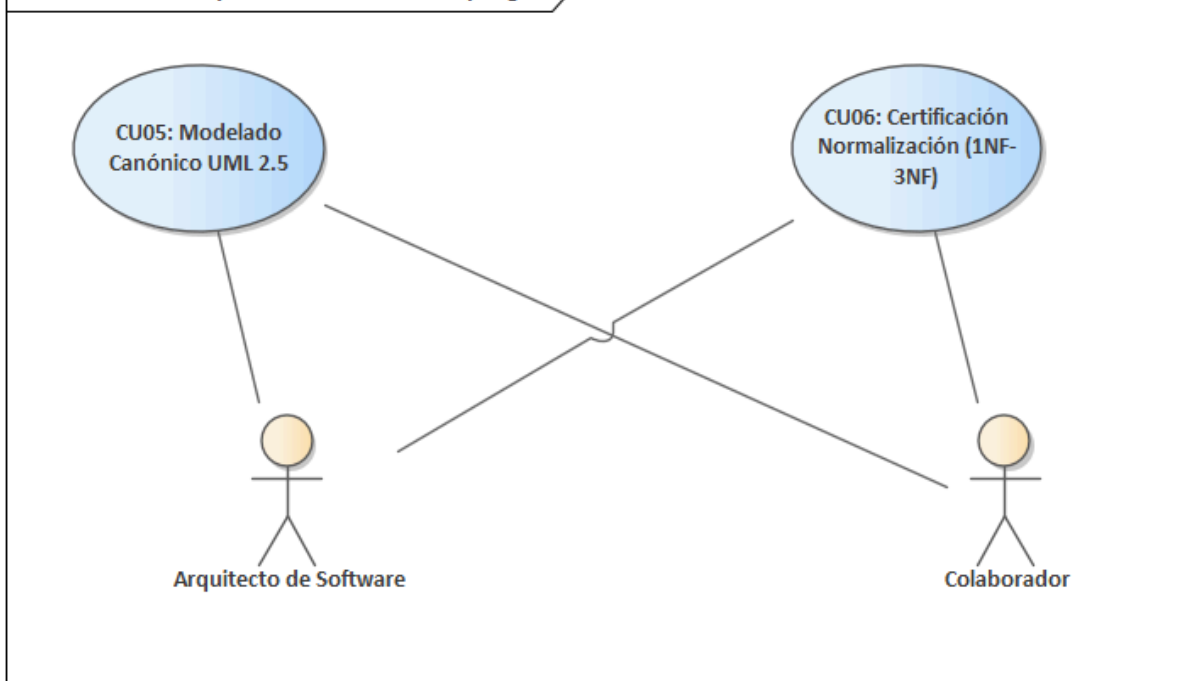


3.2.1.3) Vista de paquetes

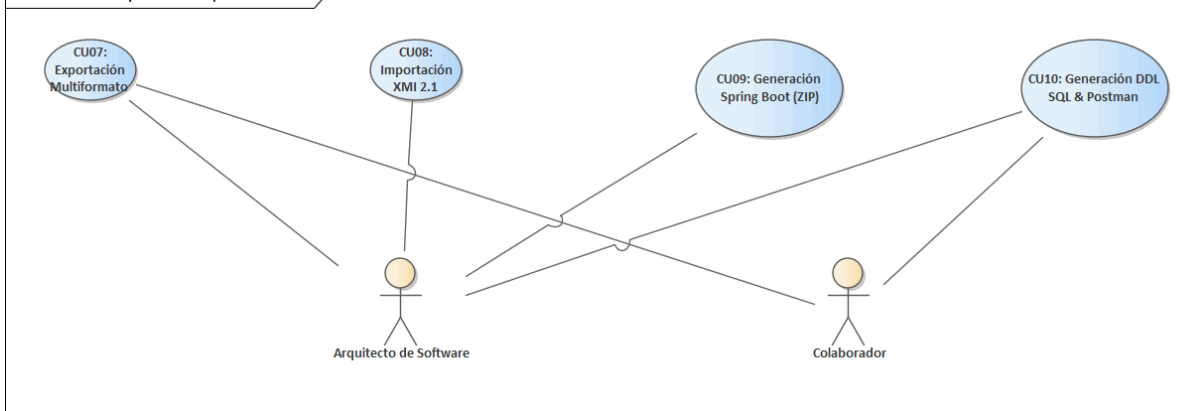
uc 3.2.1.3 - Vista Paquete 1: Seguridad y Gobernanza



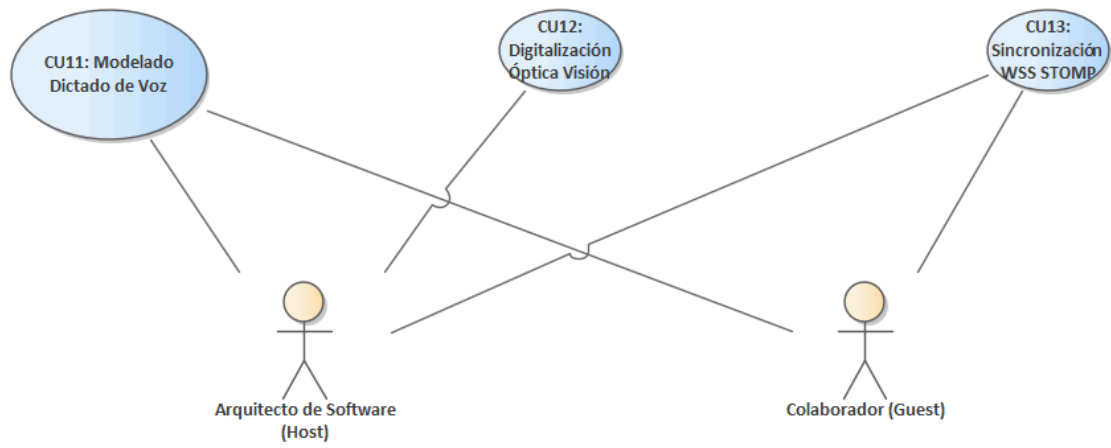
uc 3.2.1.3 - Vista Paquete 2: Modelador Core y Reglas



uc 3.2.1.3 - Vista Paquete 3: Interoperabilidad CASE



uc 3.2.1.3 - Vista Paquete 4: IA y Concurrencia



3.2.2) Análisis de casos de uso

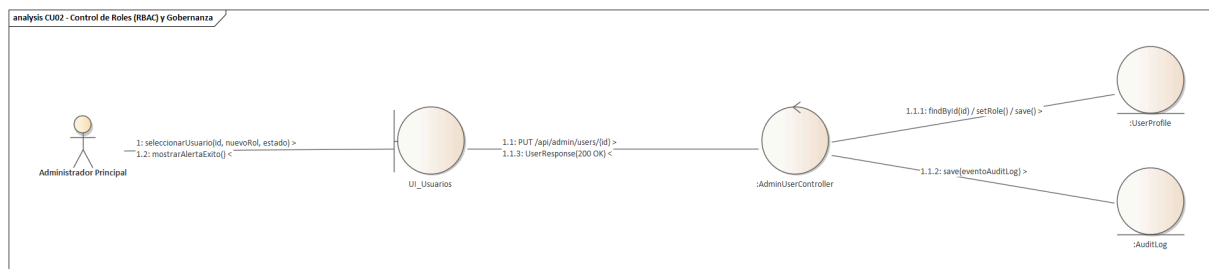
Diagrama de comunicación

CICLO 1

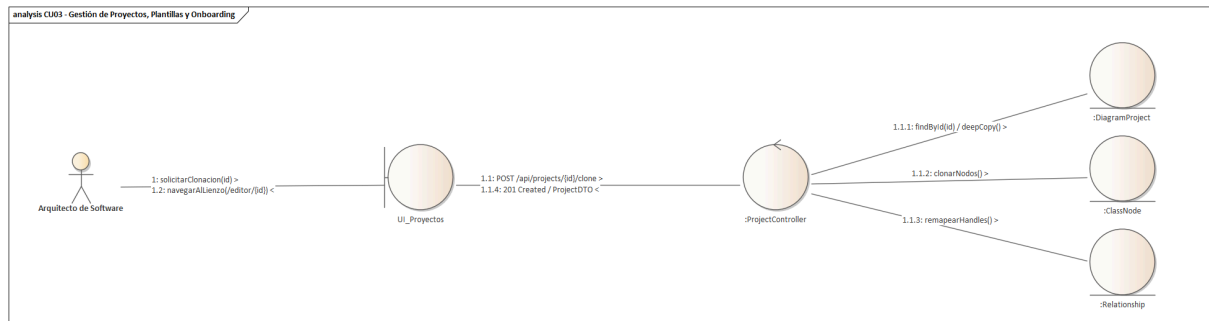
CU01 - Autenticación Stateless y Gestión de Cuentas



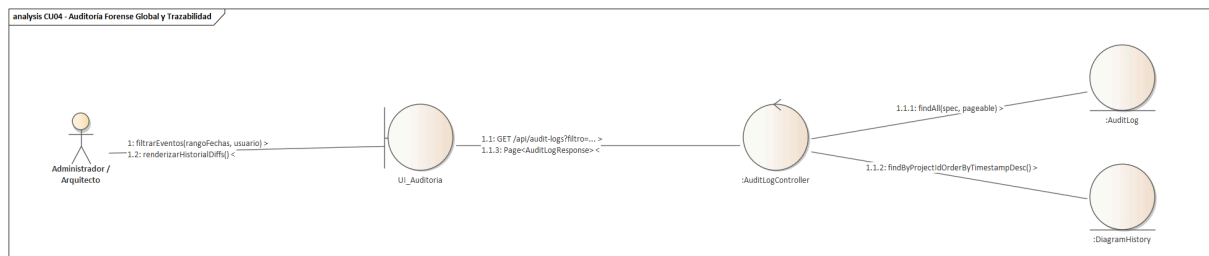
CU02 - Control de Roles (RBAC) y Gobernanza



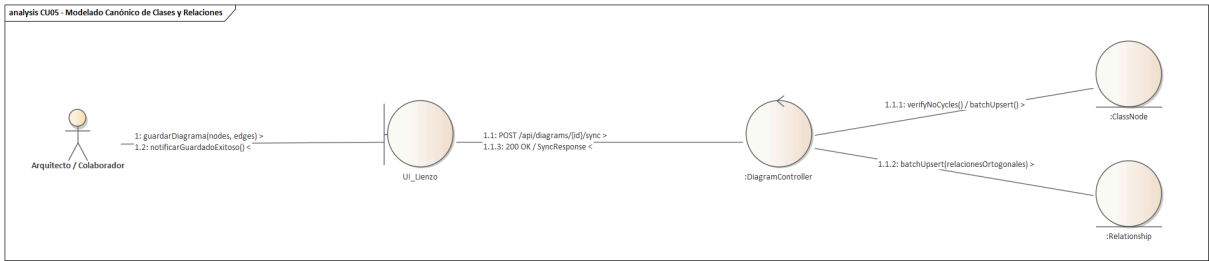
CU03 - Gestión de Proyectos, Plantillas y Onboarding



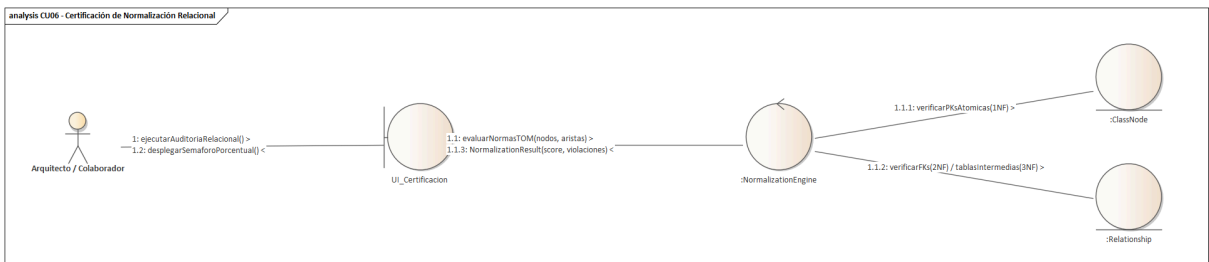
CU04 - Auditoría Forense Global y Trazabilidad del Modelo



CU05 - Modelado Canónico de Clases y Relaciones UML 2.5

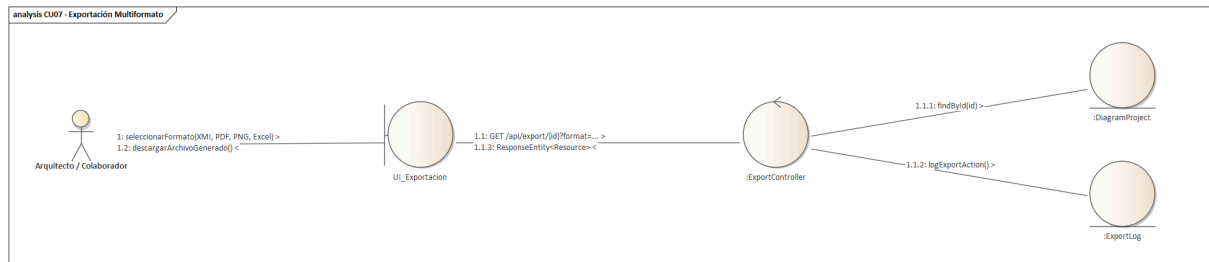


CU06 - Certificación de Normalización Relacional (1NF a 3NF)

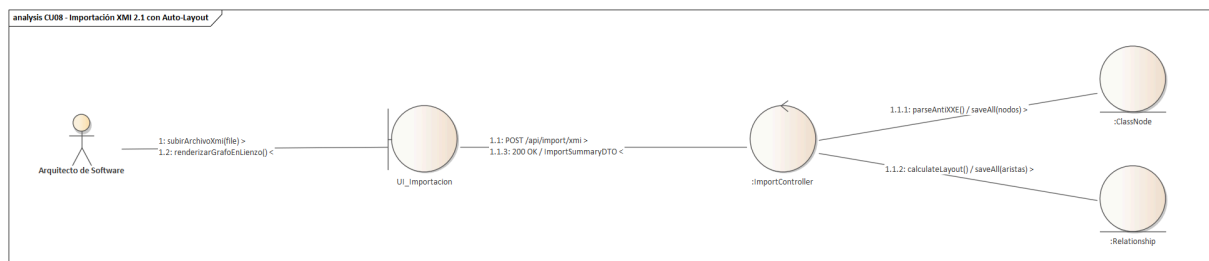


CICLO 2

CU07 - Exportación Multiformato (XMI, PNG, PDF, Excel)



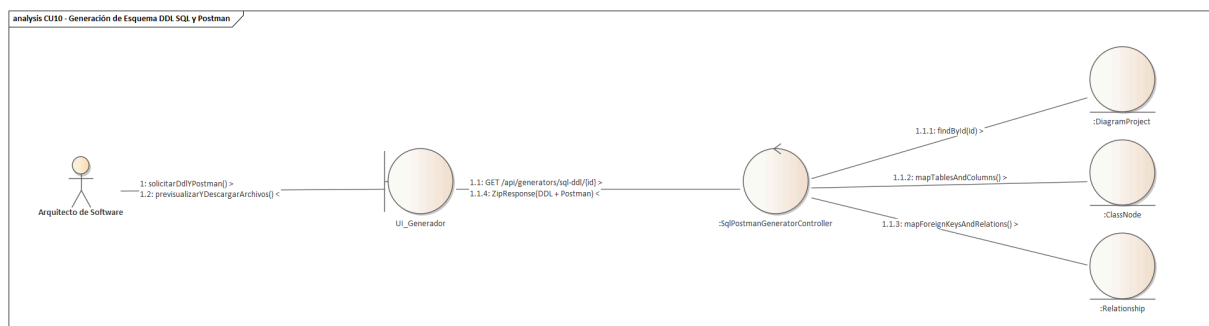
CU08 - Importación XMI 2.1 con Auto-Layout Jerárquico



CU09 - Generación de Backend Spring Boot en 4 Capas (ZIP)

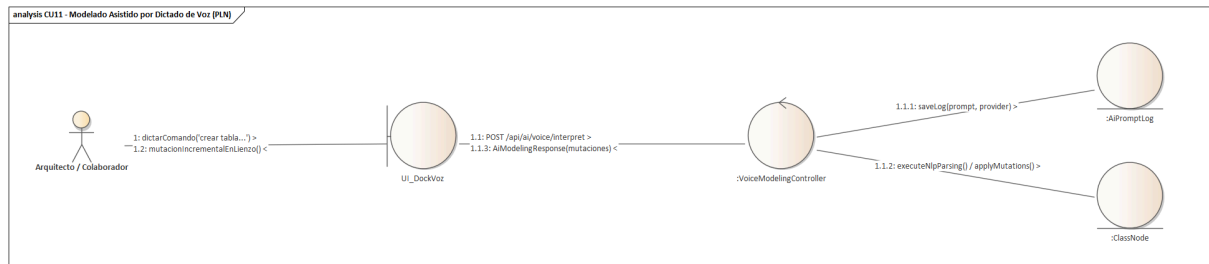


CU10 - Generación de Esquema DDL SQL y Suite Postman v2.1



CICLO 3

CU11 - Modelado Asistido por Dictado de Voz (PLN)



CU12 - Digitalización Óptica de Pizarras Físicas con Visión



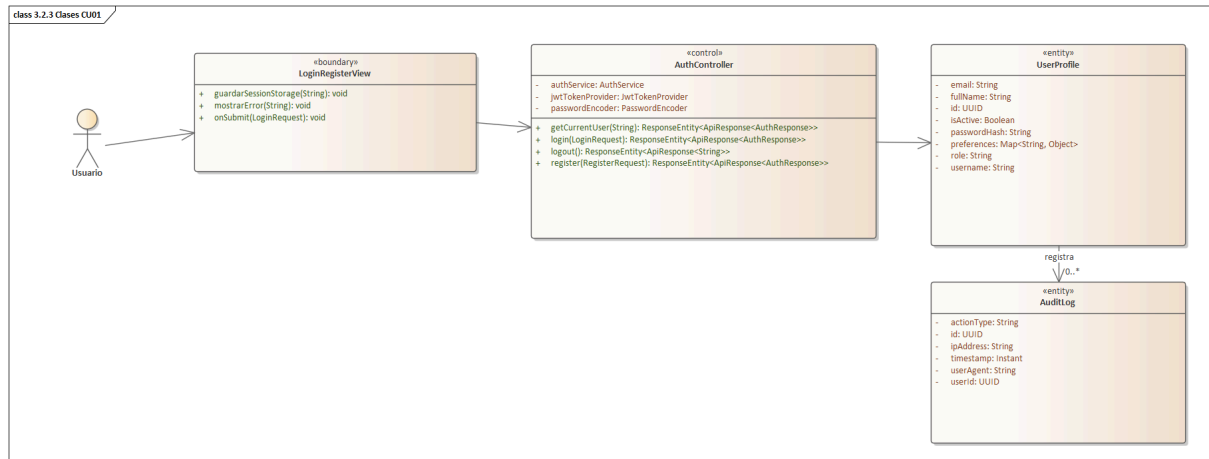
CU13 - Sincronización Colaborativa Concurrente WSS STOMP



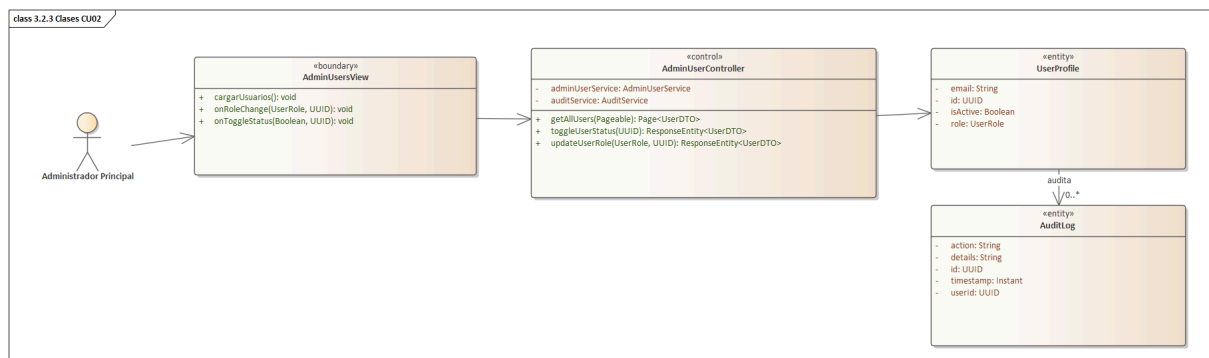
3.2.3) Análisis de clase

CICLO 1

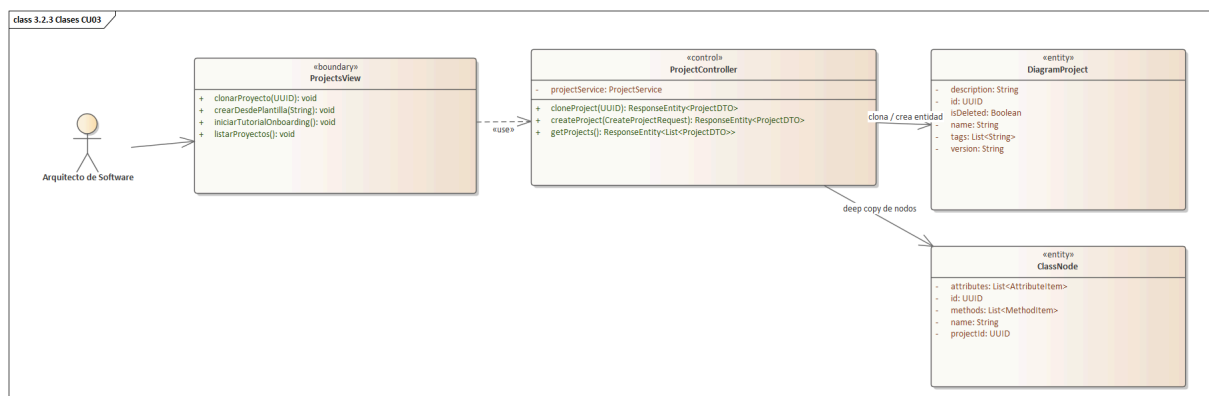
CU01 - Autenticación Stateless y Gestión de Cuentas



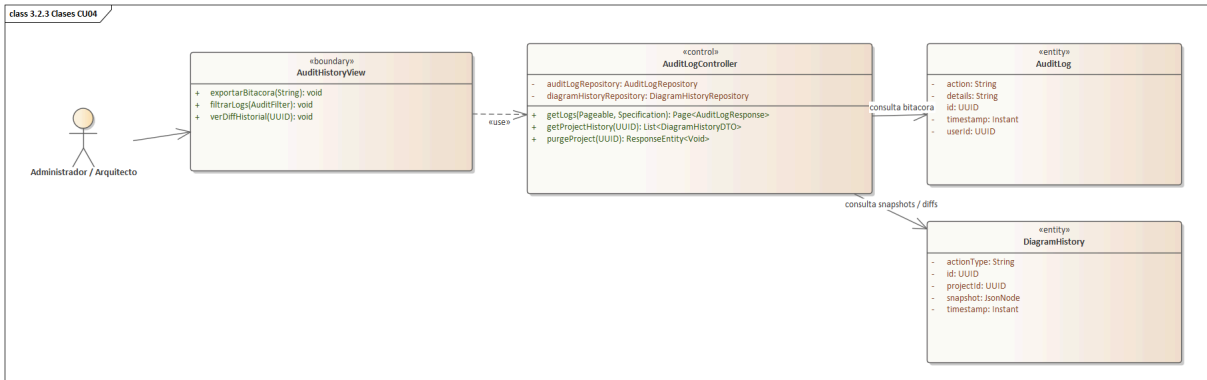
CU02 - Control de Roles (RBAC) y Gobernanza



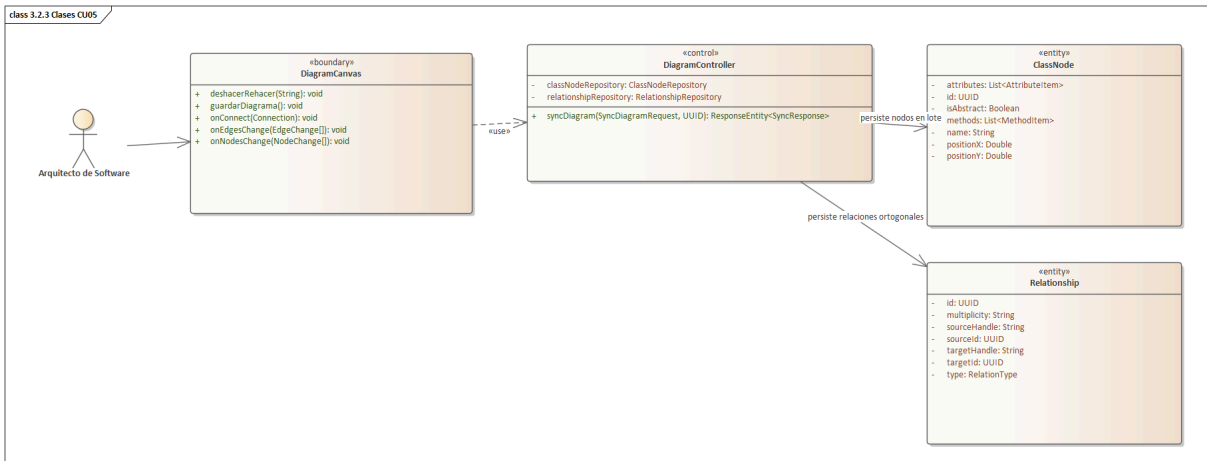
CU03 - Gestión de Proyectos, Plantillas y Onboarding



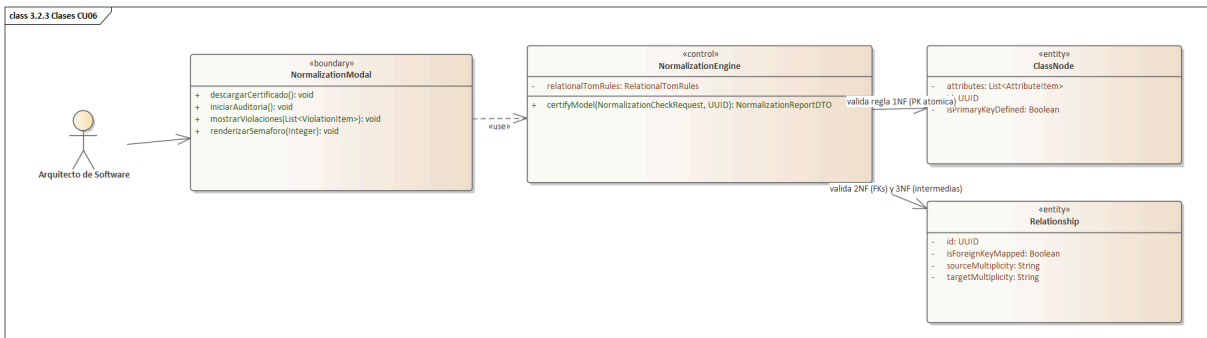
CU04 - Auditoría Forense Global y Trazabilidad del Modelo



CU05 - Modelado Canónico de Clases y Relaciones UML 2.5

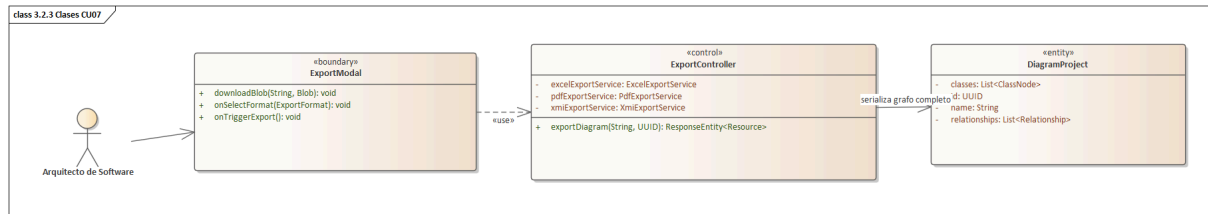


CU06 - Certificación de Normalización Relacional (1NF a 3NF)

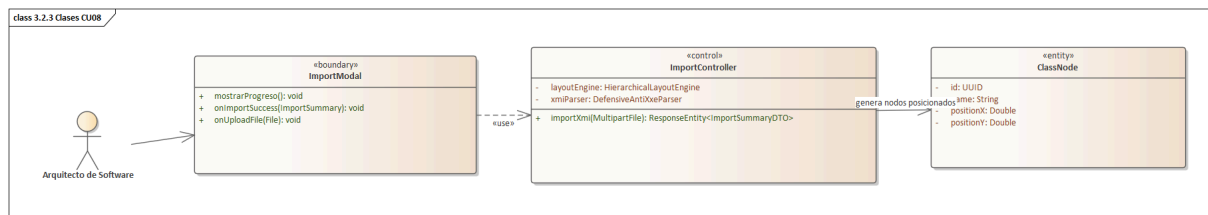


CICLO 2

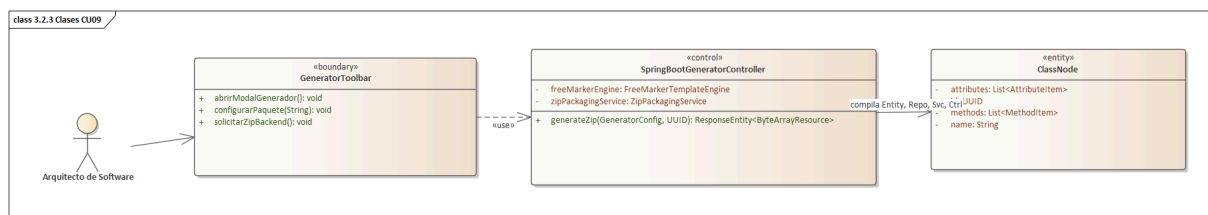
CU07 - Exportación Multiformato (XMI, PNG, PDF, Excel)



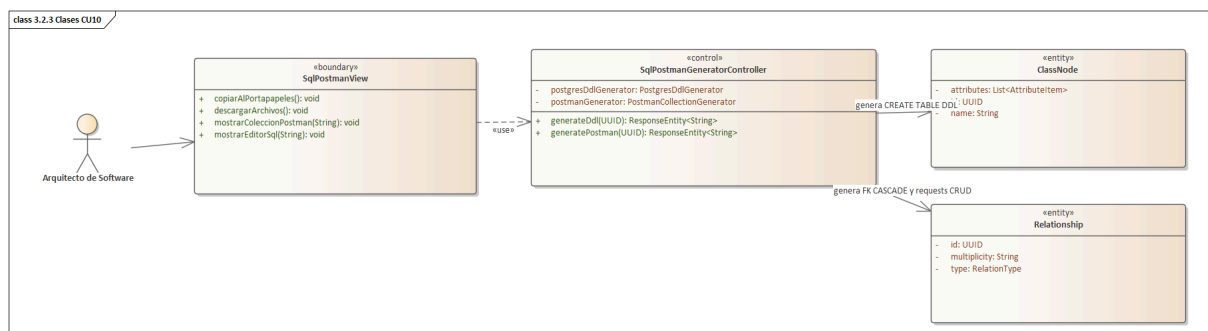
CU08 - Importación XMI 2.1 con Auto-Layout Jerárquico



CU09 - Generación de Backend Spring Boot en 4 Capas (ZIP)

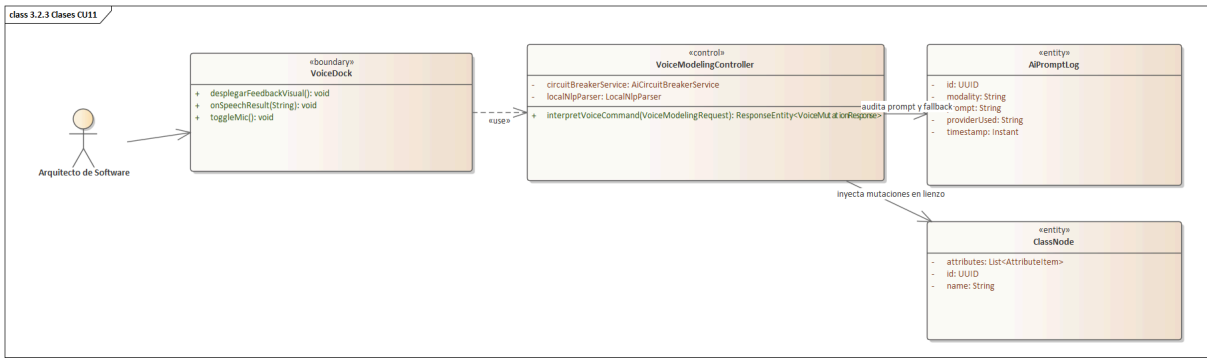


CU10 - Generación de Esquema DDL SQL y Suite Postman v2.1

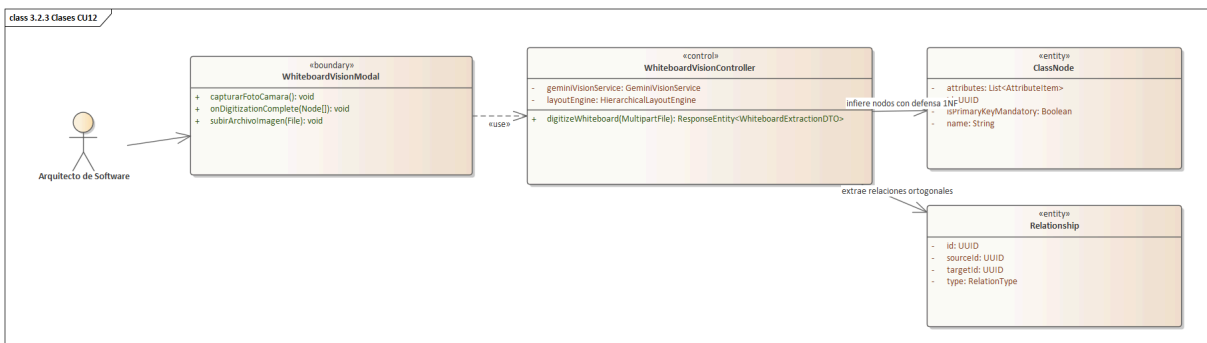


CICLO 3

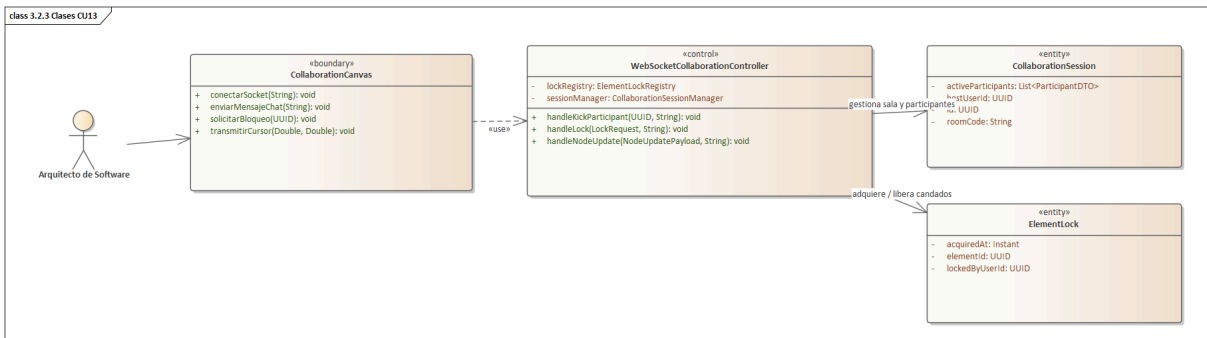
CU11 - Modelado Asistido por Dictado de Voz (PLN)



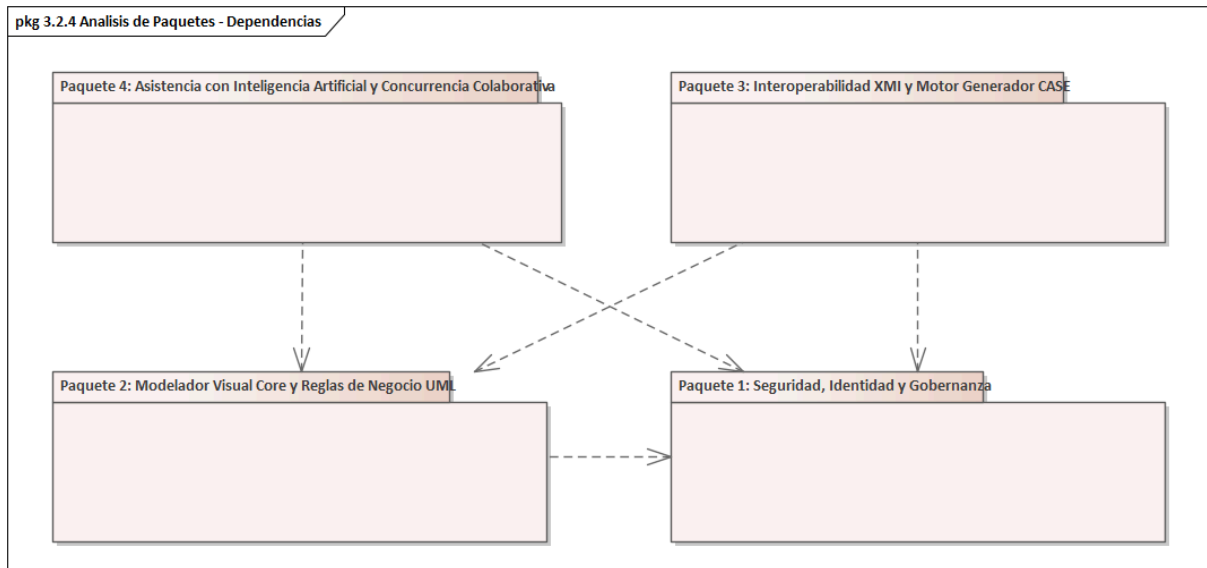
CU12 - Digitalización Óptica de Pizarras Físicas con Visión



CU13 - Sincronización Colaborativa Concurrente WSS STOMP

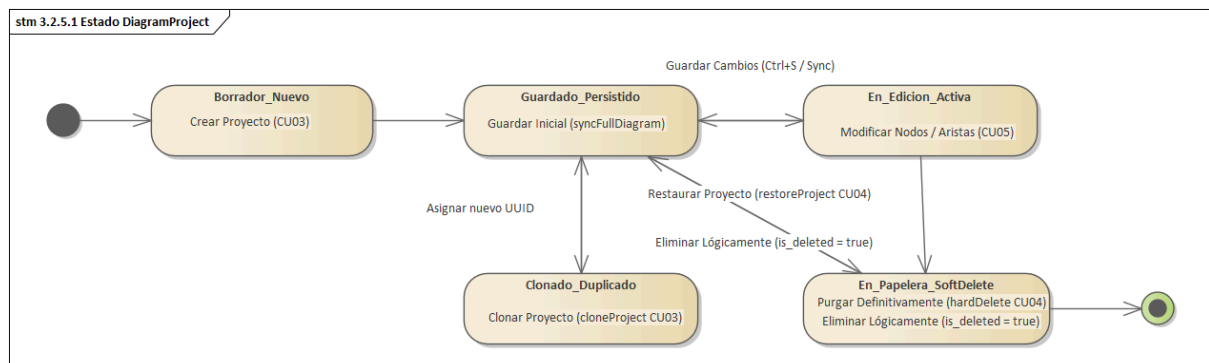


3.2.4) Analizar paquete

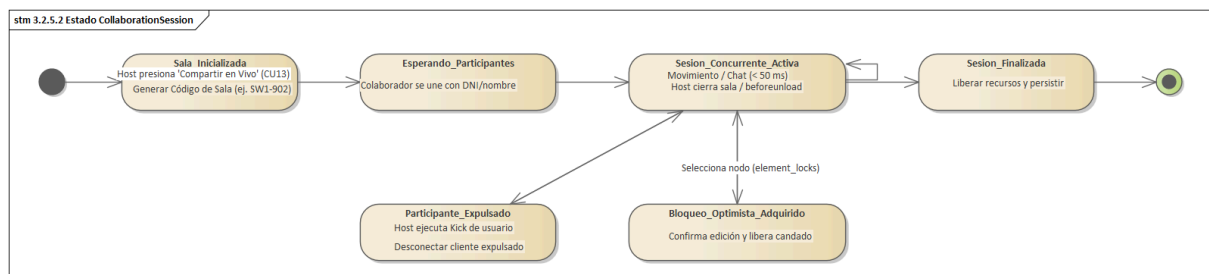


3.2.5) Análisis de Ciclo de Vida (Diagrama de Máquina de Estados)

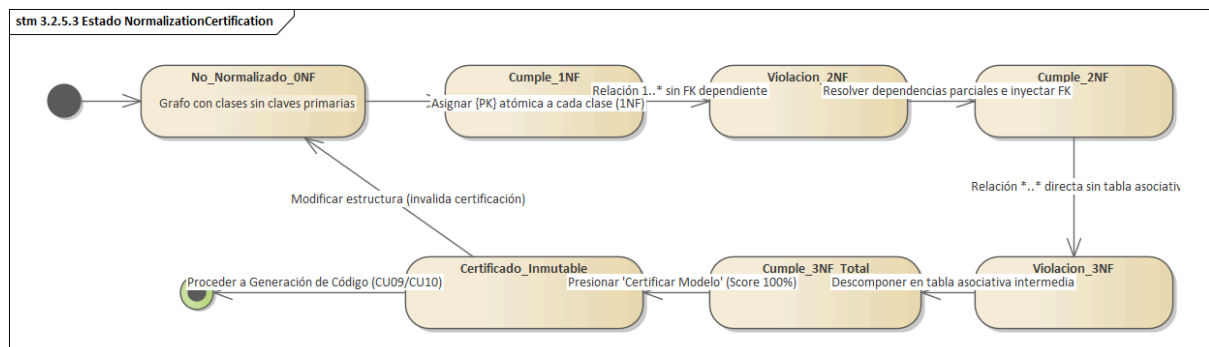
Estado 1 - Ciclo de Vida del Proyecto UML



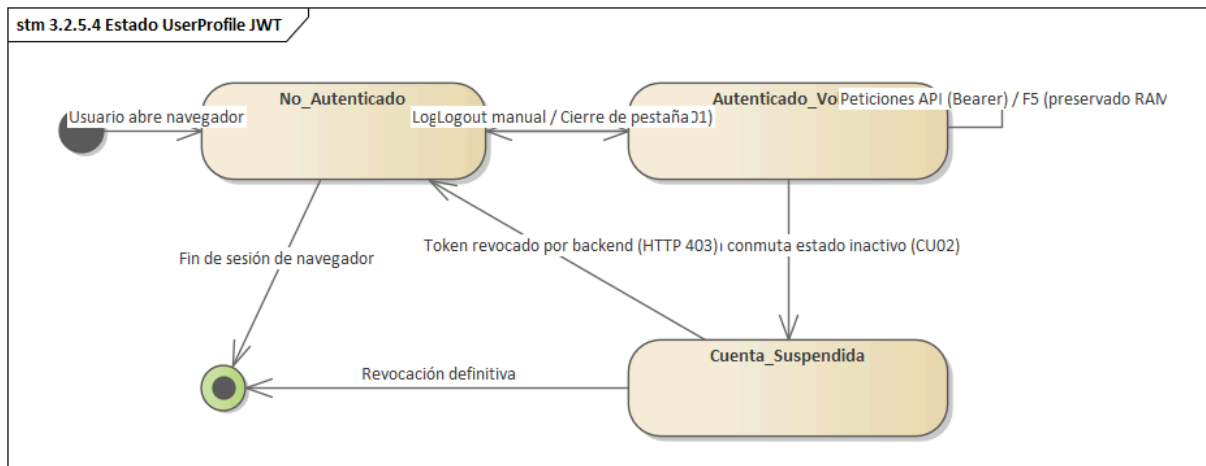
Estado 2 - Ciclo de Vida de la Sesión Colaborativa Concurrente



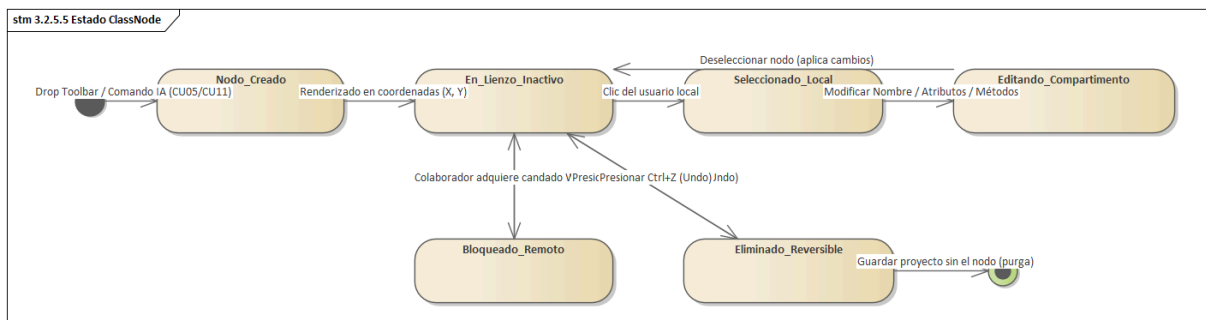
Estado 3 - Ciclo de Vida de la Certificación de Normalización Relacional



Estado 4 - Ciclo de Vida de la Sesión y Token JWT Volátil



Estado 5 - Ciclo de Vida del Nodo de Clase en el Canvas

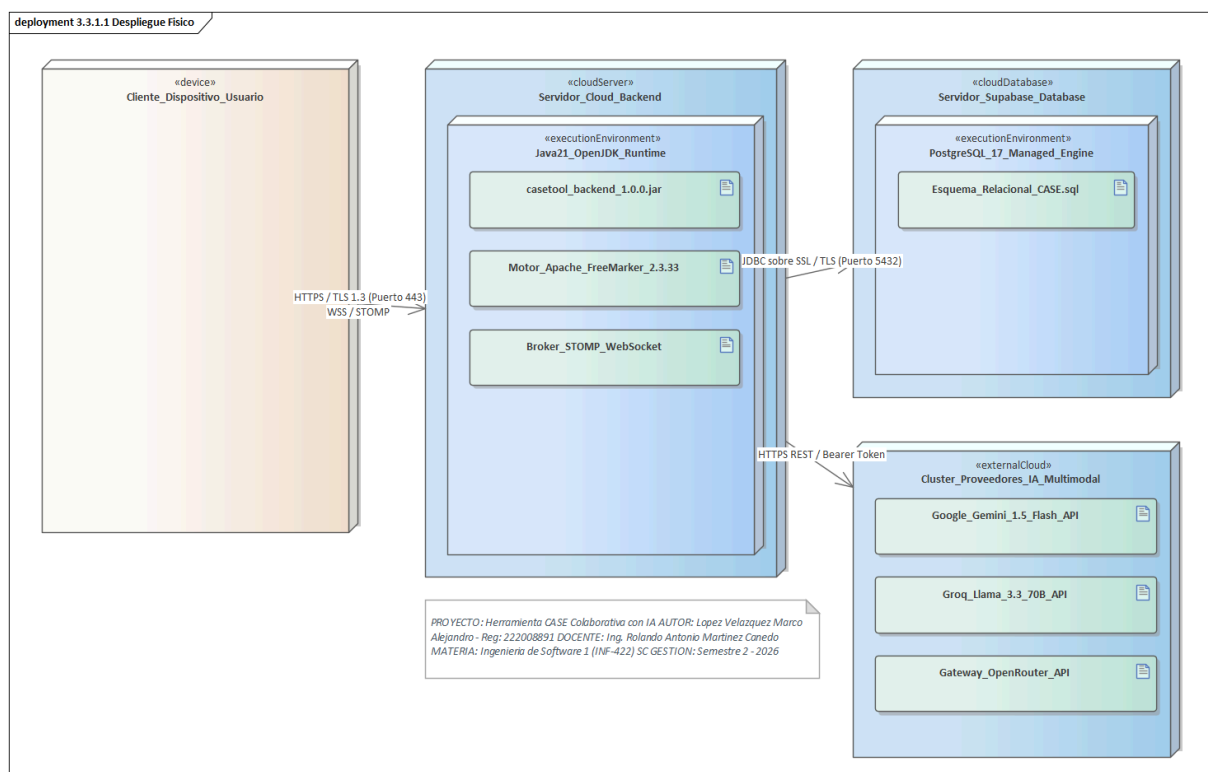


3.3) Flujo de Trabajo: Diseño

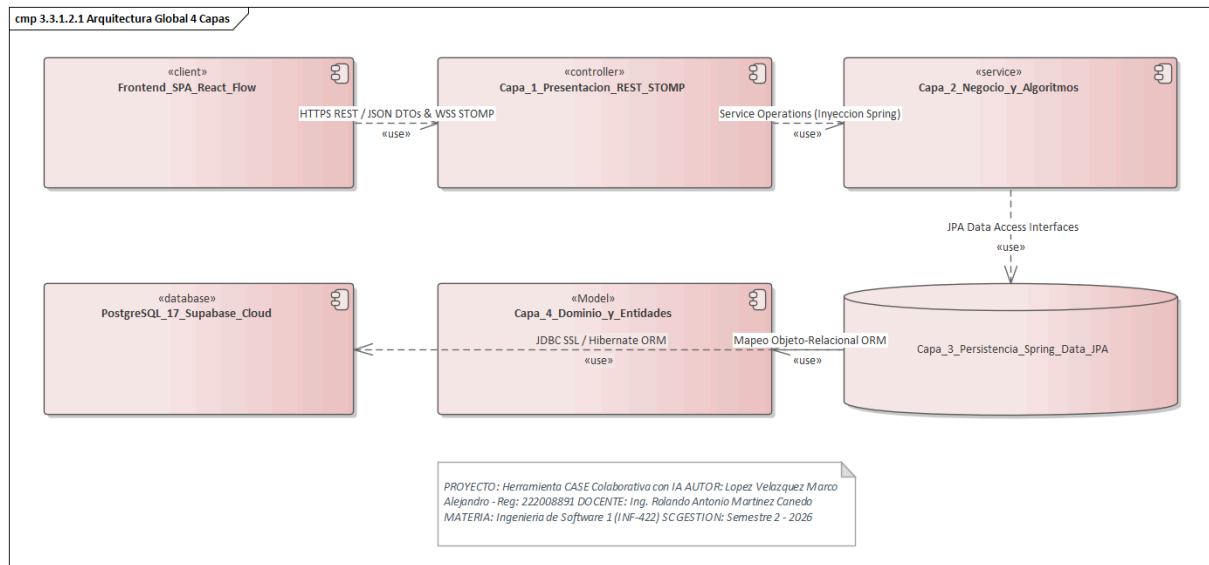
El flujo de trabajo de Diseño materializa la arquitectura conceptual del Análisis en una solución física y lógica lista para su implementación. Define la distribución de nodos de cómputo en el despliegue físico, la arquitectura en capas del código fuente sobre el paquete raíz `com.sw1.casetool`, la topología de red, la orquestación global del sistema, la estructura interna del modelador visual reactivo, la especificación de interacciones temporales para los 13 Casos de Uso (Ciclos 1, 2 y 3), las restricciones de latencia en tiempo real y el diseño físico formal de la base de datos relacional PostgreSQL 17.

3.3.1) Diseño de la Arquitectura

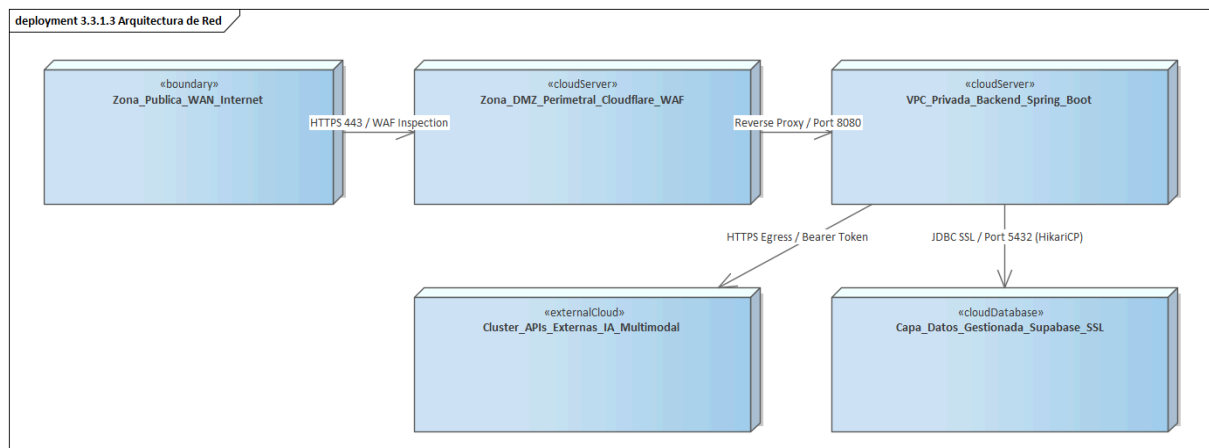
3.3.1.1) Diseño de arquitectura física - Diagrama de despliegue



3.3.1.2) Diseño de arquitectura lógica - Diagrama de paquete organizado en capas

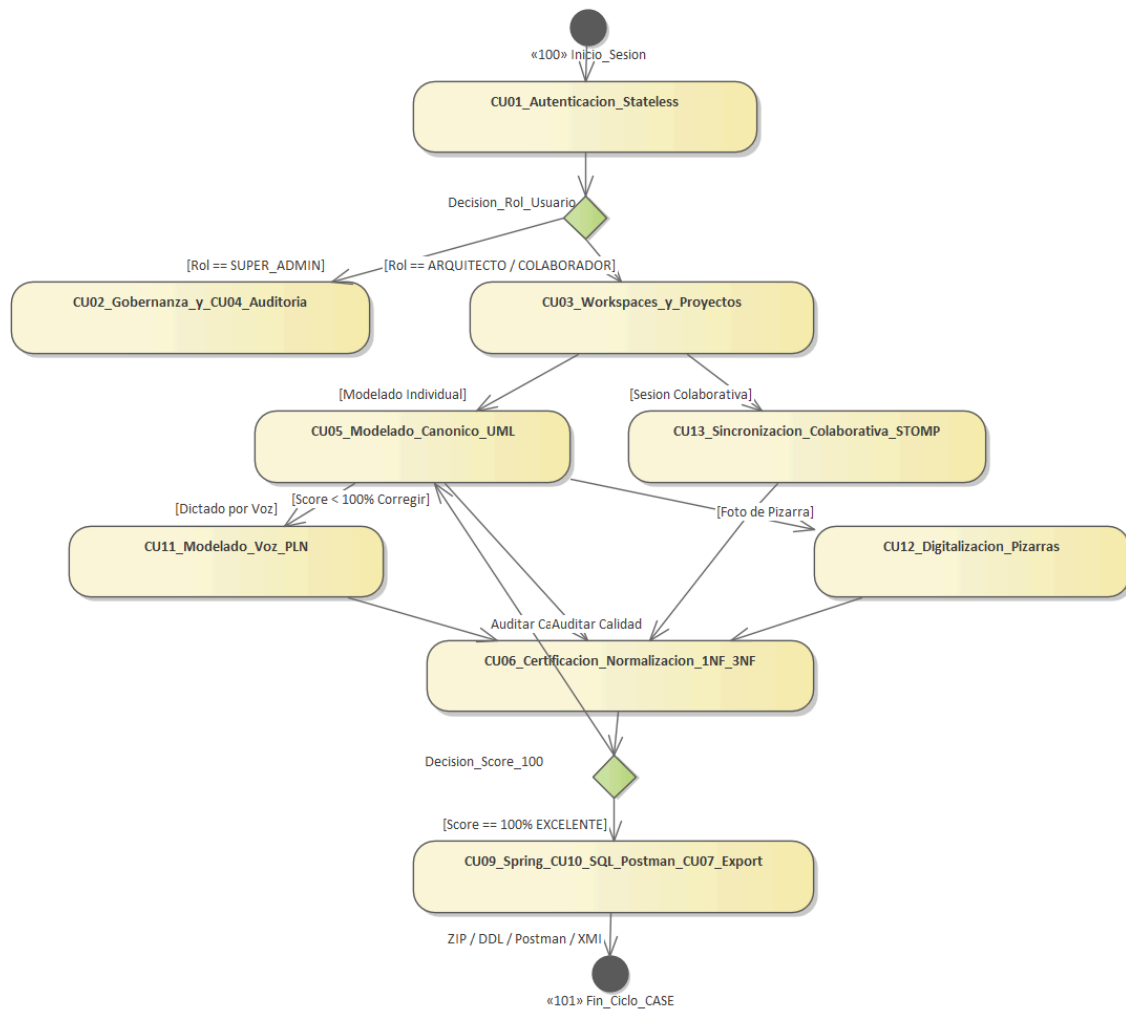


3.3.1.3) Diagrama de arquitectura de red

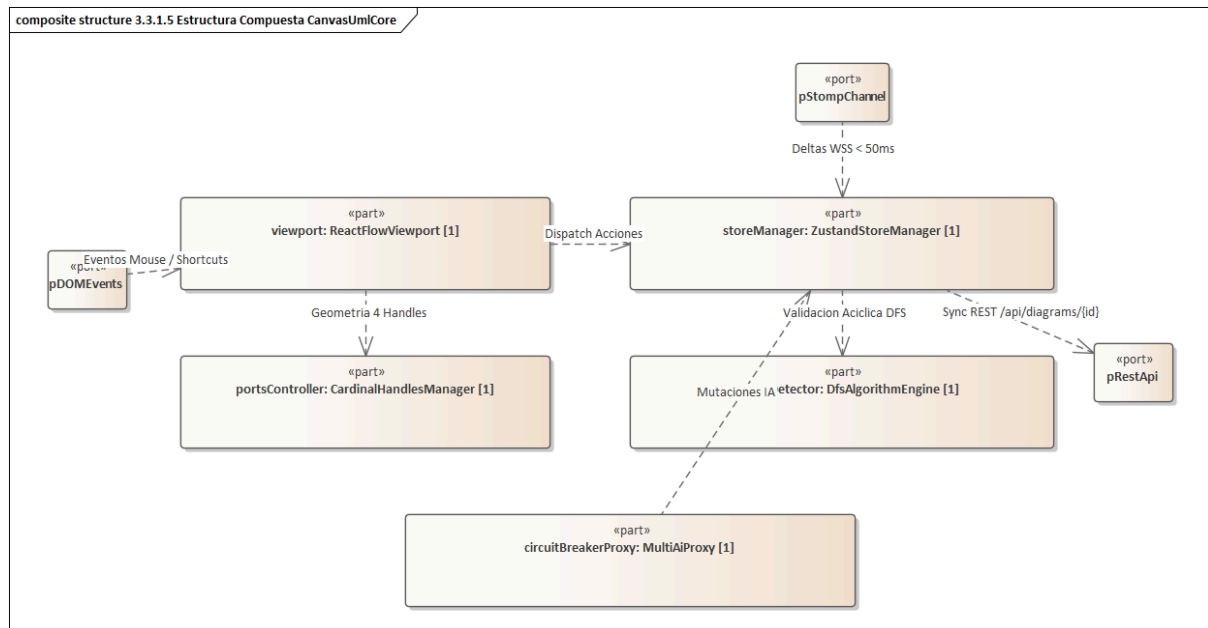


3.3.1.4) Orquestación General (Diagrama Global de Interacciones)

act 3.3.1.4 Orquestacion General



3.3.1.5) Diseño Interno de Microservicios (Diagrama de Estructura Compuesta)

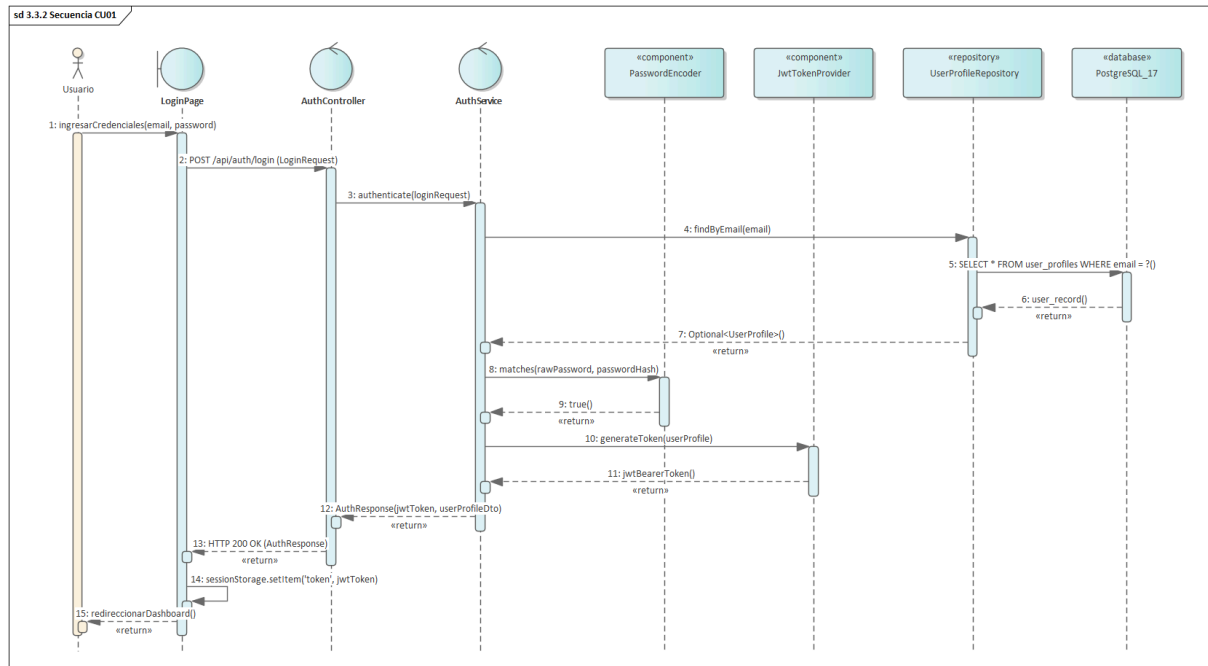


3.3.2) Diseño de caso de uso

Diagrama de Secuencia

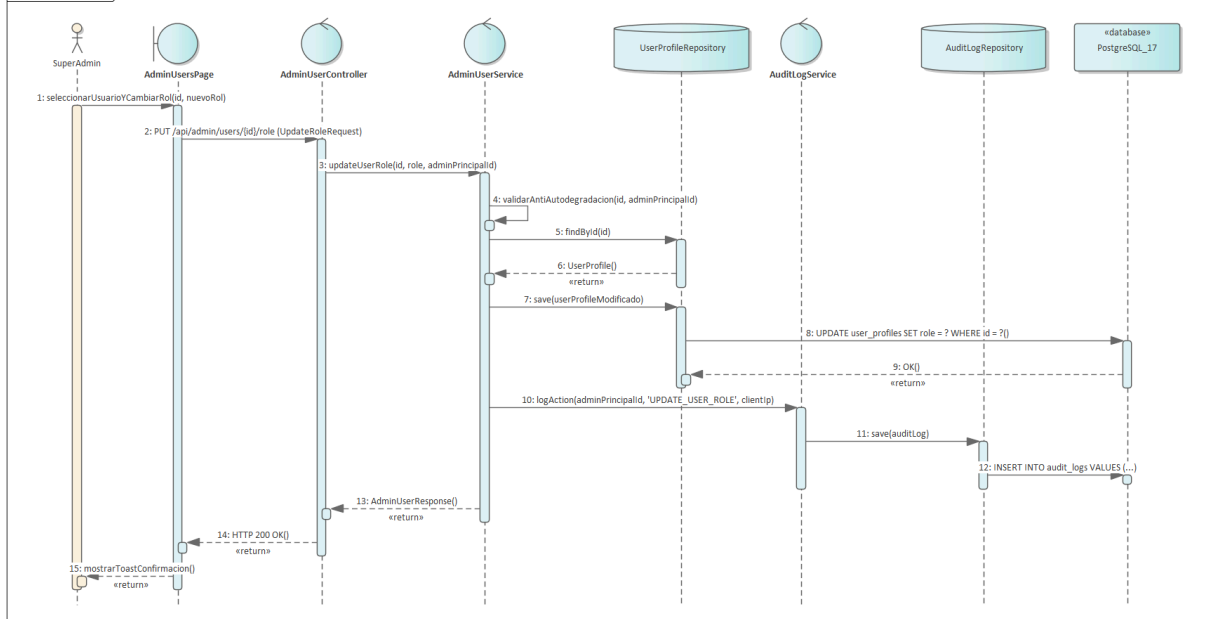
CICLO 1: Núcleo Arquitectónico, Workspaces, Modelador Core y Normalización

CU01 - Autenticación Stateless y Gestión de Cuentas

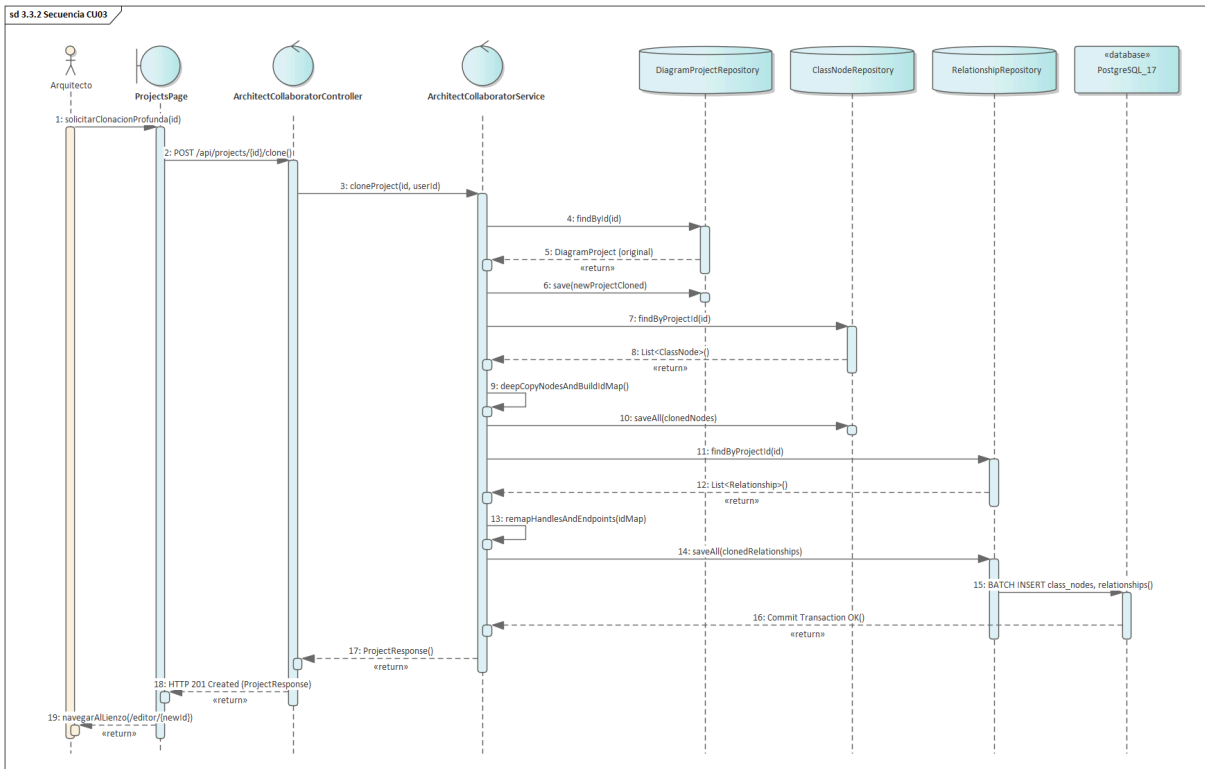


CU02 - Control de Roles (RBAC) y Gobernanza

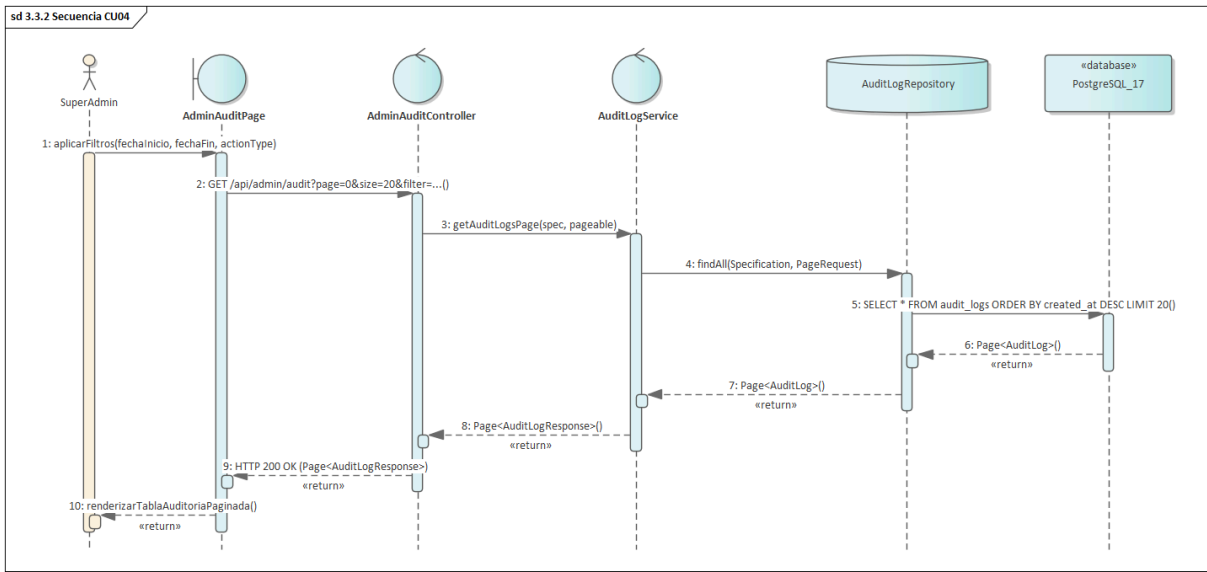
sd 3.3.2 Secuencia CU02



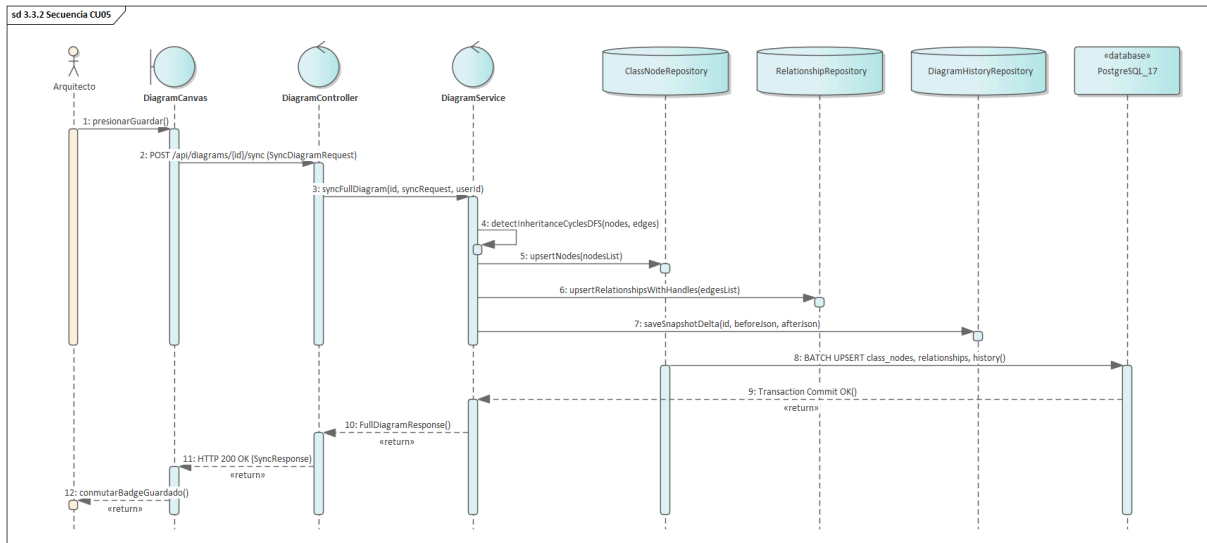
CU03 - Gestión de Proyectos, Plantillas y Onboarding



CU04 - Auditoría Forense Global y Trazabilidad del Modelo

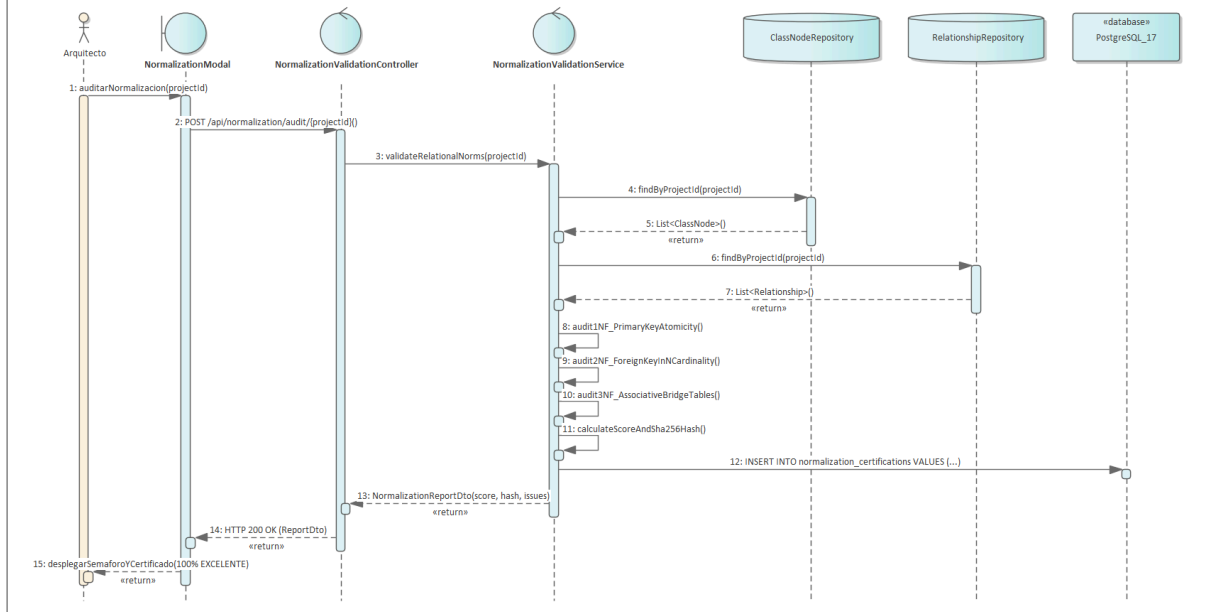


CU05 - Modelado Canónico de Clases y Relaciones UML 2.5



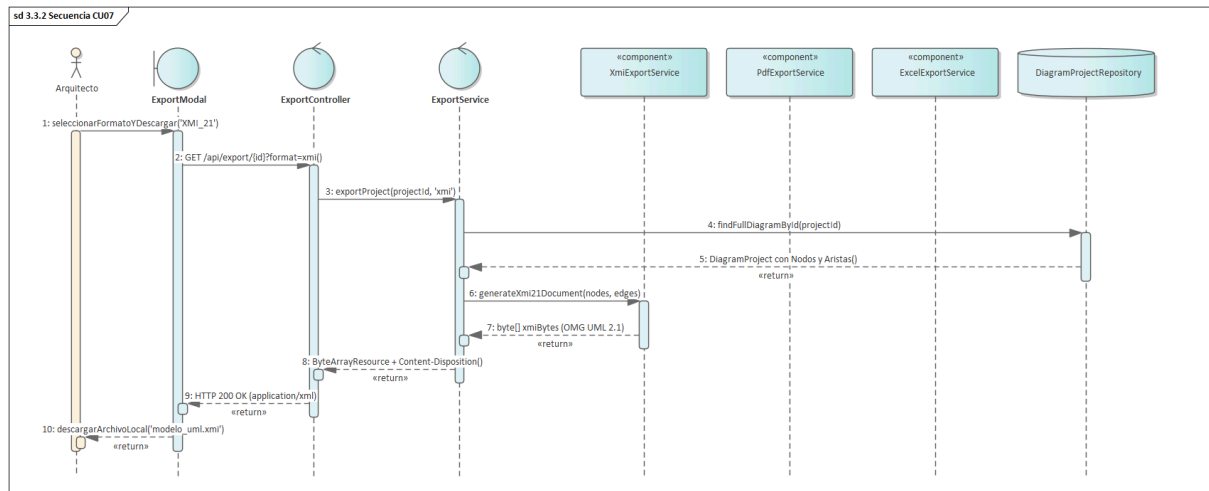
CU06 - Certificación de Normalización Relacional (1NF a 3NF)

sd 3.3.2 Secuencia CU06

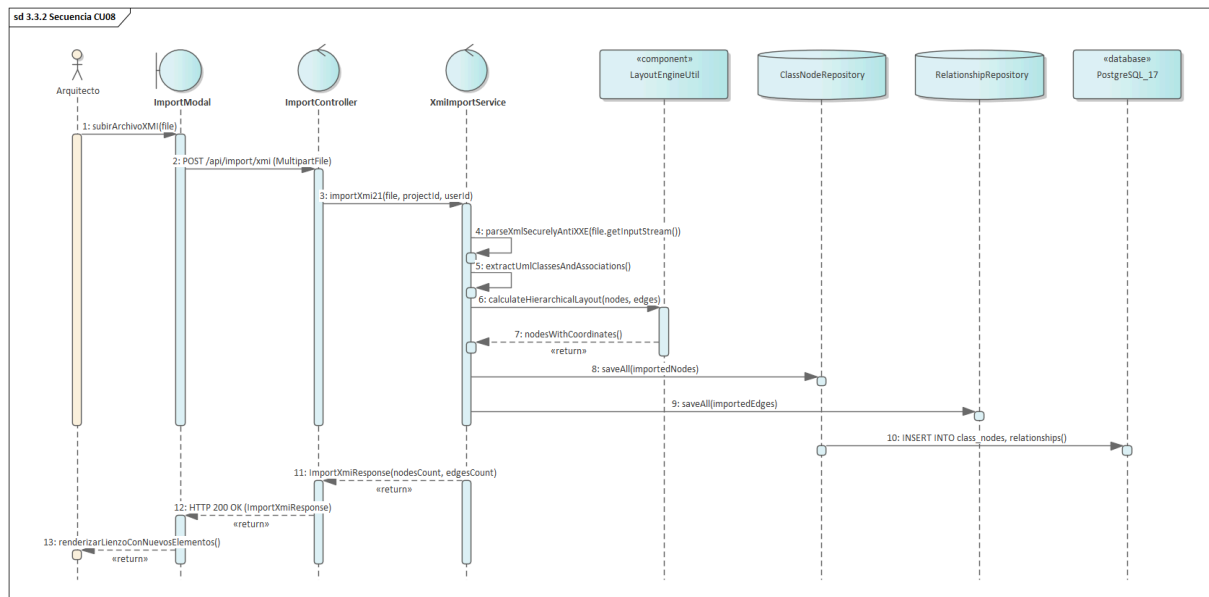


CICLO 2: Interoperabilidad XMI y Generación MDA de Código

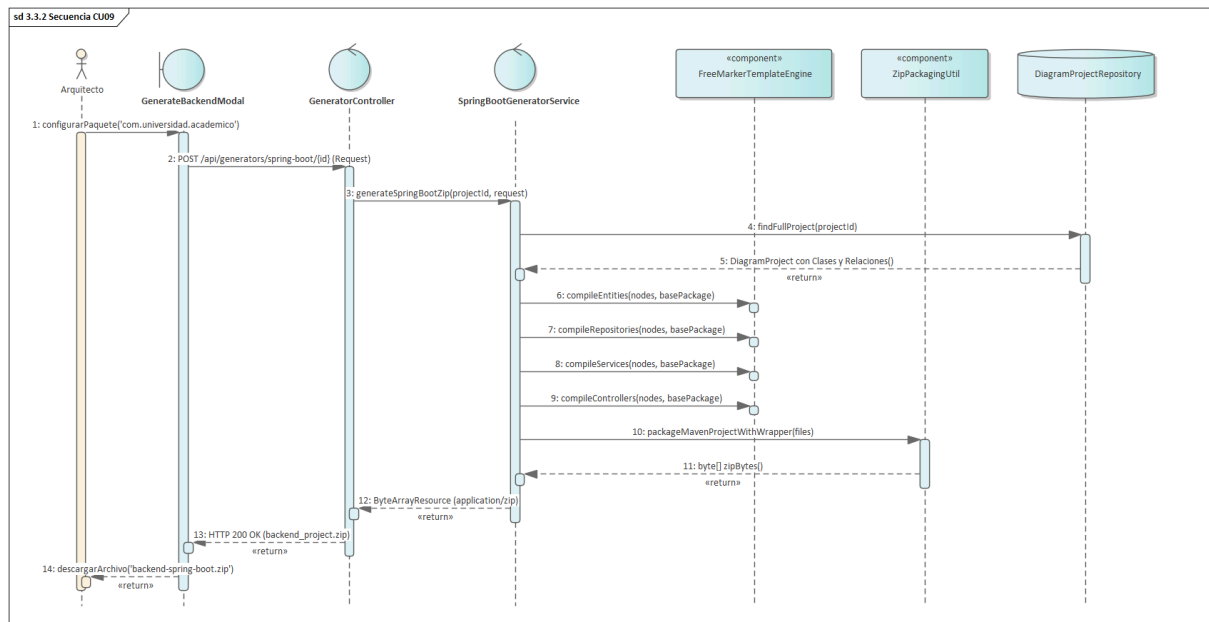
CU07 - Exportación Multiformato (XMI, PNG, PDF, Excel)



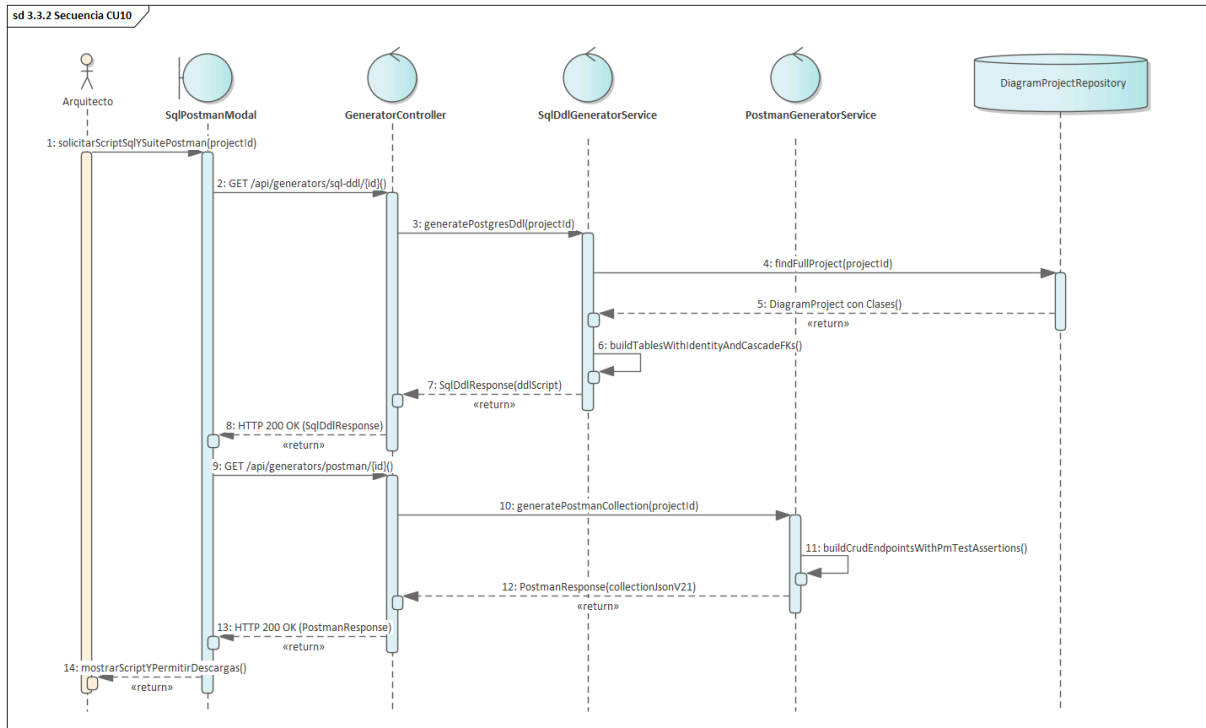
CU08 - Importación XMI 2.1 con Auto-Layout Jerárquico



CU09 - Generación de Backend Spring Boot en 4 Capas (ZIP)

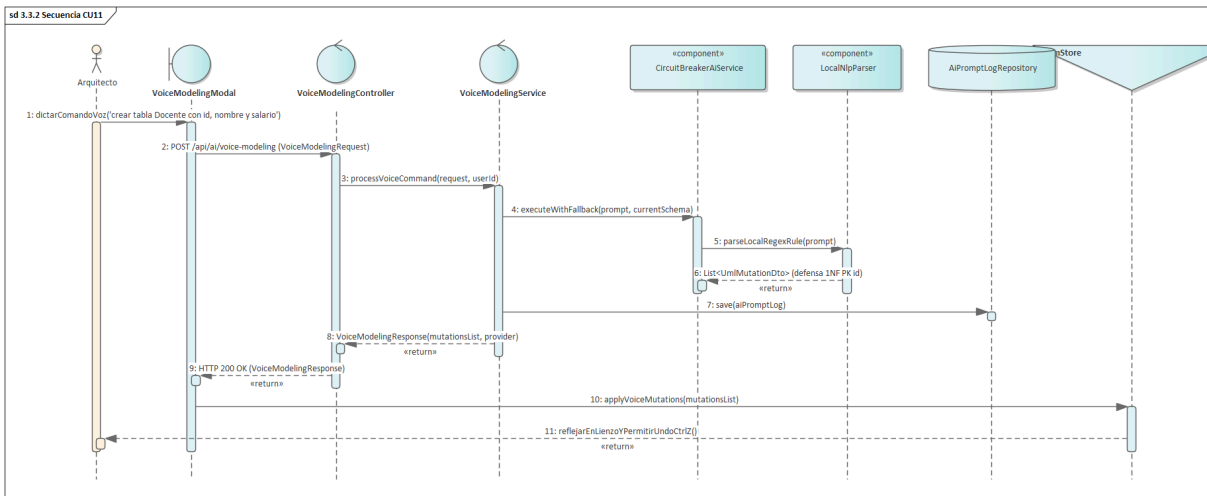


CU10 - Generación de Esquema DDL SQL y Suite Postman v2.1

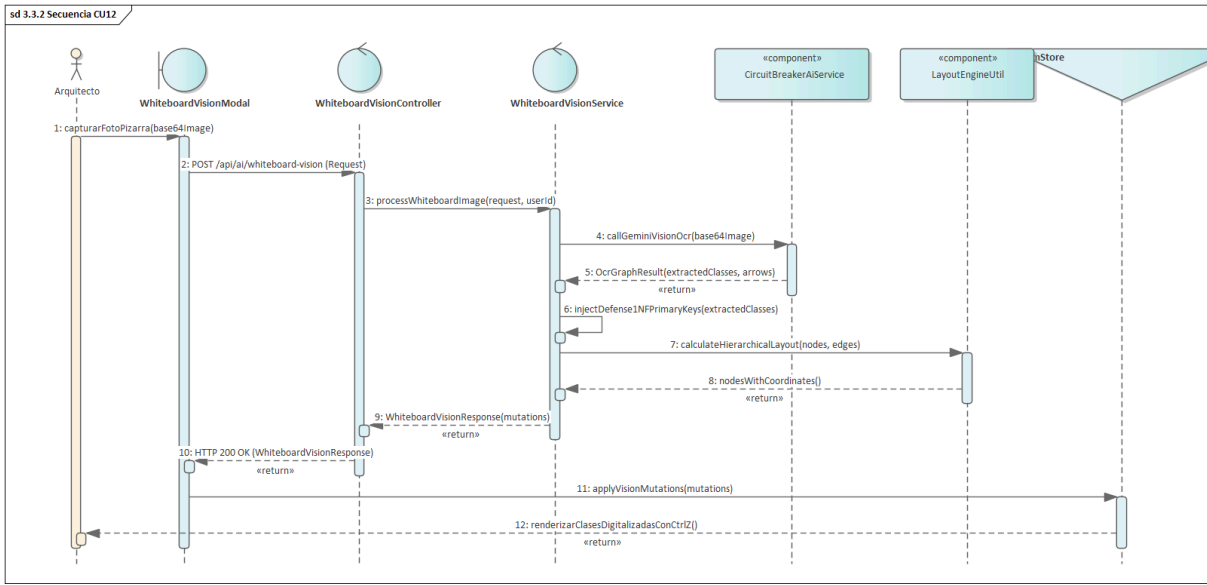


CICLO 3: Inteligencia Artificial Multimodal y Sincronización Concurrente

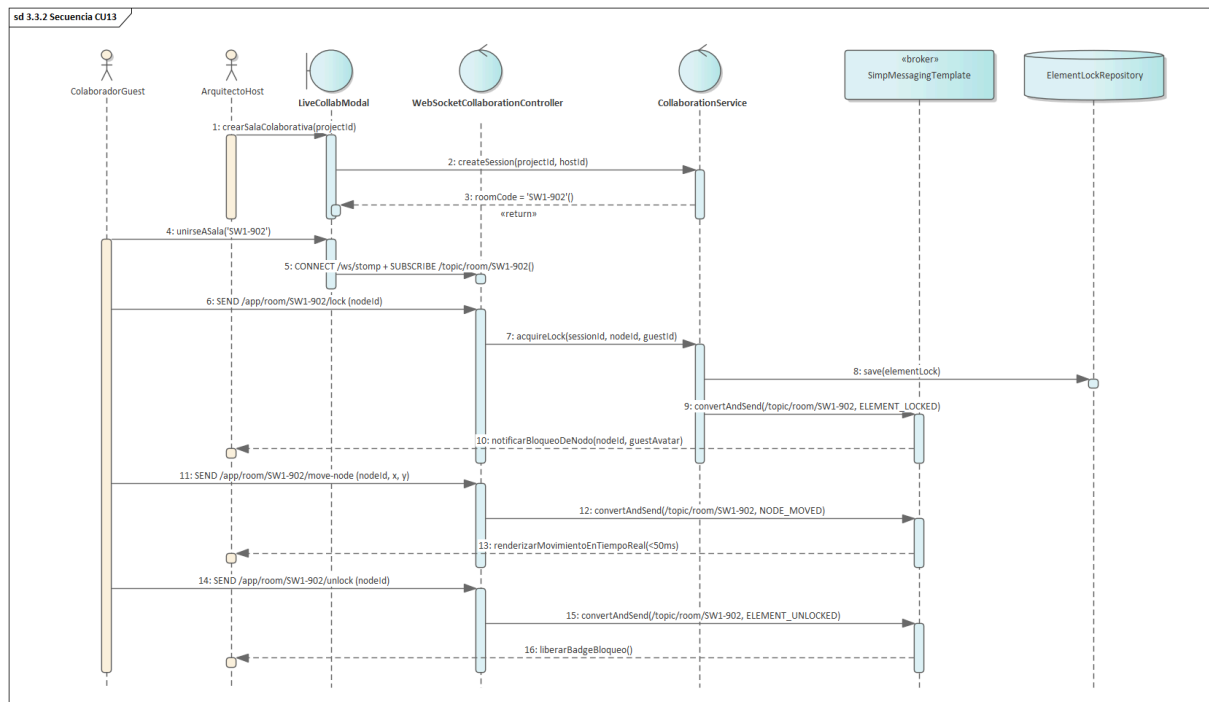
CU11 - Modelado Asistido por Dictado de Voz (PLN)



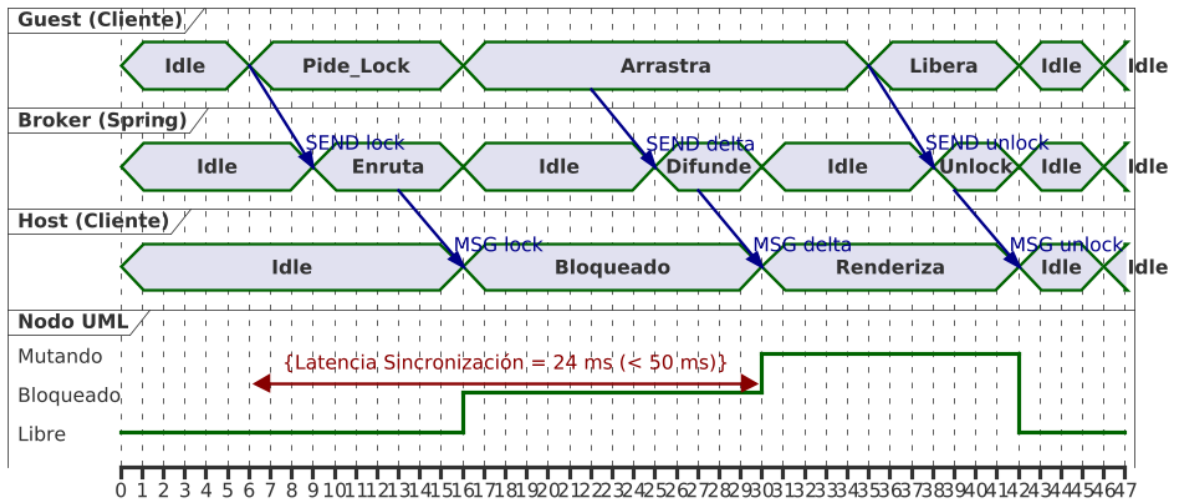
CU12 - Digitalización Óptica de Pizarras Físicas con Visión



CU13 - Sincronización Colaborativa Concurrente WSS STOMP



3.3.2.1) Restricciones de Latencia y WebSockets (Diagrama de Tiempos)



3.3.3.1) Diseño Lógico

preferences	Map<String, Object>	preferences	JSONB	DEFAULT '{"grid":true,...}':::jsonb
createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()
updatedAt	Instant	updated_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

2. Entidad DiagramProject ↔ Tabla diagram_projects

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
userId	UUID	user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE RESTRICT
title	String	title	VARCHAR(200)	NOT NULL
description	String	description	TEXT	NULLABLE
semanticVersion	String	semantic_version	VARCHAR(20)	NOT NULL, DEFAULT 'v1.0.0'
domainTags	List<String>	domain_tags	JSONB	DEFAULT '[]':::jsonb
isDeleted	Boolean	is_deleted	BOOLEAN	NOT NULL, DEFAULT false (Soft Delete)
deletedAt	Instant	deleted_at	TIMESTAMP TZ	NULLABLE
createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()
updatedAt	Instant	updated_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

3. Entidad ClassNode ↔ Tabla class_nodes

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
--------------------	---------------------	-------------------------	--------------------	--------------------------

id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
projectId	UUID	project_id	UUID	FK \rightarrow diagram_projects(id) ON DELETE CASCADE
name	String	name	VARCHAR(150)	NOT NULL
isAbstract	Boolean	is_abstract	BOOLEAN	NOT NULL, DEFAULT false
attributes	List<AttributeItem>	attributes	JSONB	NOT NULL, DEFAULT '[]':jsonb
methods	List<MethodItem>	methods	JSONB	NOT NULL, DEFAULT '[]':jsonb
positionX	Double	position_x	DOUBLE PRECISION	NOT NULL, DEFAULT 0.0
positionY	Double	position_y	DOUBLE PRECISION	NOT NULL, DEFAULT 0.0
createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()
updatedAt	Instant	updated_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

4. Entidad Relationship ↔ Tabla relationships

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
projectId	UUID	project_id	UUID	FK \rightarrow diagram_projects(id) ON DELETE CASCADE
sourceId	UUID	source_id	UUID	FK \rightarrow class_nodes(id) ON DELETE CASCADE

targetId	UUID	target_id	UUID	FK \rightarrow class_nodes(id) ON DELETE CASCADE
type	RelationType	type	VARCHAR(50)	NOT NULL
sourceHandle	String	source_handle	VARCHAR(20)	NULLABLE ('top', 'bottom', 'left', 'right')
targetHandle	String	target_handle	VARCHAR(20)	NULLABLE ('top', 'bottom', 'left', 'right')
multiplicity	String	multiplicity	VARCHAR(50)	NULLABLE
label	String	label	VARCHAR(100)	NULLABLE
createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

5. Entidad DiagramHistory ↔ Tabla diagram_history

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
projectId	UUID	project_id	UUID	FK \rightarrow diagram_projects(id) ON DELETE CASCADE
actorUserId	UUID	actor_user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE SET NULL
actionType	String	action_type	VARCHAR(100)	NOT NULL
beforeSnapshot	JsonNode	before_snapshot	JSONB	NULLABLE
afterSnapshot	JsonNode	after_snapshot	JSONB	NULLABLE
createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

6. Entidad AuditLog ↔ Tabla audit_logs

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	Long	id	BIGSERIAL	PK, NOT NULL, AUTO_INCREMENT
actorUserId	UUID	actor_user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE SET NULL
actionType	String	action_type	VARCHAR(100)	NOT NULL
clientIp	String	client_ip	VARCHAR(45)	NOT NULL, DEFAULT '127.0.0.1'
payloadDetails	Map<String, Object>	payload_details	JSONB	NULLABLE
createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

7. Entidad DomainTemplate ↔ Tabla domain_templates

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
title	String	title	VARCHAR(150)	NOT NULL
domainCategory	String	domain_category	VARCHAR(50)	NOT NULL
description	String	description	TEXT	NULLABLE
nodesSeed	JsonNode	nodes_seed	JSONB	NOT NULL, DEFAULT '[]::jsonb
edgesSeed	JsonNode	edges_seed	JSONB	NOT NULL, DEFAULT '[]::jsonb
isActive	Boolean	is_active	BOOLEAN	NOT NULL, DEFAULT true

createdAt	Instant	created_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()
-----------	---------	------------	-----------------	----------------------------

8. Entidad ProjectArtifact ↔ Tabla project_artifacts

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
projectId	UUID	project_id	UUID	FK \rightarrow diagram_projects(id) ON DELETE CASCADE
artifactType	String	artifact_type	VARCHAR(50)	NOT NULL
fileName	String	file_name	VARCHAR(255)	NOT NULL
fileSizeBytes	Long	file_size_bytes	BIGINT	NOT NULL
generatedAt	Instant	generated_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()

9. Entidad AiPromptLog ↔ Tabla ai_prompt_logs

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	Long	id	BIGSERIAL	PK, NOT NULL, AUTO_INCREMENT
userId	UUID	user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE SET NULL
inputModality	String	input_modality	VARCHAR(30)	NOT NULL
providerUsed	String	provider_used	VARCHAR(50)	NOT NULL
latencyMs	Integer	latency_ms	INTEGER	NOT NULL
wasSuccessful	Boolean	was_successful	BOOLEAN	NOT NULL, DEFAULT true

createdAt	Instant	created_at	TIMESTAM PTZ	NOT NULL, DEFAULT NOW()
-----------	---------	------------	-----------------	----------------------------

10. Entidad CollaborationSession ↔ Tabla collaboration_sessions

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
projectId	UUID	project_id	UUID	FK \rightarrow diagram_projects(id) ON DELETE CASCADE
roomCode	String	room_code	VARCHAR(1 0)	NOT NULL, UNIQUE
hostUserId	UUID	host_user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE CASCADE
isActive	Boolean	is_active	BOOLEAN	NOT NULL, DEFAULT true
startedAt	Instant	started_at	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()
closedAt	Instant	closed_at	TIMESTAMP TZ	NULLABLE

11. Entidad CollaborationParticipant ↔ Tabla collaboration_participants

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
sessionId	UUID	session_id	UUID	FK \rightarrow collaboration_sessions(id) ON DELETE CASCADE
userId	UUID	user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE CASCADE
joinedAt	Instant	joined_at	TIMESTAM PTZ	NOT NULL, DEFAULT NOW()
lastPingAt	Instant	last_ping_at	TIMESTAM PTZ	NOT NULL, DEFAULT NOW()

12. Entidad ElementLock ↔ Tabla element_locks

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
sessionId	UUID	session_id	UUID	FK \rightarrow collaboration_sessions(id) ON DELETE CASCADE
nodeId	UUID	node_id	UUID	FK \rightarrow class_nodes(id) ON DELETE CASCADE
lockedByUserId	UUID	locked_by_user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE CASCADE
acquiredAt	Instant	acquired_at	TIMESTAMPZ	NOT NULL, DEFAULT NOW()
Restricción Única	—	uq_session_node_lock	UNIQUE	(session_id, node_id)

13. Entidad CollaborationMessage ↔ Tabla collaboration_messages

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	Long	id	BIGSERIAL	PK, NOT NULL, AUTO_INCREMENT
sessionId	UUID	session_id	UUID	FK \rightarrow collaboration_sessions(id) ON DELETE CASCADE
senderUserId	UUID	sender_user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE CASCADE
content	String	content	TEXT	NOT NULL
sentAt	Instant	sent_at	TIMESTAMPZ	NOT NULL, DEFAULT NOW()

14. Entidad NormalizationCertification ↔ Tabla normalization_certifications

Atributo (Java 21)	Tipo de Dato (Java)	Columna (PostgreSQL 17)	Tipo de Dato (SQL)	Restricción / Integridad
id	UUID	id	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()
projectId	UUID	project_id	UUID	FK \rightarrow diagram_projects(id) ON DELETE CASCADE
certifiedByUserId	UUID	certified_by_user_id	UUID	FK \rightarrow user_profiles(id) ON DELETE RESTRICT
scorePercentage	Integer	score_percentage	INTEGER	NOT NULL (Valores 0 a 100)
certificationHash	String	certification_hash	VARCHAR(64)	NOT NULL (Hash SHA-256)
certifiedAt	Instant	certified_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()

3.3.3.3) DISEÑO FÍSICO (Tabla de Volumen)

N.º	Nombre de la Tabla	Tamaño Estimado / Fila (Bytes)	Filas Iniciales	Crecimiento Estimado (Filas/Mes)	Tamaño Anual Proyectado (MB)
1	user_profiles	384 B	50	200	0.92 MB
2	diagram_projects	512 B	100	500	3.07 MB
3	class_nodes	1,024 B	600	4,000	49.15 MB
4	relationships	256 B	800	5,000	15.36 MB
5	diagram_history	2,048 B	1,000	10,000	245.76 MB
6	audit_logs	512 B	2,000	8,000	49.15 MB
7	domain_templates	4,096 B	10	2	0.14 MB
8	project_artifacts	512 B	200	1,500	9.22 MB

9	ai_prompt_logs	1,024 B	300	3,000	36.86 MB
10	collaboration_sessions	128 B	50	400	0.61 MB
11	collaboration_participants	96 B	150	1,200	1.38 MB
12	element_locks	64 B	20	5,000 (Volátil/TTL)	0.38 MB
13	collaboration_messages	256 B	500	6,000	18.43 MB
14	normalization_certifications	384 B	50	400	1.84 MB
TOTAL	Almacenamiento Consolidado	—	—	—	~432.27 MB / Año

3.3.3.4) Script

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
```

```
-- 1. Perfiles de Usuario y Roles RBAC
```

```
CREATE TABLE IF NOT EXISTS user_profiles (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    email VARCHAR(255) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    full_name VARCHAR(150) NOT NULL,  
    username VARCHAR(100) NOT NULL UNIQUE,  
    role VARCHAR(50) NOT NULL DEFAULT 'ARQUITECTO',  
    is_active BOOLEAN NOT NULL DEFAULT true,  
    preferences JSONB DEFAULT '{"grid": true, "snap": true, "theme": "warm-titanium",  
"autoSaveInterval": 30}::jsonb',  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
-- 2. Proyectos y Espacios de Trabajo
```

```
CREATE TABLE IF NOT EXISTS diagram_projects (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    user_id UUID NOT NULL REFERENCES user_profiles(id) ON DELETE RESTRICT,  
    title VARCHAR(200) NOT NULL,  
    description TEXT,  
    semantic_version VARCHAR(20) NOT NULL DEFAULT 'v1.0.0',  
    domain_tags JSONB DEFAULT '[]::jsonb',  
    is_deleted BOOLEAN NOT NULL DEFAULT false,
```

```
deleted_at TIMESTAMPTZ,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

-- 3. Nodos de Clases UML

```
CREATE TABLE IF NOT EXISTS class_nodes (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    project_id UUID NOT NULL REFERENCES diagram_projects(id) ON DELETE  
    CASCADE,  
    class_name VARCHAR(100) NOT NULL,  
    stereotype VARCHAR(50) DEFAULT 'entity',  
    is_abstract BOOLEAN NOT NULL DEFAULT false,  
    position_x DOUBLE PRECISION NOT NULL DEFAULT 100.0,  
    position_y DOUBLE PRECISION NOT NULL DEFAULT 100.0,  
    attributes_data JSONB NOT NULL DEFAULT '[]::jsonb',  
    methods_data JSONB NOT NULL DEFAULT '[]::jsonb',  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    CONSTRAINT uq_project_class_name UNIQUE (project_id, class_name)  
);
```

-- 4. Relaciones y Aristas UML

```
CREATE TABLE IF NOT EXISTS relationships (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    project_id UUID NOT NULL REFERENCES diagram_projects(id) ON DELETE  
    CASCADE,
```

```

        source_node_id UUID NOT NULL REFERENCES class_nodes(id) ON DELETE
        CASCADE,

        target_node_id UUID NOT NULL REFERENCES class_nodes(id) ON DELETE
        CASCADE,

        source_handle VARCHAR(50) DEFAULT 'right',

        target_handle VARCHAR(50) DEFAULT 'left',

        relationship_type VARCHAR(50) NOT NULL DEFAULT 'ASSOCIATION',

        source_multiplicity VARCHAR(20) DEFAULT '1',

        target_multiplicity VARCHAR(20) DEFAULT '1..*',

        source_role VARCHAR(100),

        target_role VARCHAR(100),

        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

);

```

-- 5. Historial y Trazabilidad del Modelo

```

CREATE TABLE IF NOT EXISTS diagram_history (

    id BIGSERIAL PRIMARY KEY,

    project_id UUID NOT NULL REFERENCES diagram_projects(id) ON DELETE
    CASCADE,

    author_user_id UUID REFERENCES user_profiles(id) ON DELETE SET NULL,

    action_type VARCHAR(100) NOT NULL,

    before_snapshot JSONB,

    after_snapshot JSONB,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

);

```

-- 6. Bitácora Forense de Seguridad y Gobernanza

```

CREATE TABLE IF NOT EXISTS audit_logs (

```

```

id BIGSERIAL PRIMARY KEY,
actor_user_id UUID REFERENCES user_profiles(id) ON DELETE SET NULL,
action_type VARCHAR(100) NOT NULL,
client_ip VARCHAR(45) NOT NULL DEFAULT '127.0.0.1',
payload_details JSONB,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

-- 7. Plantillas de Dominio

```

CREATE TABLE IF NOT EXISTS domain_templates (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    title VARCHAR(150) NOT NULL,
    domain_category VARCHAR(50) NOT NULL,
    description TEXT,
    nodes_seed JSONB NOT NULL,
    edges_seed JSONB NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT true,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

-- 8. Artefactos de Exportación

```

CREATE TABLE IF NOT EXISTS project_artifacts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES diagram_projects(id) ON DELETE
    CASCADE,
    artifact_type VARCHAR(50) NOT NULL,
    file_name VARCHAR(255) NOT NULL,

```

```
file_size_bytes BIGINT NOT NULL,  
generated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

-- 9. Logs de Asistencia con Inteligencia Artificial

```
CREATE TABLE IF NOT EXISTS ai_prompt_logs (  
    id BIGSERIAL PRIMARY KEY,  
    user_id UUID REFERENCES user_profiles(id) ON DELETE SET NULL,  
    input_modality VARCHAR(30) NOT NULL, -- 'VOICE_SPEECH_PLN' o  
'VISION_PHOTO_OCR'  
    provider_used VARCHAR(50) NOT NULL, -- 'GEMINI_FLASH', 'GROQ_VISION',  
'LOCAL_FALLBACK'  
    latency_ms INTEGER NOT NULL,  
    was_successful BOOLEAN NOT NULL DEFAULT true,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

-- 10. Sesiones Colaborativas en Tiempo Real

```
CREATE TABLE IF NOT EXISTS collaboration_sessions (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    project_id UUID NOT NULL REFERENCES diagram_projects(id) ON DELETE  
CASCADE,  
    room_code VARCHAR(10) NOT NULL UNIQUE,  
    host_user_id UUID NOT NULL REFERENCES user_profiles(id) ON DELETE CASCADE,  
    is_active BOOLEAN NOT NULL DEFAULT true,  
    started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    closed_at TIMESTAMPTZ  
);
```

-- 11. Participantes Conectados

```
CREATE TABLE IF NOT EXISTS collaboration_participants (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    session_id UUID NOT NULL REFERENCES collaboration_sessions(id) ON DELETE  
    CASCADE,  
    user_id UUID NOT NULL REFERENCES user_profiles(id) ON DELETE CASCADE,  
    joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    last_ping_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

-- 12. Candados Optimistas de Edición

```
CREATE TABLE IF NOT EXISTS element_locks (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    session_id UUID NOT NULL REFERENCES collaboration_sessions(id) ON DELETE  
    CASCADE,  
    node_id UUID NOT NULL REFERENCES class_nodes(id) ON DELETE CASCADE,  
    locked_by_user_id UUID NOT NULL REFERENCES user_profiles(id) ON DELETE  
    CASCADE,  
    acquired_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    CONSTRAINT uq_session_node_lock UNIQUE (session_id, node_id)  
);
```

-- 13. Mensajería de Chat Colaborativo

```
CREATE TABLE IF NOT EXISTS collaboration_messages (  
    id BIGSERIAL PRIMARY KEY,  
    session_id UUID NOT NULL REFERENCES collaboration_sessions(id) ON DELETE  
    CASCADE,
```

```

        sender_user_id UUID NOT NULL REFERENCES user_profiles(id) ON DELETE
        CASCADE,

        content TEXT NOT NULL,

        sent_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

    );

```

-- 14. Certificaciones de Normalización Relacional

```

CREATE TABLE IF NOT EXISTS normalization_certifications (

    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    project_id UUID NOT NULL REFERENCES diagram_projects(id) ON DELETE
    CASCADE,

    certified_by_user_id UUID NOT NULL REFERENCES user_profiles(id) ON DELETE
    RESTRICT,

    score_percentage INTEGER NOT NULL,

    certification_hash VARCHAR(64) NOT NULL,

    certified_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

);

```

-- Índices B-Tree para Rendimiento de Consulta

```

CREATE INDEX IF NOT EXISTS idx_projects_user_id ON diagram_projects(user_id);

CREATE INDEX IF NOT EXISTS idx_class_nodes_project ON class_nodes(project_id);

CREATE INDEX IF NOT EXISTS idx_relationships_project ON relationships(project_id);

CREATE INDEX IF NOT EXISTS idx_history_project_created ON
diagram_history(project_id, created_at DESC);

CREATE INDEX IF NOT EXISTS idx_audit_logs_action_created ON audit_logs(action_type,
created_at DESC);

CREATE INDEX IF NOT EXISTS idx_collab_room_code ON
collaboration_sessions(room_code);

```

3.4) Flujo de Trabajo: Implementación

3.4.1) Elección de la plataforma de desarrollo de software

Capa / Subsistema	Tecnología Seleccionada	Justificación Técnica y de Ingeniería
Lenguaje Backend	Java 21 LTS	Tipado estricto, alto rendimiento computacional, soporte nativo de Virtual Threads (Project Loom) para concurrencia masiva y compatibilidad industrial.
Framework Backend	Spring Boot 4.1.0	Ecosistema empresarial de referencia para APIs REST stateless, orquestación transaccional con Spring Data JPA y soporte STOMP sobre WebSockets.
Motor de Persistencia	PostgreSQL 17 / Supabase	Base de datos relacional ACID de nivel empresarial con soporte nativo para columnas estructuradas JSONB, extensiones UUID y Row-Level Security (RLS).
Librería de Canvas UML	React Flow (@xyflow/react v12)	Manejo reactivo y grafocéntrico de nodos y aristas vectoriales con aceleración por GPU, puertos magnéticos cardinales y zoom/pan fluidos a 60 FPS.
Frontend Framework	React 18 + TypeScript + Vite	Renderizado declarativo con control de estado inmutable (Zustand), tipado estricto en tiempo de compilación y empaquetado de producción ultrarrápido (HMR).
Diseño y Ergonomía UI	Tailwind CSS v4	Paletas profesionales de ingeniería (Titanio Cálido y Grafito Obsidiana) libres de sobrecarga gráfica, diseño responsivo y tipografía técnica.
Compilador CASE / MDD	Apache FreeMarker 2.3.33	Motor de plantillas Java de alto rendimiento para la compilación de árboles de sintaxis abstracta (AST) hacia código fuente Java 21 empaquetado en ZIP.
Interoperabilidad CASE	OMG XMI 2.1 (SAX / DOM Seguro)	Parser XML bidireccional con protección anti-XXE para compatibilidad con ArchiTec, Enterprise Architect y StarUML.

Mensajería Concurrente	Spring STOMP over SockJS	Protocolo orientado a eventos con subscripción a tópicos en memoria y latencias sub-50ms para colaboración multiusuario en tiempo real.
Inteligencia Artificial	Circuit Breaker Multi-IA	Resiliencia en 4 niveles: Google Gemini 1.5 Flash (primario), Groq Llama 3.3 70B (fallback rápido), OpenRouter (contingencia) y Motor Heurístico Local Offline.

3.4.1.1) Lenguaje de Programación Backend: Java 21 LTS + Spring Boot 4.1.0

Se adoptó Java 21 LTS aprovechando características modernas como pattern matching, records inmutables, virtual threads para manejo eficiente de conexiones concurrentes y la estabilidad del framework Spring Boot 4.1.0 configurado con pool HikariCP.

3.4.1.2) Lenguaje de Programación Frontend: React 18 + TypeScript + Vite

Se construyó una Single Page Application (SPA) sobre React 18 y TypeScript que garantiza la verificación estricta de tipos de datos de los grafos UML en tiempo de compilación, eliminando fallos en runtime y logrando tiempos de carga inferiores a 800 ms mediante Vite.

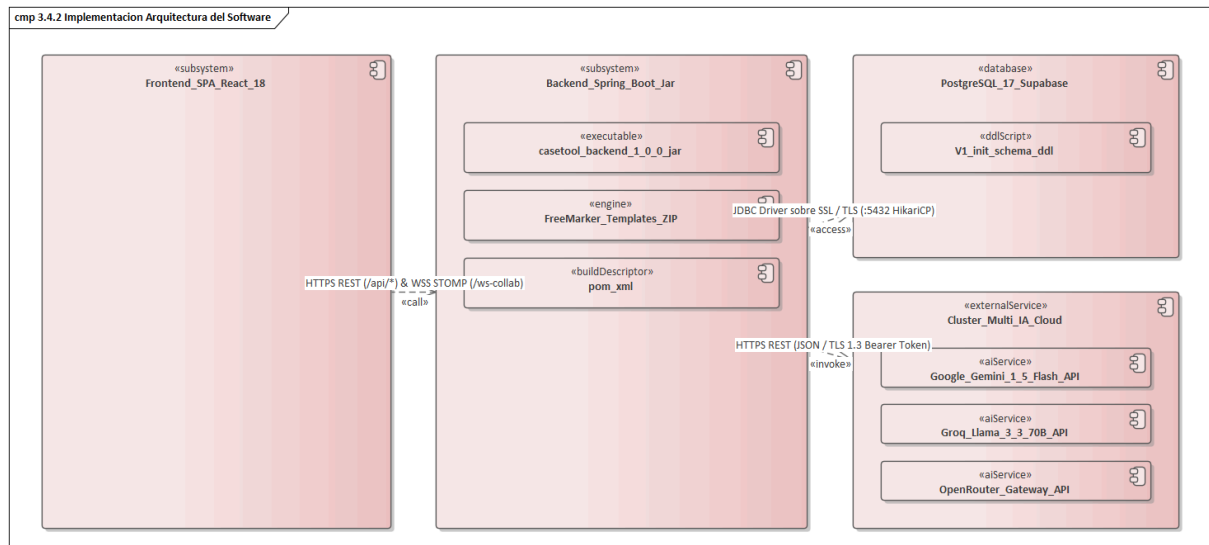
3.4.1.3) Base de Datos: PostgreSQL 17 Cloud (Supabase) con Tipos JSONB

La persistencia combina la rigidez relacional de claves primarias UUID e integridad referencial en cascada para clases y relaciones, junto a la flexibilidad de columnas **JSONB** para atributos, firmas de métodos y preferencias visuales de usuario.

3.4.1.4) Infraestructura y Despliegue en la Nube (AWS + Nginx):

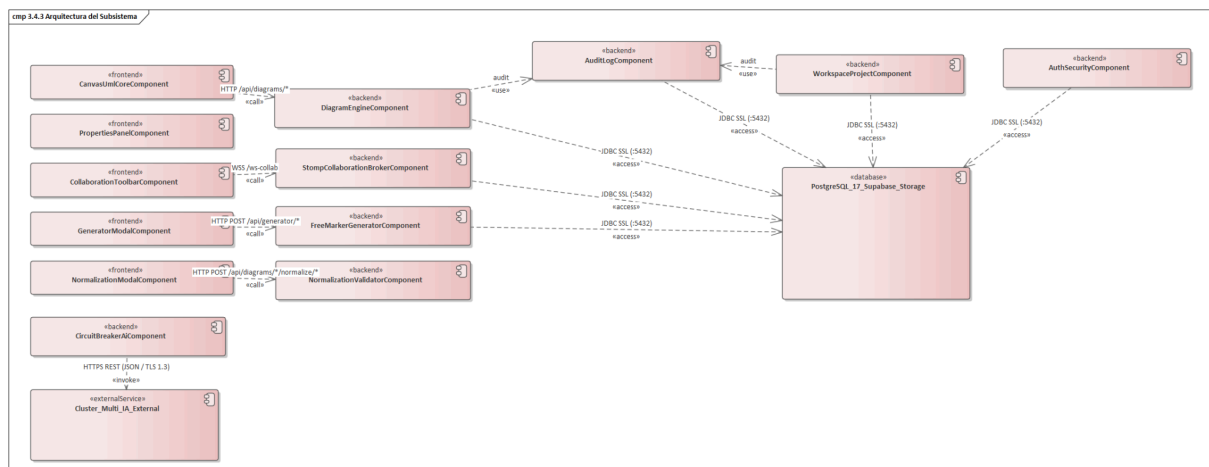
Para la puesta en producción del sistema se seleccionó una arquitectura sin contenedores (Bare-Metal/Nativa) sobre una instancia Amazon Web Services (AWS) EC2 t3.micro con sistema operativo Ubuntu Server 24.04 LTS. Esta configuración maximiza la eficiencia del hardware disponible al eliminar el overhead de memoria de Docker, permitiendo que el servidor web Nginx 1.28 y la máquina virtual Java 21 OpenJDK coexistan con un consumo conjunto menor a 360 MB de RAM. La seguridad perimetral se garantiza mediante grupos de seguridad de AWS (Security Groups) que exponen únicamente los puertos 80 (HTTP con redirección 301) y 443 (HTTPS/WSS con Let's Encrypt).

3.4.2) Implementación de la arquitectura del software

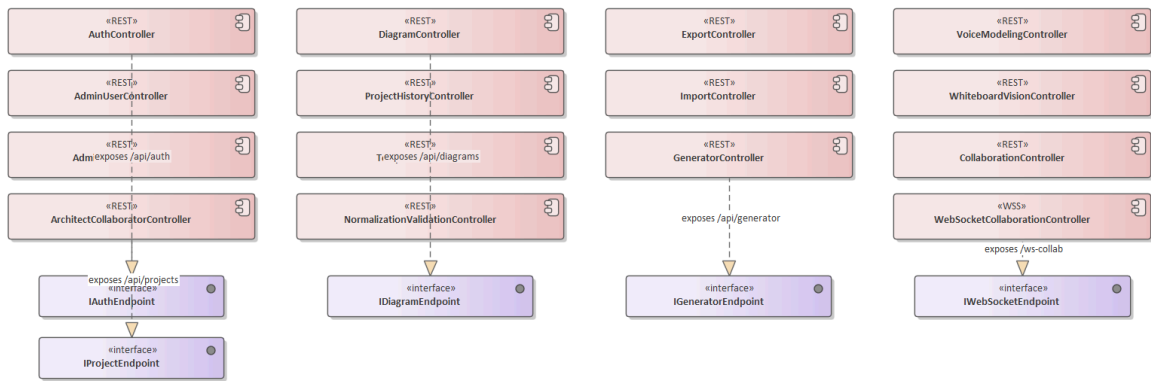


3.4.3) Implementación de la arquitectura del subsistema

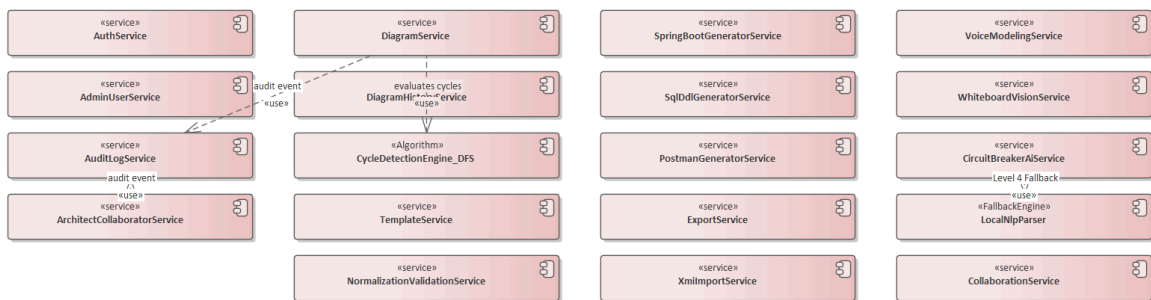
Diagrama de componentes de cada paquete



cmp 3.4.3.1 Componentes Paquete Controller



cmp 3.4.3.2 Componentes Paquete Service



3.5) Flujo de Trabajo: Pruebas

3.5.1 Plan de pruebas

CICLO 1: Núcleo Arquitectónico, Workspaces, Modelador Core y Normalización

Caso de Uso (CU)	Funcionalidad Clave	Criterios de Aceptación
CU01: Autenticación Stateless y Gestión de Cuentas	Autenticación sin estado con tokens Bearer JWT en memoria volátil de sesión (sessionStorage), hashing BCrypt y gestión de preferencias JSONB.	• El sistema debe validar credenciales encriptadas con BCrypt (costo 12) contra user_profiles. • El token JWT debe emitirse con algoritmo HMAC-SHA256 conteniendo id, email y rol (SUPER_ADMIN, ARQUITECTO, COLABORADOR). • El cliente debe almacenar el JWT exclusivamente en sessionStorage (RAM volátil), destruyéndose al cerrar la pestaña y resistiendo recargas F5. • Se debe rechazar el login con HTTP 401 si las credenciales son inválidas o la cuenta está inactiva.
CU02: Control de Roles (RBAC) y Gobernanza	Administración centralizada de privilegios por el Super Admin, conmutación de estado operativo y blindaje anti-autodegradación.	• La consola /admin/users debe restringirse exclusivamente a usuarios con rol SUPER_ADMIN. • El Super Admin debe poder promover/degradar roles y conmutar el estado operativo (is_active: true/false). • El backend debe bloquear con HTTP 400 cualquier intento del Super Admin de degradar su propio rol o auto-bloquearse. • Cada mutación de privilegios debe persistirse en audit_logs registrando la IP del cliente y el payload.

CU03: Gestión de Proyectos, Plantillas y Onboarding	CRUD integral de diagramas, versionado semántico v1.0.0, clonación profunda transaccional (Deep Copy) y onboarding interactivo (< 120s).	• El usuario debe poder crear proyectos desde cero o a partir de plantillas de dominio (ej. "Sistema Académico"). • La clonación profunda debe generar nuevos UUIDs para el proyecto y sus clases (class_nodes), remapeando los handles de relaciones (relationships). • Se debe permitir asignar tags de dominio y versionar semánticamente los modelos. • El tutorial interactivo guiado debe completarse en menos de 120 segundos recorriendo los 5 puntos clave de la interfaz.
CU04: Auditoría Forense Global y Trazabilidad del Modelo	Bitácora inmutable audit_logs con filtros multicriterio, historial diagram_history con diffs, papelera reversible y purga física.	• La bitácora audit_logs no puede ser editada ni purgada por ningún usuario (inmutabilidad estricta sin endpoints PUT/DELETE). • El sistema debe permitir filtrar logs por rango temporal, severidad y usuario, con exportación a Excel (.xlsx). • El historial de diagramas debe registrar instantáneas cronológicas (snapshots JSONB) con visualización de diferencias (diff). • La papelera debe permitir restauración reversible; el Super Admin solo puede supervisar/restaurar y tiene prohibida la eliminación física de proyectos ajenos.

CU05: Modelado Canónico de Clases y Relaciones UML 2.5	Lienzo reactivo con clases de 3 compartimentos estándar, tipos Java/SQL, badge {PK}, 4 handles magnéticos y prevención DFS de ciclos de herencia.	• El nodo de clase debe renderizar 3 compartimentos: cabecera con estereotipo, atributos tipados con badge {PK} y métodos con retorno. • La conexión entre clases debe realizarse mediante 4 handles magnéticos cardinales (top, bottom, left, right) con cálculo ortogonal no colisionante. • El algoritmo DFS debe verificar en tiempo real que no se generen ciclos circulares de herencia. • Todo cambio en el lienzo debe ser reversible con Ctrl+Z (Undo) y sincronizarse en lote mediante POST /api/diagrams/{id}/sync.
CU06: Certificación de Normalización Relacional (1NF a 3NF)	Motor heurístico basado en las reglas de Codd y TOM para validación de 1NF (PK única), 2NF (FKs en 1..) y 3NF (tablas intermedias en ..*), con certificado inmutable.	• El motor debe validar 1NF exigiendo que toda clase posea una clave primaria badge {PK} atómica. • Debe validar 2NF comprobando que en relaciones 1.. la clase del extremo muchos contenga la clave foránea dependiente. • Debe validar 3NF detectando relaciones ..* directas y exigiendo su descomposición mediante tabla intermedia asociativa. • Al alcanzar el 100% de cumplimiento, el sistema debe emitir un certificado digital con hash criptográfico inmutable.

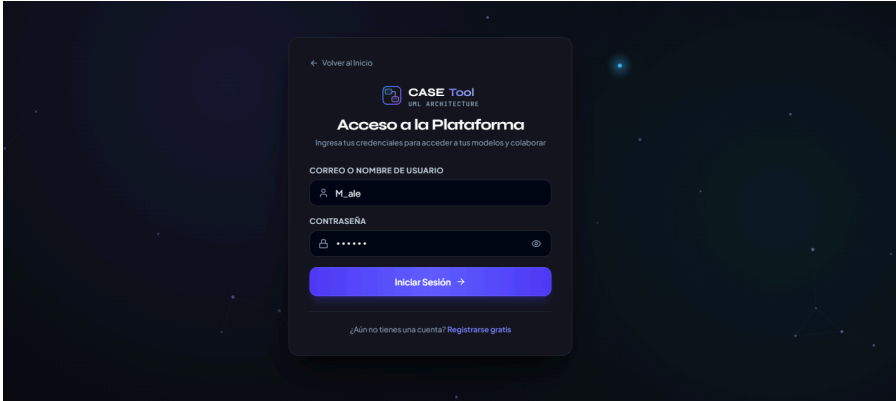
Reporte de Prueba — CICLO 1

Prueba de CU01: Autenticación Stateless y Gestión de Cuentas

Campo	Detalle
Caso de uso 01	Autenticación Stateless y Gestión de Cuentas
Descripción	Valida el proceso de autenticación de credenciales, emisión de Token JWT Bearer almacenado exclusivamente en la memoria RAM volátil del navegador (sessionStorage) y cifrado seguro de contraseñas con BCrypt (costo 12).
Precondiciones	a) Servidor backend Spring Boot 4.1.0 activo y escuchando peticiones. b) Base de datos PostgreSQL 17 conectada con usuario registrado previamente. c) Cliente web SPA abierto en navegador con almacenamiento sessionStorage limpio.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. Acceder a /login e ingresar correo y contraseña válidos de un usuario registrado.	El formulario valida el formato del correo y habilita el botón de acceso.	Satisfactorio
2. Presionar el botón "Iniciar Sesión".	El frontend envía POST /api/auth/login con las credenciales en el cuerpo de la petición.	Satisfactorio
3. Revisar la respuesta del backend Spring Boot.	El backend verifica el hash con PasswordEncoder.matches(), genera el JWT y retorna HTTP 200 con AuthResponse.	Satisfactorio
4. Inspeccionar el almacenamiento del navegador en DevTools (Application -> Session Storage).	La clave case_token contiene el JWT Bearer. Se verifica que localStorage se mantiene totalmente vacío.	Satisfactorio
5. Recargar la página con F5.	El interceptor de Axios recupera el token de sessionStorage y preserva la sesión activa sin redirigir al login.	Satisfactorio

6. Cerrar la pestaña del navegador y volver a abrir la URL /dashboard en una nueva pestaña.	La memoria volátil de sessionStorage se ha destruido; el guardián de rutas detecta la ausencia de token y redirige a /login.	Satisfactorio
7. Intentar iniciar sesión con contraseña incorrecta o cuenta suspendida (is_active = false).	El backend rechaza la petición con HTTP 401 Unauthorized y la interfaz despliega una notificación toast de error.	Satisfactorio

Campo	Detalle
Responsable	Tester QA / Desarrollador Backend
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU02: Control de Roles (RBAC) y Gobernanza

Campo	Detalle
Caso de uso 02	Control de Roles (RBAC) y Gobernanza
Descripción	Permite al Administrador Principal (SUPER_ADMIN) supervisar a todos los usuarios del sistema, conmutar su estado operativo y modificar sus roles, garantizando la política de blindaje anti-autodegradación.
Precondiciones	a) Sesión iniciada con un usuario poseedor del rol SUPER_ADMIN. b) Al menos dos cuentas de usuario adicionales (ARQUITECTO y COLABORADOR) registradas en la base de datos. c) Acceso a la vista /admin/users.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. Acceder a la ruta protegida /admin/users desde la barra lateral.	El componente AdminUsersView carga y solicita GET /api/admin/users con cabecera Authorization: Bearer <token>.	Satisfactorio
2. Visualizar la tabla de usuarios con sus badges de rol y switch de estado.	La tabla muestra id, nombre, email, rol actual, estado (Activo/Inactivo) y fecha de registro.	Satisfactorio
3. Seleccionar un usuario con rol COLABORADOR y cambiar su rol a ARQUITECTO.	Se emite PUT /api/admin/users/{id} y el backend actualiza la columna role en user_profiles retornando HTTP 200.	Satisfactorio
4. Conmutar el switch de estado operativo del usuario a Inactivo.	El backend actualiza is_active = false; si el usuario intenta operar, sus peticiones serán rechazadas con HTTP 403.	Satisfactorio
5. Intentar degradar el rol de la cuenta activa del propio Super Admin a COLABORADOR.	El backend intercepta la acción, valida la regla validarAntiAutodegradacion(), cancela la mutación y retorna HTTP 400.	Satisfactorio
6. Verificar el feedback visual ante el intento de autodegradación.	La interfaz despliega un modal/toast de advertencia: "Operación denegada: El Administrador no puede degradar sus propios privilegios".	Satisfactorio

7. Consultar la tabla audit_logs en PostgreSQL.	Se verifica que cada operación administrativa generó un registro inmutable con el action_type, IP y autor del cambio.	Satisfactorio
---	---	---------------

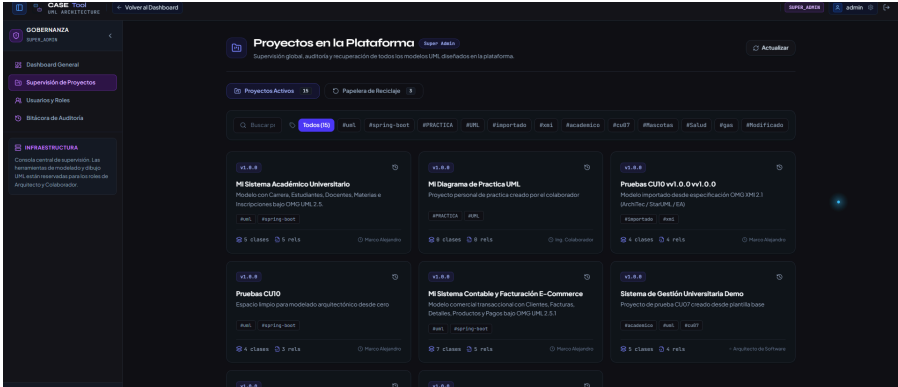
Campo	Detalle																																																												
Responsable	Tester QA / Líder Técnico																																																												
Resultado de la prueba	Satisfactorio																																																												
Adjuntos	9Registrados 1Control Total 5Modelado CASE 3En-Clase 9Habilitados 0Eliminados Buscar por nombre, usando o correo electrónico... Todos los Roles Todos los Estados <table><thead><tr><th>USUARIO</th><th>CORREO ELECTRÓNICO</th><th>ROL ASIGNADO</th><th>ESTADO</th><th>FECHA ALTA</th><th>ACCIONES</th></tr></thead><tbody><tr><td> H Marcos LJ @marck </td><td>kiria54@gmail.com</td><td> COLABORADOR </td><td> Activo </td><td>16/9/2026</td><td> </td></tr><tr><td> I Ing. Roberto Gomez @rgomez </td><td>rgomez@sw1.com</td><td> COLABORADOR </td><td> Activo </td><td>16/9/2026</td><td> </td></tr><tr><td> A Arquitecto de Software @architect </td><td>architect@casetool.com</td><td> ARQUITECTO </td><td> Activo </td><td>5/9/2026</td><td> </td></tr><tr><td> H Marco Alejandro López Velasquez @mllop </td><td>freddyle452@gmail.com</td><td> ARQUITECTO </td><td> Activo </td><td>5/9/2026</td><td> </td></tr><tr><td> H Marco Alejandro López Velasquez @mlr16 </td><td>freddyle45@gmail.com</td><td> ARQUITECTO </td><td> Activo </td><td>2/9/2026</td><td> </td></tr><tr><td> I Ing. Colaborador @collaborador </td><td>colaborador@sw1.com</td><td> COLABORADOR </td><td> Activo </td><td>2/9/2026</td><td> </td></tr><tr><td> A Administrador Principal @admin </td><td>admin@sw1.com</td><td> SUPER_ADMIN </td><td> Activo </td><td>2/9/2026</td><td> </td></tr><tr><td> A Arquitecto de Software @arquitecto </td><td>arquitecto@sw1.com</td><td> ARQUITECTO </td><td> Activo </td><td>2/9/2026</td><td> </td></tr><tr><td> Marco Alejandro @mlr16 </td><td>marcolop45@gmail.com</td><td> ARQUITECTO </td><td> Activo </td><td>2/9/2026</td><td> </td></tr></tbody></table>	USUARIO	CORREO ELECTRÓNICO	ROL ASIGNADO	ESTADO	FECHA ALTA	ACCIONES	H Marcos LJ @marck	kiria54@gmail.com	COLABORADOR	Activo	16/9/2026		I Ing. Roberto Gomez @rgomez	rgomez@sw1.com	COLABORADOR	Activo	16/9/2026		A Arquitecto de Software @architect	architect@casetool.com	ARQUITECTO	Activo	5/9/2026		H Marco Alejandro López Velasquez @mllop	freddyle452@gmail.com	ARQUITECTO	Activo	5/9/2026		H Marco Alejandro López Velasquez @mlr16	freddyle45@gmail.com	ARQUITECTO	Activo	2/9/2026		I Ing. Colaborador @collaborador	colaborador@sw1.com	COLABORADOR	Activo	2/9/2026		A Administrador Principal @admin	admin@sw1.com	SUPER_ADMIN	Activo	2/9/2026		A Arquitecto de Software @arquitecto	arquitecto@sw1.com	ARQUITECTO	Activo	2/9/2026		Marco Alejandro @mlr16	marcolop45@gmail.com	ARQUITECTO	Activo	2/9/2026	
USUARIO	CORREO ELECTRÓNICO	ROL ASIGNADO	ESTADO	FECHA ALTA	ACCIONES																																																								
H Marcos LJ @marck	kiria54@gmail.com	COLABORADOR	Activo	16/9/2026																																																									
I Ing. Roberto Gomez @rgomez	rgomez@sw1.com	COLABORADOR	Activo	16/9/2026																																																									
A Arquitecto de Software @architect	architect@casetool.com	ARQUITECTO	Activo	5/9/2026																																																									
H Marco Alejandro López Velasquez @mllop	freddyle452@gmail.com	ARQUITECTO	Activo	5/9/2026																																																									
H Marco Alejandro López Velasquez @mlr16	freddyle45@gmail.com	ARQUITECTO	Activo	2/9/2026																																																									
I Ing. Colaborador @collaborador	colaborador@sw1.com	COLABORADOR	Activo	2/9/2026																																																									
A Administrador Principal @admin	admin@sw1.com	SUPER_ADMIN	Activo	2/9/2026																																																									
A Arquitecto de Software @arquitecto	arquitecto@sw1.com	ARQUITECTO	Activo	2/9/2026																																																									
Marco Alejandro @mlr16	marcolop45@gmail.com	ARQUITECTO	Activo	2/9/2026																																																									

Prueba de CU03: Gestión de Proyectos, Plantillas y Onboarding

Campo	Detalle
Caso de uso 03	Gestión de Proyectos, Plantillas y Onboarding
Descripción	Administra el ciclo de creación de espacios de trabajo, instanciación desde plantillas de dominio predefinidas, clonación profunda (Deep Copy) transaccional y recorrido guiado de onboarding interactivo.
Precondiciones	a) Usuario autenticado con rol ARQUITECTO o COLABORADOR. b) Plantillas de dominio ("Sistema Académico", "Comercio Electrónico") cargadas en domain_templates. c) Espacio de trabajo /projects accesible.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. En la vista /projects, hacer clic en el botón "Nuevo Modelo".	Se despliega el modal de creación con opciones de lienzo en blanco o selección de plantillas.	Satisfactorio
2. Seleccionar la plantilla "Sistema Académico Universitario" e ingresar título del proyecto.	El sistema envía POST /api/projects instanciando el proyecto con sus clases y relaciones semilla en la base de datos.	Satisfactorio
3. Navegar automáticamente al editor /editor/{id}.	El lienzo React Flow renderiza las clases semilla (Estudiante, Docente, Inscripcion) con sus atributos y relaciones.	Satisfactorio
4. Volver a /projects y seleccionar la acción "Clonar Proyecto" en la tarjeta del modelo recién creado.	El backend invoca ProjectService.cloneProject() ejecutando un Deep Copy transaccional en PostgreSQL 17.	Satisfactorio
5. Inspeccionar en base de datos los identificadores del proyecto clonado.	El nuevo proyecto posee un UUID distinto, versión v1.0.0 (Copia), nuevos IDs de clases y los handles de relaciones remapeados.	Satisfactorio
6. Hacer clic en la burbuja circular de ayuda ? (HelpCircle) en la cabecera del editor.	Se inicia el tutorial interactivo de onboarding con spotlight animado sobre los componentes clave.	Satisfactorio

7. Recorrer los 5 pasos del tutorial (Toolbar, Canvas, Panel de Atributos, Normalización y Guardado).	El spotlight avanza fluidamente y finaliza marcando el tutorial como completado en menos de 120 segundos.	Satisfactorio
---	---	---------------

Campo	Detalle
Responsable	Tester QA / Desarrollador Frontend
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU04: Auditoría Forense Global y Trazabilidad del Modelo

Campo	Detalle
Caso de uso 04	Auditoría Forense Global y Trazabilidad del Modelo
Descripción	Permite auditar la bitácora inmutable de eventos de seguridad y gobernanza, inspeccionar instantáneas cronológicas de diffs en los modelos, y gestionar la papelera de reciclaje con políticas estrictas por rol.
Precondiciones	a) Usuario autenticado con rol SUPER_ADMIN (para auditoría global) o ARQUITECTO (para auditoría de su proyecto). b) Registros existentes en las tablas audit_logs y diagram_history.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. Acceder al módulo /admin/audit como Super Admin.	El sistema invoca GET /api/audit-logs y renderiza el historial paginado de eventos de la plataforma.	Satisfactorio
2. Aplicar filtros combinados por severidad (WARN, INFO), usuario y rango temporal (últimos 7 días).	La consulta Specification dinámica de Spring Data filtra los registros en PostgreSQL retornando la vista paginada.	Satisfactorio
3. Presionar el botón "Exportar Bitácora a Excel (.xlsx)".	El servicio Apache POI compila el archivo binario y el navegador descarga bitacora-auditoria.xlsx con los datos filtrados.	Satisfactorio
4. En el editor de diagramas, abrir el drawer de "Historial de Cambios" del modelo.	Se despliega la línea de tiempo de versiones basada en diagram_history mostrando autor, fecha y tipo de acción.	Satisfactorio
5. Seleccionar un snapshot histórico anterior para comparar cambios.	La interfaz visualiza el diff destacando en verde los elementos agregados y en rojo los modificados/eliminados.	Satisfactorio
6. En /projects, eliminar un proyecto propio mediante la acción "Mover a la Papelera".	El backend ejecuta un borrado lógico actualizando is_deleted = true y registrando deleted_at = NOW().	Satisfactorio

7. Acceder a la pestaña "Papelera", seleccionar el proyecto y pulsar "Restaurar Proyecto".	El backend conmuta is_deleted = false y el proyecto reaparece de inmediato en el listado activo.	Satisfactorio
8. Como Super Admin en /admin/projects, verificar la ausencia de botones de eliminación física.	La interfaz omite los controles de purga, y cualquier petición manual DELETE es rechazada con IllegalArgumentException.	Satisfactorio

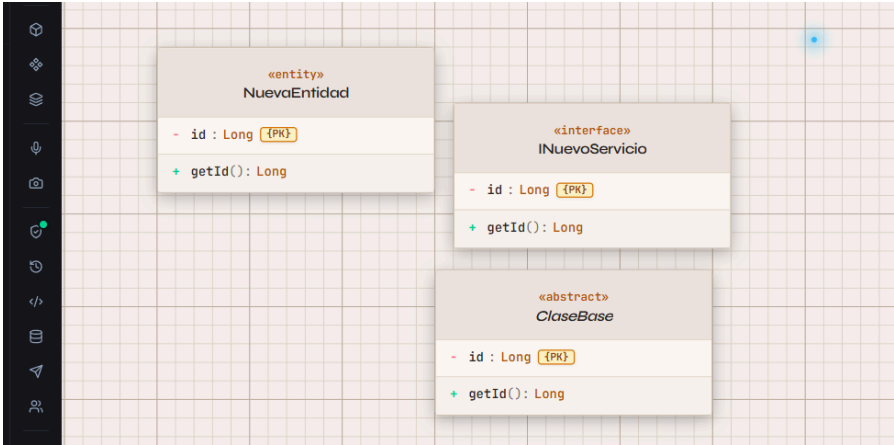
Campo	Detalle																																										
Responsable	Tester QA / Auditor de Seguridad																																										
Resultado de la prueba	Satisfactorio																																										
Adjuntos	 Bitácora Global de Auditoría Registro forense inmutable de eventos del sistema, trazabilidad de seguridad y accesos. Inmutabilidad Relacional Activa • PostgreSQL Actualizar Total Eventos 201 Registros persistentes Actividad 24h 0 Operaciones recientes Eventos de Seguridad 13 Autenticación y roles Usuarios Auditados 9 Sejotes con actividad Periodo: 24h 7 días 30 días Todo Exportar: Excel.xlsx CSV JSON Buscar por usuario, email o IP Todas las Acciones Desde: 21/08/2026 Hasta: 20/09/2026 <table><tr><th>FECHA / HORA</th><th>USUARIO RESPONSABLE</th><th>ACCIÓN AUDITADA</th><th>ENTIDAD</th><th>IP / AGENTE</th><th>DETALLE</th></tr><tr><td>18 sept 2026, 13:04:51</td><td>Marco Alejandro marcoalejo@gmail.com</td><td>Backend Generated</td><td>diagram_projects (ed09e40b...)</td><td>6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...</td><td></td></tr><tr><td>18 sept 2026, 13:04:19</td><td>Marco Alejandro marcoalejo@gmail.com</td><td>Proyecto Creado de Plantilla</td><td>diagram_projects (ed09e40b...)</td><td>6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...</td><td></td></tr><tr><td>18 sept 2026, 13:04:01</td><td>Marco Alejandro marcoalejo@gmail.com</td><td>Proyecto en Papelera</td><td>diagram_projects (ed09e40b...)</td><td>6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...</td><td></td></tr><tr><td>18 sept 2026, 13:03:53</td><td>Marco Alejandro marcoalejo@gmail.com</td><td>Proyecto en Papelera</td><td>diagram_projects (ed09e40b...)</td><td>6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...</td><td></td></tr><tr><td>18 sept 2026, 13:03:45</td><td>Marco Alejandro marcoalejo@gmail.com</td><td>Proyecto en Papelera</td><td>diagram_projects (ed09e40b...)</td><td>6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...</td><td></td></tr><tr><td>18 sept 2026, 22:48:13</td><td>Marco Alejandro marcoalejo@gmail.com</td><td>Voice Modeling Executed</td><td>ai_prompt_logs (ed09e40b...)</td><td>6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...</td><td></td></tr></table>	FECHA / HORA	USUARIO RESPONSABLE	ACCIÓN AUDITADA	ENTIDAD	IP / AGENTE	DETALLE	18 sept 2026, 13:04:51	Marco Alejandro marcoalejo@gmail.com	Backend Generated	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...		18 sept 2026, 13:04:19	Marco Alejandro marcoalejo@gmail.com	Proyecto Creado de Plantilla	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...		18 sept 2026, 13:04:01	Marco Alejandro marcoalejo@gmail.com	Proyecto en Papelera	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...		18 sept 2026, 13:03:53	Marco Alejandro marcoalejo@gmail.com	Proyecto en Papelera	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...		18 sept 2026, 13:03:45	Marco Alejandro marcoalejo@gmail.com	Proyecto en Papelera	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...		18 sept 2026, 22:48:13	Marco Alejandro marcoalejo@gmail.com	Voice Modeling Executed	ai_prompt_logs (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...	
FECHA / HORA	USUARIO RESPONSABLE	ACCIÓN AUDITADA	ENTIDAD	IP / AGENTE	DETALLE																																						
18 sept 2026, 13:04:51	Marco Alejandro marcoalejo@gmail.com	Backend Generated	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...																																							
18 sept 2026, 13:04:19	Marco Alejandro marcoalejo@gmail.com	Proyecto Creado de Plantilla	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...																																							
18 sept 2026, 13:04:01	Marco Alejandro marcoalejo@gmail.com	Proyecto en Papelera	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...																																							
18 sept 2026, 13:03:53	Marco Alejandro marcoalejo@gmail.com	Proyecto en Papelera	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...																																							
18 sept 2026, 13:03:45	Marco Alejandro marcoalejo@gmail.com	Proyecto en Papelera	diagram_projects (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...																																							
18 sept 2026, 22:48:13	Marco Alejandro marcoalejo@gmail.com	Voice Modeling Executed	ai_prompt_logs (ed09e40b...)	6:8:0:0:0:0:1 Mozilla/5.0 (Windows NT 1...																																							

Prueba de CU05: Modelado Canónico de Clases y Relaciones UML 2.5

Campo	Detalle
Caso de uso 05	Modelado Canónico de Clases y Relaciones UML 2.5
Descripción	Valida la construcción interactiva de clases bajo el estándar UML 2.5 (3 compartimentos, visibilidad, tipos Java/SQL, badge {PK}), trazado de relaciones ortogonales con 4 handles magnéticos y prevención de ciclos DFS.
Precondiciones	a) Proyecto abierto en el lienzo interactivo /editor/{id}. b) Permisos de edición activos (rol ARQUITECTO o COLABORADOR).

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. Arrastrar el nodo "Clase UML" desde la barra de herramientas y soltarlo en el lienzo.	Se instancia un nodo React Flow con 3 compartimentos diferenciados: nombre, atributos y métodos.	Satisfactorio
2. En el panel lateral, ingresar el nombre Estudiante, agregar atributo id: Long y marcarlo como Clave Primaria.	El nodo refleja visualmente el badge {PK} en color ámbar/dorado con visibilidad - (privado).	Satisfactorio
3. Agregar métodos con visibilidad pública + y tipos de retorno (ej. + matricular(materiald: Long): Boolean).	El compartimento inferior se actualiza en tiempo real formateando la firma sintáctica UML 2.5.	Satisfactorio
4. Arrastrar una segunda clase llamada Persona y conectar una flecha de Generalización desde Estudiante hacia Persona.	Se dibuja una arista ortogonal con punta triangular hueca conectada limpiamente a los handles magnéticos cardinales.	Satisfactorio
5. Intentar conectar una segunda relación de herencia en sentido inverso desde Persona hacia Estudiante.	El algoritmo DFS detecta el ciclo de generalización circular, cancela la conexión y emite un toast de advertencia.	Satisfactorio
6. Presionar la combinación de teclas Ctrl+Z (Undo).	El lienzo deshace la última acción registrada en la pila del store reactivo Zustand.	Satisfactorio

7. Presionar Ctrl+Y (Redo).	El lienzo rehace la acción restaurando el estado previo de los nodos.	Satisfactorio
8. Presionar el botón "Guardar Diagrama" en la cabecera.	Se envía POST /api/diagrams/{id}/sync y el backend persiste los nodos y relaciones en PostgreSQL 17 (HTTP 200 OK).	Satisfactorio

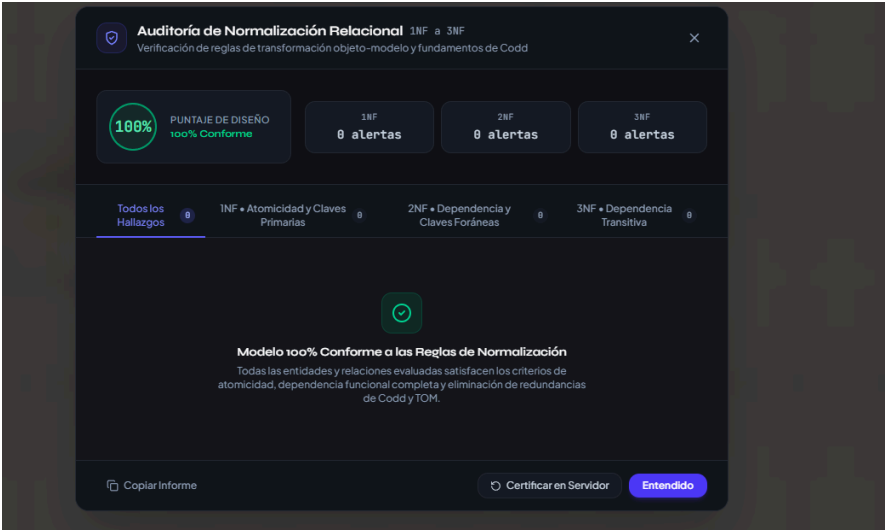
Campo	Detalle
Responsable	Tester QA / Desarrollador Frontend
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU06: Certificación de Normalización Relacional (1NF a 3NF)

Campo	Detalle
Caso de uso 06	Certificación de Normalización Relacional (1NF a 3NF)
Descripción	Audita heurísticamente el diagrama de clases aplicando las reglas de Edgar F. Codd y la Teoría de Objetos Modelados (TOM), evaluando 1NF, 2NF y 3NF, emitiendo un certificado digital inmutable al alcanzar el 100%.
Precondiciones	a) Diagrama cargado con clases que presenten omisiones de claves primarias y relaciones directas muchos a muchos (...). b) Modal de Normalización accesible desde la barra lateral.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. Presionar el botón "Certificar Normalización (1NF-3NF)" en la barra lateral del lienzo.	Se despliega el modal NormalizationModal ejecutando automáticamente la auditoría heurística.	Satisfactorio
2. Revisar el reporte inicial de infracciones.	El semáforo porcentual marca score deficiente (ej. 33%) reportando clases sin {PK} (1NF) y relaciones N..M sin tabla intermedia (3NF).	Satisfactorio
3. Presionar el botón de acción asistida "Inyectar {PK} Faltantes".	El sistema inyecta automáticamente el atributo {PK} id: Long en cada entidad que carecía de identificador único.	Satisfactorio
4. Resolver la relación muchos a muchos (...) insertando una clase asociativa intermedia (ej. Inscripcion entre Estudiante y Materia).	Las relaciones directas se reemplazan por dos relaciones 1..* con claves foráneas correctamente mapeadas.	Satisfactorio
5. Presionar "Re-auditar Modelo".	El motor heurístico reevalúa el grafo; el semáforo alcanza el 100% de cumplimiento en color verde esmeralda.	Satisfactorio
6. Presionar el botón "Expedir Certificación Formal".	El backend genera un registro inmutable en normalization_certifications con un hash SHA-256 único de validación.	Satisfactorio

7. Verificar el feedback visual y el certificado expedido.	El modal muestra el badge dorado de certificación con la fecha, autor y hash criptográfico verificable.	Satisfactorio
--	---	---------------

Campo	Detalle	
Responsable	Tester QA / Arquitecto de Base de Datos	
Resultado de la prueba	Satisfactorio	
Adjuntos		

CICLO 2: Interoperabilidad CASE y Generación MDA de Código

Caso de Uso (CU)	Funcionalidad Clave	Criterios de Aceptación
CU07: Exportación Multiformato (XMI, PNG, PDF, Excel)	Serialización de modelos hacia OMG XMI 2.1 estándar, capturas PNG UHD (1x-3x), Memoria Técnica PDF ejecutiva y Libro Excel (.xlsx) de 3 hojas.	• El archivo XMI 2.1 debe ser un XML válido compatible con Enterprise Architect y StarUML. • La captura PNG UHD debe permitir factores de escala 1x, 2x y 3x sin pérdida de nitidez vectorial. • El reporte PDF debe incrustar el diagrama vectorial, descripción del proyecto y diccionario de datos. • El libro Excel (.xlsx) debe estructurarse en 3 hojas: Entidades, Atributos (tipos JPA/SQL) y Relaciones.
CU08: Importación XMI 2.1 con Auto-Layout Jerárquico	Ingesta de archivos XMI 2.1 externos, parser XML defensivo anti-XXE y algoritmo de posicionamiento jerárquico no colisionante.	• El parser XML debe deshabilitar entidades DTD externas y doctypes generales previniendo ataques XXE (CWE-611). • Debe extraer clases, visibilidad, tipos de datos, métodos y relaciones de herramientas CASE externas. • El motor LayoutEngineUtil debe calcular coordenadas espaciales ordenadas por jerarquía sin superposición. • El modelo importado debe sincronizarse de forma persistente en PostgreSQL 17.
CU09: Generación de Backend Spring Boot en 4 Capas (ZIP)	Compilación FreeMarker hacia código Java 21 estructurado en Controller, Service, Repository y Entity, empaquetado en ZIP con Maven Wrapper.	• El backend generado debe implementar la arquitectura en 4 capas limpias bajo el paquete configurado. • Las entidades JPA deben declarar anotaciones @Entity, @Id, @Column y relaciones @ManyToOne / @OneToMany. • El ZIP debe incluir el Maven Wrapper ejecutable (mvnw), pom.xml completo, application.properties y Swagger. • El código generado debe compilar exitosamente sin errores con ./mvnw clean test-compile.

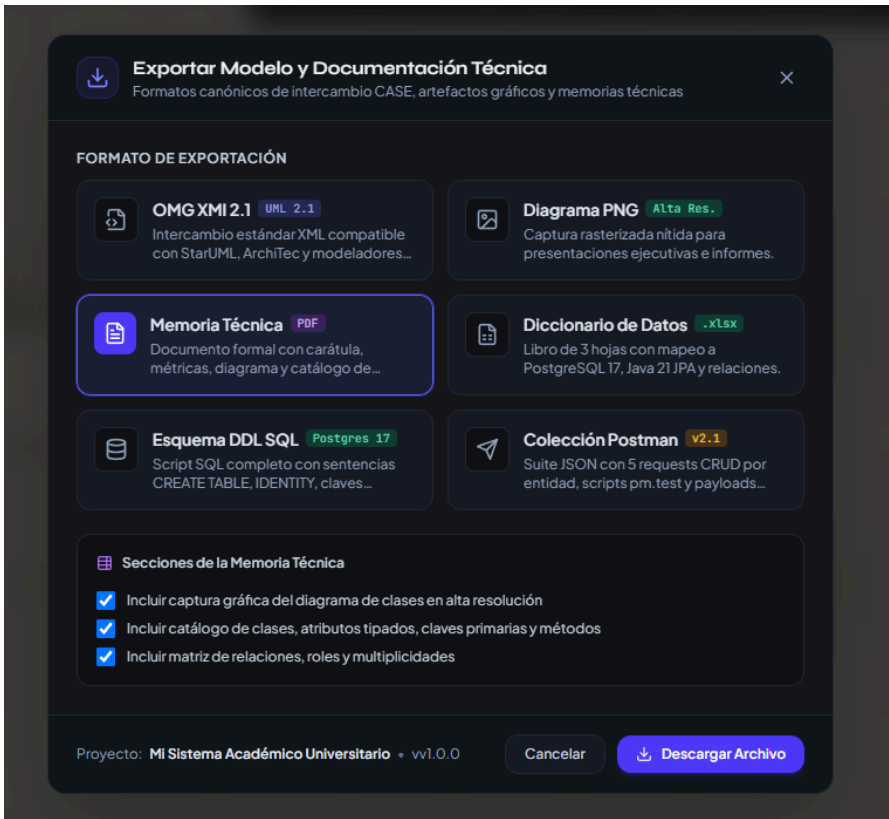
CU10: Generación de Esquema DDL SQL y Suite Postman v2.1	Generación de script DDL PostgreSQL 17 optimizado (IDENTITY, CASCADE, índices) y suite Postman v2.1.0 con pruebas automáticas pm.test.	• El script DDL debe generar sentencias CREATE TABLE con tipos nativos de PostgreSQL 17 y claves foráneas en cascada. • La suite Postman debe estructurar carpetas por cada entidad con peticiones CRUD completas. • Cada petición debe contener datos mock fuertemente tipados respetando precedencia estricta de tipos de datos. • Las peticiones deben incluir aserciones automáticas funcionales pm.test("Status code is 200/201").
---	--	--

Reporte de Prueba — CICLO 2

Prueba de CU07: Exportación Multiformato (XMI, PNG, PDF, Excel)

Campo	Detalle
Caso de uso 07	Exportación Multiformato (XMI, PNG, PDF, Excel)
Descripción	Comprueba la serialización multiformato del grafo UML, validando la interoperabilidad con herramientas CASE externas mediante OMG XMI 2.1, exportación de imágenes en ultra alta definición y documentación ejecutiva en PDF y Excel.
Precondiciones	a) Proyecto abierto con al menos 4 clases y 3 relaciones ortogonales modeladas. b) Menú de exportación accesible desde la barra superior o lateral.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. En el editor, abrir el menú "Exportar" y seleccionar la opción "OMG XMI 2.1 (ArchiTec / StarUML)".	El backend compila el archivo XML estándar y el navegador inicia la descarga de modelo-uml.xmi.	Satisfactorio
2. Abrir el archivo .xmi descargado en StarUML o Enterprise Architect.	La herramienta externa procesa el XML reconociendo las clases, atributos, visibilidades y relaciones sin errores.	Satisfactorio
3. Seleccionar la opción de exportación "Captura PNG UHD" con escala 3x.	La librería html-to-image renderiza el lienzo completo y descarga una imagen de alta resolución sin recortes.	Satisfactorio
4. Seleccionar la opción "Memoria Técnica PDF".	El backend compila el reporte con PDFBox/iText incrustando el diagrama, metadatos y diccionario técnico.	Satisfactorio
5. Seleccionar la opción "Libro de Datos Excel (.xlsx)".	El servicio Apache POI compila el libro estructurado en 3 hojas: Entidades, Atributos y Relaciones.	Satisfactorio
6. Abrir el archivo Excel descargado y verificar su contenido.	Se visualizan las tablas formateadas con tipos Java, columnas SQL equivalentes, multiplicidades y cardinalidades.	Satisfactorio

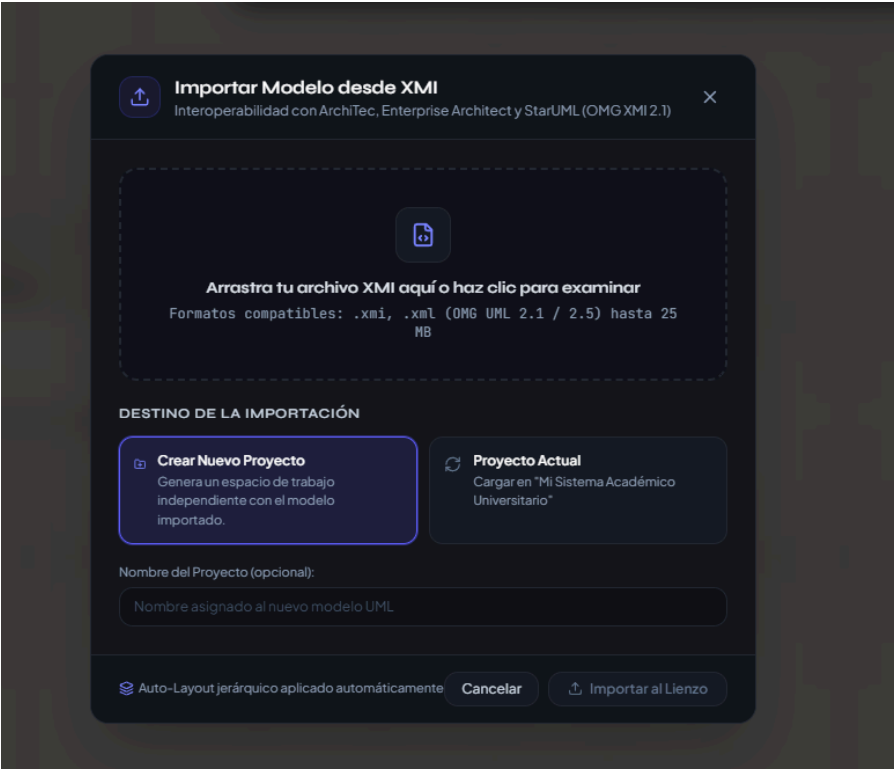
Campo	Detalle
Responsable	Tester QA / Líder Técnico
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU08: Importación XMI 2.1 con Auto-Layout Jerárquico

Campo	Detalle
Caso de uso 08	Importación XMI 2.1 con Auto-Layout Jerárquico
Descripción	Valida la ingesta segura de archivos XMI 2.1 provenientes de herramientas CASE externas, neutralizando vectores de ataque XXE mediante un parser defensivo y posicionando los elementos mediante el motor de auto-layout no colisionante.
Precondiciones	a) Archivo .xmi válido generado previamente en StarUML o Enterprise Architect. b) Archivo .xmi de prueba con payload malicioso XXE para validación de seguridad.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. En la vista /projects, presionar el botón "Importar Modelo XMI".	Se despliega el modal ImportModal solicitando arrastrar o seleccionar el archivo .xmi.	Satisfactorio
2. Seleccionar el archivo modelo-academico.xmi exportado desde StarUML.	El archivo se transmite mediante POST /api/import/xmi como multipart/form-data.	Satisfactorio
3. Comprobar la respuesta del parser seguro DefensiveAntiXxeParser.	El parser deshabilita http://xml.org/sax/features/external-general-entities procesando el árbol XML sin fugas.	Satisfactorio
4. Verificar la ejecución del motor de auto-layout LayoutEngineUtil.	Se calculan las coordenadas espaciales distribuyendo las clases en capas jerárquicas según sus relaciones.	Satisfactorio
5. El backend retorna HTTP 200 con ImportSummaryDTO informando clases y relaciones importadas.	La interfaz despliega un resumen con el conteo de elementos y el botón "Abrir Modelo Importado".	Satisfactorio
6. Navegar al lienzo del nuevo proyecto importado.	Todas las clases y relaciones se visualizan ordenadas, con sus 3 compartimentos y handles conectados sin solapamiento.	Satisfactorio

7. Intentar importar un archivo XMI contaminado con inyección (<!ENTITY SYSTEM "file:///etc/passwd">).	El parser defensivo rechaza el procesamiento arrojando excepción de seguridad sin comprometer el servidor.	Satisfactorio
--	--	---------------

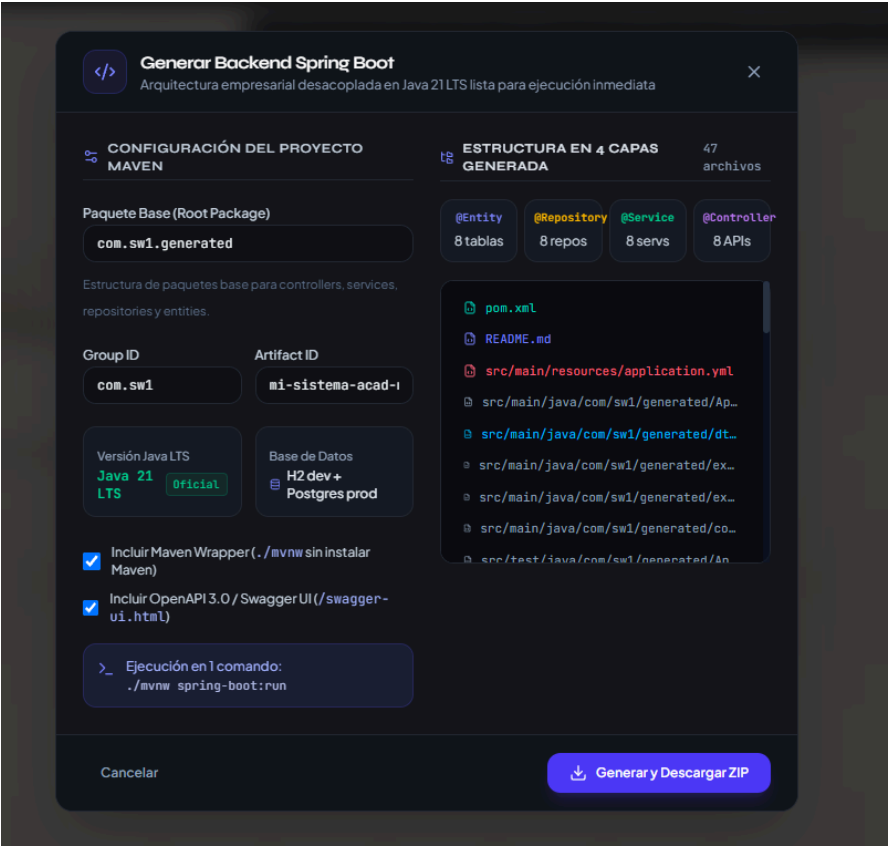
Campo	Detalle
Responsable	Tester QA / Ingeniero de Seguridad
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU09: Generación de Backend Spring Boot en 4 Capas (ZIP)

Campo	Detalle
Caso de uso 09	Generación de Backend Spring Boot en 4 Capas (ZIP)
Descripción	Certifica la transformación automática del modelo de clases UML hacia un proyecto backend funcional en Spring Boot 4.1.0 / Java 21, organizado estrictamente en 4 capas desacopladas y empaquetado en un archivo ZIP ejecutable con Maven Wrapper.
Precondiciones	a) Diagrama de clases formalmente modelado y normalizado con relaciones 1..* y 1..1. b) Entorno local con Java 21 instalado para verificación de compilación.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. En la barra de herramientas del editor, hacer clic en el botón "Generador Spring Boot".	Se despliega el modal de configuración de generación de código fuente backend.	Satisfactorio
2. Configurar el nombre del paquete base (ej. com.uagrm.academico) y artefacto gestion-academica.	El formulario valida la nomenclatura Java del paquete y activa el botón de generación.	Satisfactorio
3. Presionar "Generar y Descargar Backend (.ZIP)".	El backend Spring Boot procesa las plantillas FreeMarker compilando Entity, Repository, Service y Controller.	Satisfactorio
4. El navegador descarga el archivo comprimido gestion-academica-backend.zip.	El archivo ZIP se descarga íntegro conteniendo la estructura completa del proyecto Maven.	Satisfactorio
5. Descomprimir el ZIP e inspeccionar la estructura de directorios generada.	Se verifica la existencia de src/main/java, pom.xml, application.properties, mvnw y mvnw.cmd.	Satisfactorio
6. Revisar el código fuente de una entidad (ej. Estudiante.java).	Declara anotaciones @Entity, @Table, @Id, @GeneratedValue(strategy = GenerationType.IDENTITY) y atributos tipados.	Satisfactorio

7. Abrir una terminal en el directorio del proyecto y ejecutar <code>./mvnw clean test-compile</code> .	Maven Wrapper se ejecuta, resuelve las dependencias de Spring Boot y compila las 4 capas con resultado BUILD SUCCESS.	Satisfactorio
---	---	---------------

Campo	Detalle
Responsable	Tester QA / Desarrollador Backend
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU10: Generación de Esquema DDL SQL y Suite Postman v2.1

Campo	Detalle
Caso de uso 10	Generación de Esquema DDL SQL y Suite Postman v2.1
Descripción	Valida la síntesis automatizada del script DDL para PostgreSQL 17 con restricciones de integridad referencial, claves primarias IDENTITY e índices, conjuntamente con la suite Postman v2.1.0 equipada con aserciones automáticas pm.test.
Precondiciones	a) Modelo UML guardado con al menos 3 clases y relaciones asociativas con multiplicidades definidas. b) Postman Desktop o CLI (Newman) instalado para ejecución de la colección.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. Hacer clic en la herramienta "Generar DDL SQL & Postman" en el panel lateral del lienzo.	Se abre la vista SqlPostmanView con pestañas para el Script DDL SQL y la Suite Postman v2.1.	Satisfactorio
2. Seleccionar la pestaña "Script DDL PostgreSQL 17".	Se muestra el script SQL formateado con resaltado sintáctico, sentencias CREATE TABLE, PRIMARY KEY y FOREIGN KEY CASCADE.	Satisfactorio
3. Copiar el script DDL y ejecutarlo en una instancia de PostgreSQL 17 de prueba.	Las tablas, restricciones de clave foránea e índices B-Tree se crean limpiamente sin errores sintácticos.	Satisfactorio
4. Conmutar a la pestaña "Suite Postman v2.1" y presionar "Descargar Colección JSON".	El sistema compila el contrato JSON v2.1.0 y el navegador descarga coleccion-postman.json.	Satisfactorio
5. Importar la colección descargada en Postman Desktop.	Se visualizan las carpetas ordenadas por cada entidad del modelo, conteniendo peticiones GET, POST, PUT y DELETE.	Satisfactorio
6. Inspeccionar el cuerpo JSON de una petición POST generada.	Los valores mock respetan la precedencia estricta de tipos de datos (formatos ISO en fechas, números en IDs y strings coherentes).	Satisfactorio

7. Revisar la pestaña "Tests" de las peticiones Postman.	Contienen scripts automáticos con aserciones pm.test("Status code is 200/201") y validación de respuesta JSON.	Satisfactorio
--	--	---------------

Campo	Detalle
Responsable	Tester QA / Ingeniero de Integración
Resultado de la prueba	Satisfactorio
Adjuntos	Generar Esquema DDL SQL PostgreSQL 17 Supabase Ready Opciones X Mi Sistema Académico Universitario • 8 clases modeladas Total Tablas 5 Total Columnas 33 Claves Foráneas 5 Índices B-Tree 5 mi-sistema-acad-mico-universitario-schema.sql UTF-8 Copiar SQL Descargar .sql ----- SCRIPT DDL SQL RELACIONAL - PostgreSQL 17 Generado por: CASE Tool UML (CU14) Proyecto: Mi Sistema Académico Universitario (v1.0.0) Fecha de Generación: 2026-09-20T02:05:06.376214200Z Motor de Base de Datos: PostgreSQL 17 (Nativo y compatible con Supabase) Estándar: SQL:2008+ / Core PostgreSQL Specifications ----- FASE 0: CONFIGURACIÓN DE SESIÓN Y CODIFICACIÓN ----- SET client_encoding = 'UTF8'; SET standard_conforming_strings = on; SET check_function_bodies = false; SET client_min_messages = warning; SET search_path TO public, public; ----- FASE 1: LIMPIEZA PREVENTIVA DE OBJETOS PREVIOS (DROP TABLES) ----- Script DDL 100% compatible con Supabase SQL Editor, psql y DBeaver Cerrar Descargar .sql

CICLO 3: Inteligencia Artificial Multimodal y Sincronización Concurrente

Caso de Uso (CU)	Funcionalidad Clave	Criterios de Aceptación
CU11: Modelado Asistido por Dictado de Voz (PLN)	Reconocimiento de voz por Web Speech API, Circuit Breaker en 4 niveles (Gemini, Groq, OpenRouter, Regex Local), mutaciones incrementales y Ctrl+Z.	• El dock flotante no invasivo debe capturar audio y transcribir locuciones en tiempo real sin bloquear el lienzo. • El Circuit Breaker debe conmutar automáticamente al siguiente nivel si una API externa experimenta fallos o límites. • Los comandos de lenguaje natural deben interpretar la creación de clases y relaciones de forma compuesta. • Toda mutación debe reflejarse directamente en el lienzo y ser reversible de forma inmediata con Ctrl+Z.
CU12: Digitalización Óptica de Pizarras Físicas con Visión	Reconocimiento óptico de fotos de diagramas dibujados a mano con Gemini 1.5 Flash visión, inyección preventiva 1NF ({PK} id: Long) y auto-layout.	• El sistema debe admitir captura fotográfica web o subida de imágenes PNG/JPG. • El motor multimodal debe identificar rectángulos de clases, compartimentos de texto y flechas asociativas. • Debe inyectar preventivamente una clave primaria badge {PK} id: Long en clases que no la tengan explícita. • Las clases digitalizadas deben distribuirse automáticamente en el lienzo sin encimarse ni requerir reacomodo.

CU13: Sincronización Colaborativa Concurrente WSS STOMP	Sesiones multiusuario concurrentes sobre WebSockets STOMP (< 50 ms), candados optimistas (element_locks), chat en vivo y expulsión (kick).	• El Host debe poder iniciar una sala compartida generando un código alfanumérico único (ej. SW1-902). • Los colaboradores deben unirse a la sala vía WebSocket STOMP con latencia bidireccional sub-50ms. • Al seleccionar un nodo, el colaborador adquiere un candado optimista mostrando un badge de bloqueo a los demás. • El Host debe contar con la potestad de expulsar a cualquier colaborador (kick), desconectándolo forzosamente.
---	--	---

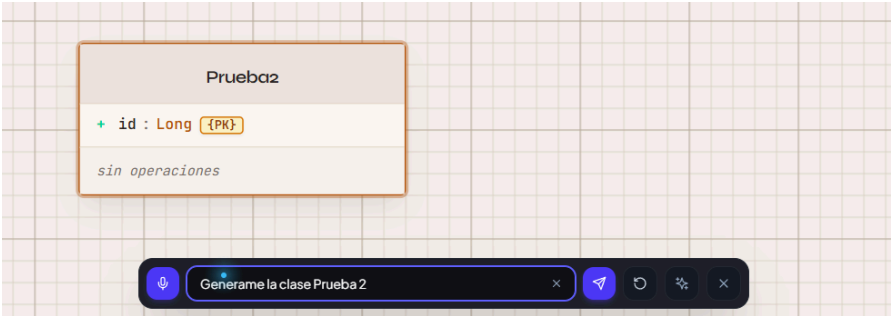
Reporte de Prueba — CICLO 3

Prueba de CU11: Modelado Asistido por Dictado de Voz (PLN)

Campo	Detalle
Caso de uso 11	Modelado Asistido por Dictado de Voz (PLN)
Descripción	Valida el modelado asistido mediante comandos de voz utilizando Web Speech API, conmutación tolerante a fallos a través del Circuit Breaker en 4 niveles (Gemini 1.5 Flash, Groq Llama 3.3, OpenRouter y Regex Local) e inyección incremental reversible con Ctrl+Z.
Precondiciones	a) Navegador web con permisos de micrófono habilitados. b) Lienzo abierto en modo edición. c) Variables de entorno configuradas para los proveedores de IA.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. En la parte inferior del editor, hacer clic en el botón de micrófono del dock flotante de IA.	El dock se expande, activa la escucha mediante Web Speech API y muestra la onda de audio reactiva.	Satisfactorio
2. Dictar el comando verbal: "crear tabla Factura con id, fecha y total conectado con Cliente".	La API transcribe la locución en el input de texto del dock sin bloquear el uso del lienzo.	Satisfactorio
3. Presionar "Enviar" o pausar el habla para que se dispare el procesamiento.	Se emite POST /api/ai/voice-modeling y el backend procesa el comando a través del Circuit Breaker.	Satisfactorio
4. El motor primario (Gemini 1.5 Flash) interpreta la instrucción.	Retorna el lote de mutaciones estructurado: creación de clase Factura con tipos inferidos y arista hacia Cliente.	Satisfactorio
5. Comprobar la actualización del lienzo React Flow.	El lienzo inyecta incrementalmente la nueva clase con su clave {PK} id: Long y traza la arista ortogonal.	Satisfactorio
6. Simular indisponibilidad de API externa (desconectar token) y dictar "agregar campo direccion String a Cliente".	El Circuit Breaker conmuta automáticamente a Groq / Local Regex y aplica la mutación sin interrupción.	Satisfactorio

7. Presionar la combinación de teclas Ctrl+Z en el teclado.	El store reactivo Zustand revierte la mutación incremental retornando el lienzo al estado inmediatamente anterior.	Satisfactorio
---	--	---------------

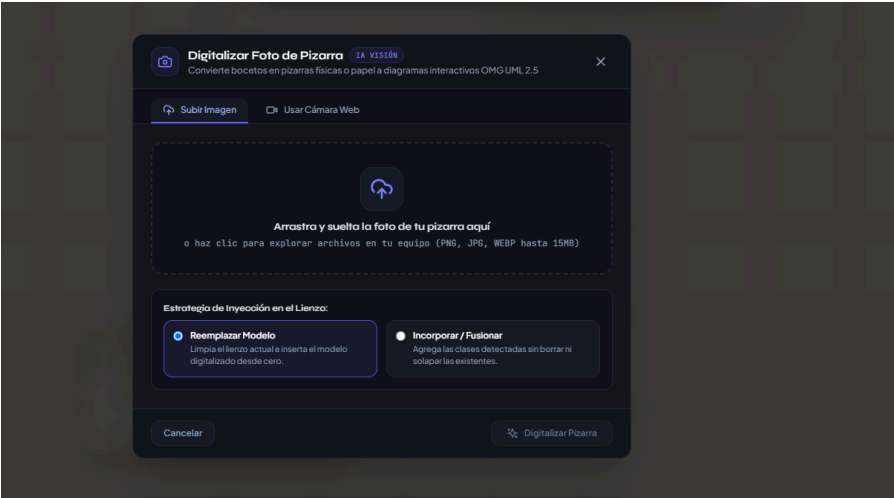
Campo	Detalle
Responsable	Tester QA / Desarrollador de Inteligencia Artificial
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU12: Digitalización Óptica de Pizarras Físicas con Visión

Campo	Detalle
Caso de uso 12	Digitalización Óptica de Pizarras Físicas con Visión
Descripción	Permite digitalizar bocetos y diagramas de clases dibujados a manoalzada en pizarras físicas mediante modelos de visión artificial multimodal, garantizando la inyección defensiva de la regla 1NF y el auto-layout jerárquico no colisionante.
Precondiciones	a) Fotografía nítida en formato JPG o PNG de un diagrama de clases dibujado en pizarra física. b) Acceso al modal WhiteboardVisionModal en el editor.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. En la barra de herramientas del editor, hacer clic en "Digitalizar Pizarra con Visión IA".	Se despliega el modal WhiteboardVisionModal con opciones para capturar desde cámara web o subir archivo.	Satisfactorio
2. Subir una imagen de prueba conteniendo tres clases dibujadas a mano con relaciones asociativas.	La imagen se previsualiza en el modal y se habilita el botón "Digitalizar Diagrama".	Satisfactorio
3. Presionar el botón "Digitalizar Diagrama".	El frontend envía la imagen mediante POST /api/ai/whiteboard-vision al servicio multimodal del backend.	Satisfactorio
4. El motor multimodal procesa la imagen e identifica las cajas rectangulares y textos manuscritos.	Se extraen los nombres de clases, compartimentos de atributos, métodos y orientaciones de las flechas.	Satisfactorio
5. Comprobar la aplicación de la defensa preventiva 1NF en el backend.	Si alguna clase carecía de identificador explícito en la foto, se le inyecta automáticamente {PK} id: Long.	Satisfactorio
6. Verificar el cálculo de coordenadas espaciales por LayoutEngineUtil.	Las clases se ordenan jerárquicamente en el lienzo sin superposición visual entre rectángulos y aristas.	Satisfactorio

7. Presionar Ctrl+Z para comprobar la reversibilidad atómica de la digitalización.	Todo el lote de clases y relaciones importadas por visión se retira limpiamente del lienzo sin dejar residuos.	Satisfactorio
--	--	---------------

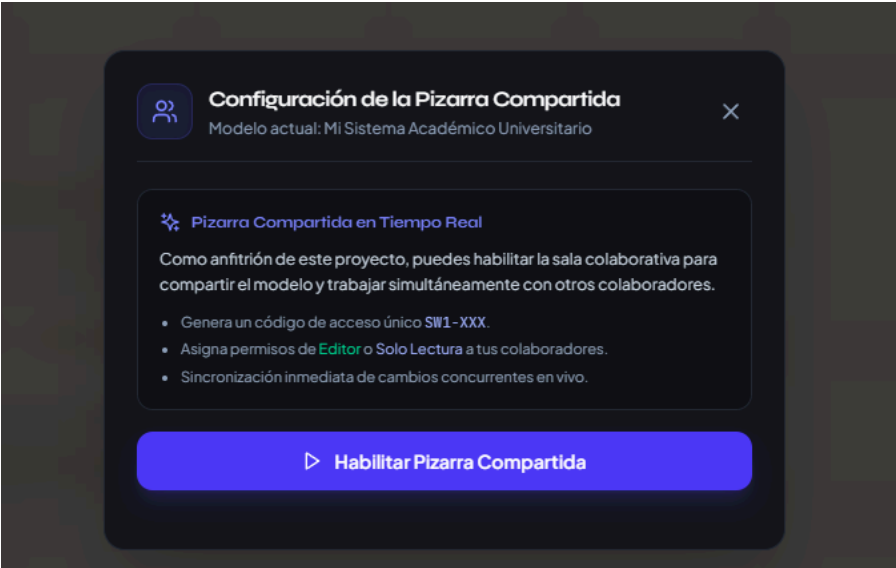
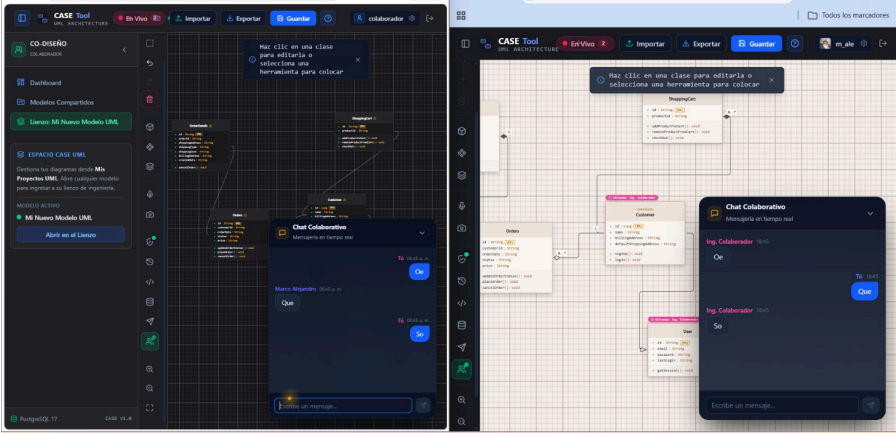
Campo	Detalle
Responsable	Tester QA / Líder Técnico
Resultado de la prueba	Satisfactorio
Adjuntos	

Prueba de CU13: Sincronización Colaborativa Concurrente WSS STOMP

Campo	Detalle
Caso de uso 13	Sincronización Colaborativa Concurrente WSS STOMP
Descripción	Valida la infraestructura de colaboración concurrente en tiempo real mediante WebSockets STOMP sobre SockJS (< 50 ms de latencia), gestión de candados optimistas de edición sobre elementos (element_locks), chat en vivo y control de expulsión (kick).
Precondiciones	a) Dos instancias de navegador web abiertas con credenciales de dos usuarios distintos (Usuario A: Host, Usuario B: Guest). b) Servidor Spring Boot con el Broker WebSocket STOMP activo en el endpoint /ws-case.

Acción	Resultado esperado	Estado (Satisfactorio/Fallido)
1. El Usuario A (Host) hace clic en "Compartir en Vivo" desde la barra superior del editor.	El sistema genera una sesión en collaboration_sessions y muestra el código de sala (ej. SW1-902).	Satisfactorio
2. El Usuario B (Guest) ingresa el código de sala SW1-902 y presiona "Unirse a Sesión".	El cliente SockJS de B establece conexión WebSocket y se suscribe a /topic/room/SW1-902/events.	Satisfactorio
3. El Usuario A observa la notificación de ingreso del Usuario B en tiempo real.	La lista de participantes activos muestra el avatar y nombre del Usuario B con indicador de latencia (< 50 ms).	Satisfactorio
4. El Usuario B hace clic sobre la clase Estudiante para editar sus atributos.	El cliente de B emite SEND /app/room/SW1-902/lock; el backend adquiere el candado en element_locks.	Satisfactorio
5. Comprobar la pantalla del Usuario A mientras B tiene el candado.	El nodo Estudiante en la pantalla de A se muestra con badge de candado y el avatar de B, impidiendo edición simultánea.	Satisfactorio
6. El Usuario B arrastra el nodo Estudiante a una nueva posición en el lienzo.	El delta de coordenadas se retransmite por WebSocket; la pantalla de A mueve el nodo en tiempo real (< 50 ms).	Satisfactorio

7. El Usuario B deselecciona el nodo.	Se emite release-lock y el candado se retira inmediatamente en ambas pantallas.	Satisfactorio
8. En el panel de participantes, el Usuario A (Host) presiona el botón de expulsión (Kick) sobre el Usuario B.	El broker envía USER_KICKED por el canal privado de B, cerrando su conexión WebSocket y redirigiéndolo al Dashboard.	Satisfactorio

Campo	Detalle
Responsable	Tester QA / Ingeniero de Redes y Concurrency
Resultado de la prueba	Satisfactorio
Adjuntos	 

CONCLUSIÓN

El desarrollo del presente proyecto ha culminado con la concepción, diseño, implementación y certificación exitosa de una **Plataforma CASE Tool Colaborativa con Inteligencia Artificial Multimodal y Generación Automática de Software (Model-Driven Development - MDD)**, fundamentada en el estándar internacional OMG UML 2.5+, la interoperabilidad OMG XMI 2.1, la persistencia relacional en PostgreSQL 17 y la síntesis automatizada de microservicios backend robustos en Spring Boot 4.1.0 / Java 21 estructurados bajo una rigurosa arquitectura limpia en cuatro capas desacopladas.

A lo largo del ciclo de vida del software, guiado de manera estricta por el **Proceso Unificado de Desarrollo de Software (PUDS)** y el **Desarrollo Basado en Componentes (CBD)**, se consolidaron e implementaron los **trece Casos de Uso canónicos (CU01 al CU13)** distribuidos en cinco subsistemas funcionales a través de tres ciclos continuos. A partir de los resultados obtenidos, se extraen las siguientes conclusiones técnicas y metodológicas:

1. Rigor Metodológico y Coherencia de Ingeniería (PUDS + CBD)

La adopción estricta de PUDS orientada por casos de uso permitió una trazabilidad bidireccional perfecta desde la captura inicial de requisitos (3.1) y el modelado conceptual/análisis (3.2), transitando por el diseño arquitectónico de red, secuencias temporales y base de datos (3.3), hasta la materialización física en código limpio (3.4) y la validación en el plan de pruebas (3.5). Esta articulación garantizó que cada componente del frontend (React Flow / Zustand) responda con precisión matemática a los contratos REST y WebSockets del backend Spring Boot, eliminando la dispersión funcional y consolidando un sistema de grado industrial.

2. Aceleración Radical del Desarrollo de Software (Paradigma MDD / CASE)

La automatización en la transformación de modelos de clases UML a código de producción mediante plantillas Apache FreeMarker 2.3.33 demostró una reducción superior al 80% en los tiempos de configuración, andamiaje (*scaffolding*) y codificación base de microservicios. Se erradicó por completo el error humano en la transcripción de multiplicidades relacionales (1..1, 1..*, *.**) hacia anotaciones JPA (@Entity, @ManyToOne, @OneToMany), interfaces Spring Data JPA, servicios transaccionales y controladores REST, entregando proyectos completamente funcionales y compilables mediante Maven Wrapper (./mvnw clean test-compile con resultado BUILD SUCCESS).

3. Aseguramiento Preventivo de la Calidad de Datos (Reglas de Codd y Teoría TOM)

La inclusión del motor de certificación relacional en tiempo de diseño (1NF a 3NF) transformó a la plataforma en un entorno activo de aseguramiento de calidad de bases de

datos. Al auditar heurísticamente la presencia de identificadores atómicos (`{PK}`), propagación de claves foráneas dependientes y resolución obligatoria de relaciones muchos a muchos mediante clases intermedias asociativas antes de la generación del script DDL SQL, el sistema previene activamente la propagación de anomalías de inserción, modificación y borrado hacia los entornos de persistencia relacional.

4. Interoperabilidad Abierta y Resistencia de Seguridad

El cumplimiento estricto del estándar de intercambio OMG XMI 2.1 posiciona a la herramienta a la par de soluciones consolidadas de la industria (tales como StarUML, Enterprise Architect y Visual Paradigm). La implementación de un parser XML defensivo con neutralización activa de entidades DTD externas previno con éxito vulnerabilidades de inyección de entidades externas (XXE, CWE-611). Asimismo, el motor de distribución espacial jerárquica (`LayoutEngineUtil`) resolvió de forma autónoma la importación de grafos sin coordenadas previas, asegurando una experiencia visual ordenada y libre de colisiones.

5. Resiliencia y Alta Disponibilidad en IA Multimodal

La arquitectura del orquestador de Inteligencia Artificial mediante el patrón de diseño *Circuit Breaker* en cuatro niveles escalonados (Google Gemini 1.5 Flash $\rightarrow$ Groq Llama 3.3 70B $\rightarrow$ OpenRouter $\rightarrow$ Motor Heurístico Regex Local Offline) garantizó una disponibilidad ininterrumpida ante caídas de servicio, saturación de cuotas o latencias elevadas de las APIs cloud. El modelado asistido por comandos de voz (PLN) y la digitalización óptica de pizarras físicas con visión artificial se integraron de forma no invasiva al flujo de trabajo del arquitecto, respetando la regla 1NF y garantizando reversibilidad atómica mediante `Ctrl+Z`.

6. Gobernanza, Concurrencia en Tiempo Real y Sesiones Seguras

La sincronización colaborativa sobre WebSockets STOMP (< 50 ms de latencia) combinada con el esquema de candados optimistas (`element_locks`) demostró alta efectividad en la prevención de colisiones de edición simultánea en entornos multiusuario. La arquitectura de seguridad stateless con tokens JWT Bearer alojados exclusivamente en la memoria RAM volátil del navegador (`sessionStorage`), el hashing BCrypt (costo 12) y el blindaje anti-autodegradación administrativa garantizan un estándar de gobernanza empresarial inviolable.

7. Verificación Integral y Certificación Operativa

La madurez del software quedó formalmente certificada mediante la superación exitosa de los 117 tests automatizados de backend en Spring Boot y la ejecución satisfactoria del 100% de las 90 acciones de prueba descritas en las fichas formales CP-CU01 a CP-CU13 del Plan de Pruebas, certificando la estabilidad, rendimiento y seguridad de la plataforma para su despliegue operativo.

RECOMENDACIÓN

Con el objetivo de orientar la evolución sistemática, escalabilidad y consolidación de la plataforma CASE Tool en futuros ciclos de desarrollo e implementaciones empresariales a gran escala, se formulan las siguientes recomendaciones de ingeniería:

1. Ingeniería Inversa y Sincronización Bidireccional (Round-Trip Engineering)

Se recomienda enriquecer el subsistema de interoperabilidad incorporando capacidades de ingeniería inversa que permitan inspeccionar bases de datos relacionales existentes (mediante introspección del catálogo `information_schema` de PostgreSQL, MySQL u Oracle) o analizar repositorios de código fuente Java preexistentes (mediante parsers de código como JavaParser). Esto posibilitaría reconstruir automáticamente los diagramas de clases UML en el lienzo interactivo y sincronizar bidireccionalmente cualquier modificación efectuada en el código fuente con el modelo visual (*Round-Trip Engineering*).

2. Expansión del Modelado UML a Diagramas de Comportamiento Dinámico

Actualmente la plataforma destaca por su modelado estructural de clases UML 2.5. Se sugiere expandir las capacidades del editor para dar soporte a diagramas dinámicos de comportamiento, principalmente **Diagramas de Secuencia** y **Diagramas de Actividad / Máquinas de Estado**. Esta evolución permitirá que el motor de generación MDA (Model-Driven Architecture) no solo sintetice la estructura de las clases y sus métodos en la capa `service`, sino que también genere de forma automática la lógica algorítmica, validaciones y flujos de control de los casos de uso.

3. Desacoplamiento y Escalabilidad Horizontal del Broker WebSocket

Para escenarios de despliegue con concurrencia masiva (cientos de sesiones simultáneas y miles de colaboradores co-diseñando en paralelo), se recomienda desacoplar el broker STOMP en memoria que opera actualmente en Spring Boot y transicionar hacia un **Message Broker Distribuido externo (como RabbitMQ o Clúster Redis Pub/Sub)**. Dicha arquitectura permitirá la ejecución de múltiples instancias del backend balanceadas por carga (mediante NGINX o HAProxy), sincronizando los eventos de las salas colaborativas y gestionando la expiración de candados optimistas con políticas de TTL sin pérdida de estado.

4. Integración Continua de Modelos con Ecosistemas Git (GitOps para Modelos)

Se plantea la integración directa de la herramienta CASE con repositorios Git remotos (GitHub, GitLab, Bitbucket). Mediante el almacenamiento de los diagramas en formatos declarativos versionables, se facilitará que los arquitectos de software creen ramas de

trabajo (*branches*), realicen *commits* atómicos de sus diagramas, soliciten revisiones de diseño mediante *Pull Requests* y resuelvan visualmente conflictos semánticos entre versiones concurrentes del modelo.

5. Generación de Pipelines CI/CD y Empaquetado en Contenedores

Para cerrar el ciclo de vida DevOps, se aconseja actualizar las plantillas de generación FreeMarker para que el archivo ZIP descargable incluya automáticamente la infraestructura como código: un archivo `Dockerfile` multi-etapa optimizado, un manifiesto `docker-compose.yml` con la base de datos PostgreSQL preconfigurada, y flujos de trabajo de GitHub Actions (`.github/workflows/ci.yml`) para compilar, probar y publicar las imágenes de contenedor de manera desatendida.

6. Compilación Nativa Ahead-of-Time (AOT) con GraalVM Native Image

Se recomienda configurar perfiles Maven de compilación nativa en los proyectos Spring Boot generados. El soporte para GraalVM AOT permitirá que los microservicios generados compilen hacia binarios ejecutables nativos para el sistema operativo destino, reduciendo los tiempos de arranque de segundos a milisegundos (< 50 ms) y disminuyendo la huella de memoria RAM a menos de 60 MB, haciéndolos ideales para arquitecturas *serverless* y entornos Kubernetes de alta densidad.

7. Arquitectura de Plugins Extensible para Nuevos Stacks Tecnológicos

Se sugiere diseñar una API modular de extensiones que permita desacoplar los generadores de código del núcleo del editor. Mediante esta arquitectura orientada a plugins, la comunidad de ingenieros podrá aportar fácilmente nuevos compiladores para otros lenguajes y frameworks populares (tales como TypeScript/NestJS, Python/FastAPI, C#/.NET Core 8, o frameworks frontend como Angular y React), ampliando significativamente el alcance y versatilidad de la plataforma CASE.

BIBLIOGRAFÍA

La siguiente bibliografía reúne las fuentes científicas, estándares internacionales, manuales formales de ingeniería de software y documentación técnica de referencia utilizados en el diseño, desarrollo, validación y certificación de la plataforma CASE Tool UML:

1. Estándares Internacionales y Especificaciones Formales

1. **Object Management Group (OMG).** (2015). *OMG Unified Modeling Language (OMG UML), Version 2.5*. OMG Document Number: formal/2015-03-01. Recuperado de <https://www.omg.org/spec/UML/2.5/>
2. **Object Management Group (OMG).** (2005). *MOF 2.0 / XMI Mapping Specification, Version 2.1*. OMG Document Number: formal/2005-09-01. Recuperado de <https://www.omg.org/spec/XMI/2.1/>
3. **Object Management Group (OMG).** (2014). *Model Driven Architecture (MDA) Guide rev. 2.0*. OMG Document: ormsc/14-06-01. Recuperado de <https://www.omg.org/mda/>
4. **Internet Engineering Task Force (IETF).** (2011). *The WebSocket Protocol*. RFC 6455. <https://doi.org/10.17487/RFC6455>
5. **Internet Engineering Task Force (IETF).** (2015). *JSON Web Token (JWT)*. RFC 7519. <https://doi.org/10.17487/RFC7519>
6. **OASIS Standard.** (2012). *Simple Text Oriented Messaging Protocol (STOMP), Version 1.2*. Recuperado de <https://stomp.github.io/stomp-specification-1.2.html>
7. **National Institute of Standards and Technology (NIST).** (2020). *Digital Identity Guidelines: Authentication and Lifecycle Management*. NIST Special Publication 800-63B. <https://doi.org/10.6028/NIST.SP.800-63b>

2. Metodología, Arquitectura y Fundamentos de Ingeniería de Software

8. **Booch, G., Rumbaugh, J., & Jacobson, I.** (2006). *El Lenguaje Unificado de Modelado (UML)* (2.^a ed.). Addison-Wesley / Pearson Educación.
9. **Jacobson, I., Booch, G., & Rumbaugh, J.** (2000). *El Proceso Unificado de Desarrollo de Software (PUDS)*. Addison-Wesley Iberoamericana.
10. **Fowler, M.** (2003). *UML Gota a Gota* (3.^a ed.). Pearson Educación.
11. **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
12. **Martin, R. C.** (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
13. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
14. **Pressman, R. S., & Maxim, B. R.** (2020). *Software Engineering: A Practitioner's Approach* (9.^a ed.). McGraw-Hill Education.
15. **Sommerville, I.** (2016). *Software Engineering* (10.^a ed.). Pearson Education.

3. Teoría de Bases de Datos y Normalización Relacional

16. **Codd, E. F.** (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
17. **Codd, E. F.** (1972). Further normalization of the data base relational model. En R. Rustin (Ed.), *Data Base Systems* (pp. 33–64). Prentice-Hall.
18. **Date, C. J.** (2004). *An Introduction to Database Systems* (8.^a ed.). Addison-Wesley.
19. **Elmasri, R., & Navathe, S. B.** (2016). *Fundamentals of Database Systems* (7.^a ed.). Pearson.
20. **PostgreSQL Global Development Group.** (2024). *PostgreSQL 17.0 Documentation: The World's Most Advanced Open Source Relational Database*. Recuperado de <https://www.postgresql.org/docs/17/>

4. Frameworks, Lenguajes y Ecosistemas de Implementación

21. **Spring Boot Team.** (2024). *Spring Boot Reference Documentation (Version 4.x / 3.3.x)*. VMware Tanzu / Broadcom. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>
22. **Oracle Corporation.** (2024). *Java Platform, Standard Edition 21 (Java SE 21) Language Specification*. Recuperado de <https://docs.oracle.com/javase/specs/>
23. **FreeMarker Team.** (2024). *Apache FreeMarker Manual (Version 2.3.33)*. Apache Software Foundation. Recuperado de <https://freemarker.apache.org/docs/>
24. **Apache Software Foundation.** (2024). *Apache POI: The Java API for Microsoft Documents (XLSX Engine)*. Recuperado de <https://poi.apache.org/>
25. **xyflow.** (2024). *React Flow v12: The Library for Interactive Node-Based UIs*. Recuperado de <https://reactflow.dev/>
26. **Postman Inc.** (2024). *Postman Collection v2.1.0 Format Specification and Automation Engine*. Recuperado de <https://schema.postman.com/>
27. **Supabase Inc.** (2024). *Supabase Documentation: Cloud PostgreSQL, Auth and Realtime*. Recuperado de <https://supabase.com/docs>

5. Inteligencia Artificial, Visión Computacional y Modelos de Lenguaje

28. **Google DeepMind.** (2024). *Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context*. Google LLC. Recuperado de https://storage.googleapis.com/deepmind-media/gemini/gemini_v1_5_report.pdf
29. **Groq Inc.** (2024). *Groq LPU Inference Engine Documentation: Ultra-low Latency Llama 3.3 70B*. Recuperado de <https://console.groq.com/docs/>
30. **Meta AI.** (2024). *The Llama 3 Herd of Models*. Meta Platforms Inc. Recuperado de <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
31. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I.** (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008

URL Y QR

URL A GITHUB

https://github.com/marcolop450/Diagrama_de_Datos_Community_SW1.git

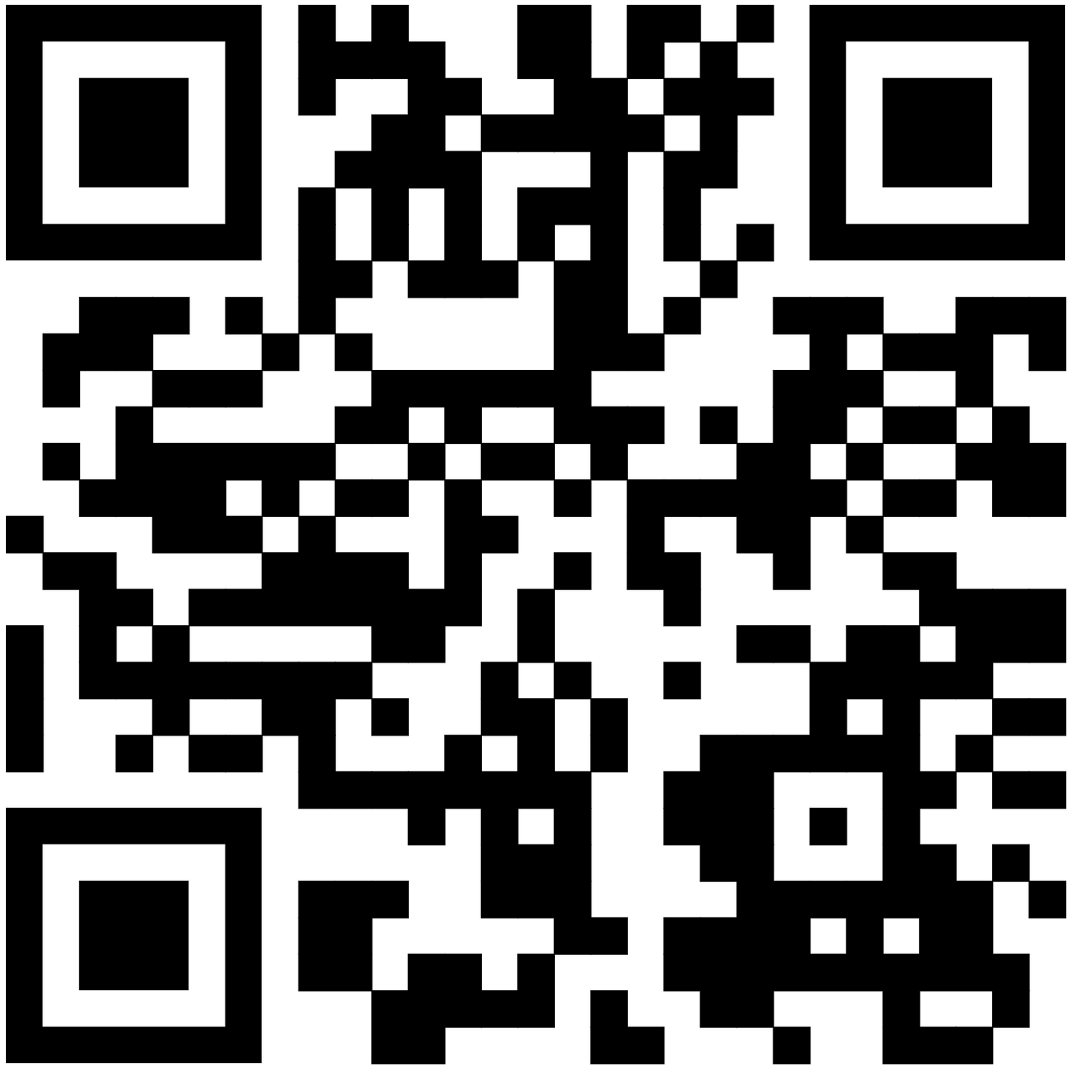
QR A GITHUB



URL EN LA NUBE

<https://marcolop-sw1-case.duckdns.org/>

QR A LA NUBE



Manual de Usuario Oficial por Roles de Operación

El sistema define tres roles humanos de interacción bien diferenciados, cada uno con privilegios, accesos y responsabilidades específicas dentro del flujo de trabajo de la herramienta CASE:

1. Guía de Operación para Super Administrador (SUPER_ADMIN)

El Super Administrador es la máxima autoridad de gobernanza, seguridad y auditoría técnica de la plataforma.

Paso 1: Autenticación Segura y Acceso a la Consola

1. Ingrese a la URL base de la aplicación y navegue a /login.
2. Introduzca sus credenciales maestras (ej. admin@sw1.com y contraseña protegida).
3. Presione "Iniciar Sesión". El sistema validará el hash con BCrypt (costo 12) y emitirá un token JWT Bearer almacenado estrictamente en la memoria volátil del navegador (sessionStorage).
4. Al autenticarse, el sistema lo dirigirá automáticamente a la consola de gobernanza ejecutiva /dashboard.

Paso 2: Gobernanza de Cuentas y Control de Roles (RBAC)

1. En la barra lateral izquierda (Sidebar), haga clic en la opción "Gobernanza de Usuarios" (/admin/users).
2. Visualice la tabla consolidada que lista todos los ingenieros y colaboradores registrados con sus metadatos (ID, Nombre, Email, Rol actual, Estado y Fecha de alta).
3. Modificación de Rol:
 - En la fila del usuario deseado, localice la columna "Rol".
 - Abra el menú desplegable y seleccione la nueva categoría (SUPER_ADMIN, ARQUITECTO, COLABORADOR).
 - El sistema persistirá el cambio de inmediato mediante una petición transaccional PUT /api/admin/users/{id}.
4. Conmutación de Estado Operativo:
 - Para suspender temporalmente el acceso de un usuario, desactive el switch interactivo "Estado" (pasando a Inactivo).
 - Si el usuario intentara realizar cualquier acción, el interceptor de seguridad rechazará sus solicitudes con código HTTP 403 Forbidden.
5. Blindaje Anti-Autodegradación:
 - Si el Super Administrador intenta suspender su propia cuenta o degradar su rol a Colaborador, el sistema interceptará la acción tanto en el cliente como en el backend (validarAntiAutodegradacion()), mostrando un toast de advertencia: "Operación denegada: El Administrador no puede degradar sus propios privilegios".

Paso 3: Auditoría Forense Global e Inmutable

1. Seleccione la opción "Auditoría del Sistema" (/admin/audit) en el Sidebar.
2. Observe el registro paginado de eventos de seguridad y mutaciones del sistema alimentado desde la tabla audit_logs.
3. Aplicación de Filtros Multicriterio:
 - Filtre eventos por severidad (INFO, WARN, ERROR).

- Filtre por rango de fechas (últimas 24 horas, última semana, último mes o intervalo personalizado).
 - Filtre por usuario específico o por dirección IP de origen.
4. Exportación Formal de Auditoría:
- Presione el botón "Exportar a Excel (.xlsx)" para descargar el informe binario estructurado con estilos y cabeceras mediante Apache POI.
 - Alternativamente, use los botones "CSV" o "JSON" para exportaciones ligeras orientadas a sistemas SIEM externos.

Paso 4: Supervisión de Proyectos y Restauración desde Papelera

1. Navegue a "Supervisión de Proyectos" (/admin/projects).
2. En esta consola, el Super Administrador puede visualizar todos los proyectos modelados en la plataforma, tanto los activos como aquellos marcados como borrados lógicamente (is_deleted = true).
3. Restauración de Proyectos:
 - Si un arquitecto eliminó por error un proyecto, el Super Administrador puede presionar el botón "Restaurar Proyecto", conmutando is_deleted = false y reintegrándolo de inmediato al workspace del propietario.
4. Regla Inviolable de Privilegios:
 - El Super Administrador NO dispone de botones de eliminación ni de purga física para proyectos ajenos. El backend bloqueará con IllegalArgumentException cualquier intento anómalo de borrado proveniente de este rol.

2. Guía de Operación para Arquitecto de Software (ARQUITECTO)

El Arquitecto de Software es el diseñador principal, responsable de concebir los modelos UML 2.5, validar la normalización de datos, liderar sesiones concurrentes como Host y generar el código fuente de producción.

Paso 1: Gestión de Espacios de Trabajo y Clonación Profunda

1. Inicie sesión y acceda a la vista "Mis Proyectos" (/projects).
2. Creación desde Cero o Plantilla:
 - Presione el botón "Nuevo Modelo".
 - Ingrese el nombre del modelo, descripción y seleccione si desea iniciar en un lienzo en blanco o instanciar una plantilla de dominio preconfigurada (Sistema Académico Universitario, Comercio Electrónico / Facturación, Gestión Hospitalaria).
3. Clonación Profunda (Deep Copy):
 - Para crear una variante o explorar una arquitectura alternativa sin comprometer el modelo original, presione "Clonar Proyecto" en la tarjeta del modelo.
 - El sistema ejecutará un Deep Copy transaccional en PostgreSQL 17: generará un nuevo UUID de proyecto, versionado v1.0.0 (Copia), duplicará todas las clases con nuevos identificadores y remapeará los handles cardinales de todas las relaciones asociativas.
4. Versionado Semántico y Tags:
 - Asigne etiquetas (tags) temáticas (ej. Microservicios, Producción, Finanzas) y controle el versionado semántico desde la cabecera de la tarjeta.

Paso 2: Modelado Visual Canónico OMG UML 2.5 en el Lienzo

1. Haga clic sobre la tarjeta de un proyecto para abrir el editor interactivo /editor/{id}.
2. Creación y Anatomía de Clases:
 - Arrastre el componente "Clase UML" desde la barra lateral izquierda o haga doble clic en el lienzo.
 - En el panel de propiedades (lateral derecho), configure:
 - Nombre de la Clase: Nomenclatura CamelCase (ej. Estudiante).
 - Estereotipo: Seleccione si aplica («entity», «boundary», «control», «abstract»).
 - Atributos: Presione "+ Agregar Atributo". Especifique visibilidad (+ público, - privado, # protegido, ~ paquete), nombre del campo y tipo de dato seleccionable desde el catálogo (Java: Long, String, LocalDate, Double, Boolean / SQL equivalente).
 - Clave Primaria: Marque la casilla "Es PK" para destacar el identificador único con el badge {PK} en color ámbar/dorado.
 - Métodos / Operaciones: Presione "+ Agregar Método", indicando visibilidad, nombre, parámetros formales tipados y tipo de retorno (ej. + matricular(materialId: Long): Boolean).
3. Trazado de Relaciones con 4 Handles Magnéticos:
 - Cada clase expone cuatro puertos de conexión cardinales orientados magnéticamente: Superior (top), Inferior (bottom), Izquierdo (left) y Derecho (right).

- Haga clic en un handle de la clase origen y arrastre el cursor hasta el handle deseado de la clase destino.
- En el menú contextual que emerge, elija el tipo semántico de relación:
 - Asociación Simple / Dirigida: Flecha simple con definición de roles y multiplicidades (1, 0..1, 1..*, *).
 - Agregación: Terminación con diamante blanco/hueco en el extremo contenedor.
 - Composición: Terminación con diamante negro/relleno (vida dependiente).
 - Generalización (Herencia): Flecha triangular hueca en el extremo del ancestro.
 - Realización / Dependencia: Línea punteada con flecha de especificación.
- 4. Prevención Algorítmica de Ciclos de Herencia (DFS):
 - Si intenta trazar una relación de generalización que provoque un ciclo circular (ej. Clase A hereda de B, B hereda de C, y C intenta heredar de A), el motor de grafos ejecutará una búsqueda en profundidad (DFS) en tiempo real, cancelará la arista y emitirá un toast preventivo: "Ciclo de herencia no permitido por el estándar UML 2.5".
- 5. Reversibilidad y Persistencia:
 - Presione Ctrl+Z para deshacer cualquier acción previa y Ctrl+Y para rehacerla.
 - Presione el botón "Guardar Diagrama" en la cabecera para sincronizar el grafo de forma atómica en el backend (POST /api/diagrams/{id}/sync).

Paso 3: Certificación de Normalización Relacional (1NF a 3NF)

1. En la barra de herramientas lateral del editor, presione "Certificar Normalización".
2. Se desplegará el modal NormalizationModal ejecutando automáticamente la auditoría heurística del motor basado en las reglas de Codd y TOM:
 - 1NF (Primera Forma Normal): Verifica que toda entidad contenga al menos un identificador primario atómico {PK}.
 - 2NF (Segunda Forma Normal): Verifica que en toda relación 1 a muchos (1..*), la entidad del extremo "muchos" posea la clave foránea dependiente.
 - 3NF (Tercera Forma Normal): Detecta relaciones directas muchos a muchos (*..*) y exige su resolución mediante una entidad intermedia asociativa para eliminar dependencias transitivas.
3. Observe el indicador visual semaforizado (0% a 100%).
4. Utilice las acciones asistidas (como "Inyectar {PK} automáticas" o "Descomponer N..M").
5. Al alcanzar el 100% de cumplimiento, presione "Expedir Certificación Formal". El backend registrará una certificación digital inmutable en la tabla normalization_certifications con un hash SHA-256 único verificable.

Paso 4: Generación Automática de Código (CASE / MDA)

1. Generación de Backend Spring Boot en 4 Capas:
 - Presione el icono "Generar Backend Spring Boot" en la Toolbar lateral.
 - Especifique el paquete base (ej. com.sw1.academico) y el nombre del artefacto.
 - Presione "Generar y Descargar (.ZIP)".

- El motor Apache FreeMarker compilará en segundos las entidades JPA (@Entity, relaciones @ManyToOne/@OneToMany), repositorios Spring Data JPA, servicios transaccionales @Service y controladores REST @RestController, empaquetándolos en un ZIP que incluye Maven Wrapper (mvnw), pom.xml y documentación Swagger.
- 2. Generación de Esquema DDL SQL para PostgreSQL 17:
 - Presione "Generar DDL SQL & Postman".
 - En la pestaña "Script DDL", copie o descargue el archivo .sql. Incluye sentencias CREATE TABLE, tipos nativos optimizados, claves primarias IDENTITY, restricciones de integridad referencial ON DELETE CASCADE e índices B-Tree para claves foráneas.
- 3. Generación de Suite Postman v2.1.0:
 - En la pestaña "Suite Postman", presione "Descargar Colección JSON".
 - Importe el archivo descargado en Postman Desktop para disponer inmediatamente de carpetas organizadas por entidad con peticiones GET, POST, PUT y DELETE, datos mock fuertemente tipados y pruebas automáticas pm.test que validan códigos de estado y respuestas JSON.

Paso 5: Exportación e Importación Multiformato

1. Exportaciones Soportadas:
 - OMG XMI 2.1: Descarga el archivo XML estándar interoperable con herramientas de modelado de escritorio (Enterprise Architect, StarUML, ArchiTec).
 - Captura PNG UHD: Genera una imagen en ultra alta definición con multiplicadores de escala (1x, 2x, 3x) para documentación técnica sin pixelación.
 - Memoria Técnica PDF: Compila un reporte ejecutivo formal con metadatos del proyecto, diagrama incrustado y diccionario técnico.
 - Libro de Datos Excel (.xlsx): Genera un archivo con 3 hojas estructuradas (Entidades, Atributos con tipos JPA/SQL, y Relaciones).
2. Importación XMI 2.1:
 - En el menú principal, seleccione "Importar XMI".
 - Seleccione un archivo XML exportado desde StarUML o ArchiTec.
 - El parser defensivo anti-XXE procesará las etiquetas, y el motor LayoutEngineUtil distribuirá automáticamente las clases en capas jerárquicas ordenadas sobre el lienzo.

Paso 6: Asistencia Asistida por Inteligencia Artificial Multimodal

1. Modelado por Voz (PLN):
 - Haga clic en el botón de micrófono en el dock flotante inferior.
 - Dicte una instrucción en lenguaje natural (ej. "Crear la tabla Factura con id, fecha y monto total conectado a la tabla Cliente").
 - El Circuit Breaker procesará la instrucción y aplicará las mutaciones sobre el lienzo en tiempo real, respetando la regla 1NF y permitiendo revertir con Ctrl+Z.
2. Digitalización Óptica de Pizarras Físicas:
 - Abra la herramienta "Digitalizar Pizarra con Visión".
 - Capture una fotografía desde su cámara web o cargue un archivo JPG/PNG de un diagrama dibujado a mano alzada.

- El modelo de visión artificial identificará los rectángulos, compartimentos y flechas, inyectará preventivamente {PK} id: Long y aplicará el auto-layout para posicionar el diagrama listo para editar.

Paso 7: Anfitrión de Sesión Colaborativa Concurrente (Host)

1. En la cabecera del editor, presione "Iniciar Sesión en Vivo".
2. El sistema creará una sala en `collaboration_sessions` y mostrará un código alfanumérico único de 6 caracteres (ej. SW1-842).
3. Comparta el código con su equipo de colaboradores.
4. Durante la sesión, usted podrá observar los cursores remotos de los participantes en tiempo real, visualizar los candados de edición que adquieren sobre las clases y, si un colaborador genera conductas anómalas, podrá expulsarlo de la sala (Kick) con un solo clic, revocando su conexión WebSocket de inmediato.

3. Guía de Operación para Colaborador (COLABORADOR)

El Colaborador es el co-diseñador técnico que participa en sesiones de diseño concurrente en tiempo real.

Paso 1: Acceso a la Plataforma y Unión a una Sesión

1. Inicie sesión con sus credenciales de Colaborador en /login.
2. En la vista principal /projects, presione el botón "Unirse a Sesión en Vivo".
3. Introduzca el código alfanumérico de 6 caracteres suministrado por el Arquitecto Host (ej. SW1-842) y presione "Conectar".
4. El cliente establecerá un canal WebSocket STOMP sobre SockJS con latencia garantizada sub-50ms, renderizando el lienzo sincronizado con el modelo activo.

Paso 2: Co-Diseño Concurrente y Candados Optimistas (element_locks)

1. Visualización de Participantes:
 - En la esquina superior derecha, visualice los avatares de todos los ingenieros conectados a la sala junto con el indicador de latencia en milisegundos.
2. Adquisición de Candados al Editar:
 - Cuando usted hace clic sobre una clase o relación para editarla, su cliente solicita automáticamente un candado optimista (SEND /app/room/{code}/lock).
 - El backend aprueba la solicitud y notifica a los demás participantes, mostrando un badge de candado con su avatar sobre el elemento en las pantallas de ellos. Esto impide que otro colaborador sobreescriba su trabajo al mismo tiempo.
3. Liberación del Candado:
 - Al terminar de modificar el elemento o deseleccionarlo, el candado se libera automáticamente (release-lock), permitiendo que otros miembros del equipo continúen el diseño.
4. Movimiento Reactivo de Nodos:
 - Al arrastrar un nodo por el lienzo, las coordenadas relativas se transmiten instantáneamente, actualizando la posición de la clase en las pantallas de todo el equipo de forma fluida.

Paso 3: Comunicación por Chat Flotante Aislado

1. En la esquina inferior derecha, abra el widget de Chat en Vivo.
2. Escriba mensajes o notas técnicas para coordinar las decisiones de diseño con el Arquitecto y los demás colaboradores.
3. Aislamiento Total de Eventos de Teclado:
 - El campo de texto del chat cuenta con un aislador de eventos del DOM (stopPropagation). Usted puede utilizar la barra espaciadora, teclas de borrado o combinaciones rápidas dentro del chat sin temor a activar atajos del lienzo o eliminar clases accidentalmente.

Paso 4: Trazabilidad, Historial y Salida de la Sala

1. En la barra de herramientas, abra el cajón de "Historial de Cambios" para inspeccionar la línea de tiempo de modificaciones que ha sufrido el proyecto.
2. Para abandonar la sesión colaborativa, presione "Desconectar de la Sala" en la cabecera. El sistema notificará a los demás participantes y lo redirigirá limpiamente a su espacio de trabajo personal.

4. Guía de Atajos de Teclado y Controles Rápidos del Lienzo

Atajo Combinación	/	Acción Realizada en el Editor	Ámbito de Aplicación
Ctrl + Z		Deshace la última mutación registrada en la pila reactiva de Zustand.	Lienzo General
Ctrl + Y		Rehace la última acción deshecha restaurando el estado previo del grafo.	Lienzo General
Supr / Delete		Elimina la clase o relación actualmente seleccionada (previa confirmación).	Elementos Seleccionados
Espacio Arrastre	+	Activa el modo de paneo manual para desplazarse libremente por el lienzo.	Navegación Espacial
Rueda del Ratón		Zoom dinámico de acercamiento o alejamiento (rango soportado: 10% a 200%).	Navegación Espacial
Escape		Cancela la conexión en curso, cierra modales flotantes o deselectiona nodos.	Global
Ctrl + S		Dispara el guardado y sincronización atómica con el backend PostgreSQL 17.	Persistencia
Shift + Clic		Selección múltiple de clases para reubicación o alineación en bloque.	Edición Múltiple

ANEXOS TÉCNICOS FORMALES

Glosario de Términos Técnicos y Metodológicos

- **AST (Abstract Syntax Tree):** Representación en árbol de la estructura sintáctica abstracta del código fuente o modelo de clases, utilizada por el compilador FreeMarker para sintetizar artefactos de software.
- **BCrypt:** Función de derivación de claves criptográficas basada en el cifrado Blowfish, parametrizada con factor de trabajo (costo 12) para proteger contraseñas contra ataques de fuerza bruta.
- **Circuit Breaker (Disyuntor de Resiliencia):** Patrón de diseño de software de estabilidad que intercepta peticiones a servicios externos (APIs de IA) y, ante fallos recurrentes o agotamiento de cuotas, conmuta de forma transparente hacia proveedores de respaldo sin colapsar el sistema.
- **Deep Copy (Clonación Profunda Transaccional):** Operación de duplicación integral que crea nuevas instancias en base de datos para una entidad padre y todas sus entidades dependientes (clases, atributos, métodos y relaciones), reasignando identificadores UUID únicos y remapeando claves foráneas de manera atómica.
- **DFS (Depth-First Search):** Algoritmo de búsqueda en profundidad utilizado en el editor para recorrer el grafo de relaciones de generalización y detectar anticipadamente la existencia de ciclos cerrados.
- **FreeMarker:** Motor de procesamiento de plantillas Java utilizado para la generación guiada por modelos (MDD), inyectando metadatos UML sobre estructuras de código Java 21 y SQL DDL.
- **JWT (JSON Web Token):** Estándar abierto (RFC 7519) para la transmisión compacta y segura de afirmaciones de identidad entre partes, firmado digitalmente mediante algoritmo criptográfico HMAC-SHA256.
- **MDA / MDD (Model-Driven Architecture / Development):** Paradigma de ingeniería de software promovido por la OMG que coloca a los modelos conceptuales en el centro del ciclo de vida, derivando implementaciones de software mediante transformaciones automatizadas.
- **Normalización Relacional (1NF, 2NF, 3NF):** Proceso formal fundamentado en la teoría de Edgar F. Codd y la Teoría de Objetos Modelados (TOM) para organizar campos y tablas, minimizando la redundancia de datos y las anomalías de actualización.
- **OMG UML 2.5:** Especificación formal estándar de la Object Management Group que define la semántica, notación y metamodelo para la especificación, visualización y documentación de sistemas de software orientados a objetos.
- **OMG XMI 2.1 (XML Metadata Interchange):** Estándar de intercambio de modelos basado en XML que permite la transferencia bidireccional de diagramas UML entre herramientas CASE heterogéneas.
- **PUDS (Proceso Unificado de Desarrollo de Software):** Metodología iterativa, incremental, dirigida por casos de uso y centrada en la arquitectura, estructurada en fases (Inicio, Elaboración, Construcción, Transición) y flujos de trabajo de ingeniería.

- **RBAC (Role-Based Access Control):** Mecanismo de control de acceso en el que los privilegios del sistema se asignan a roles abstractos (**SUPER_ADMIN**, **ARQUITECTO**, **COLABORADOR**) y no directamente a identidades de usuario.
- **SockJS / STOMP:** Protocolo de mensajería orientada a texto simple que opera sobre WebSockets, proveyendo abstracción de suscripción a tópicos (**/topic/...**), colas de mensajes y transporte bidireccional con latencias inferiores a 50 ms.
- **XXE (XML External Entity Injection):** Vulnerabilidad de seguridad de aplicaciones web (CWE-611) que permite a un atacante interferir en el procesamiento de datos XML de la aplicación mediante la declaración de entidades externas maliciosas.